PHYSICS SIMULATIONS FOR FURTHER MATH TUTOR

Centre Number: 46121

Candidate number: 9685

COURSE CODE: 7517/C

Name: Danylo Poluektov

Contents

1. Analysis 3
1.1 Problem Background 3
1.2 Description of the current system 4
1.3 Prospective user(s) 4
1.4 Proposed solution 4
1.5 Process diagram 5
1.6 Data flow diagram 6
1.7 Acceptable limitations 7
1.8 Objectives 7
1.9 Evidence of analysis 8
2. Design 11
2.1 Overview 11
2.2 General system for all simulations 11
2.3 IPSO Chart 14
2.4 Menu 15
6. Appendix2.4.1 Settings 16
2.4.2 Launcher.py 16
2.4.3 Main_menu.py 16
2.4.4 Config_manager.py 16
2.5 Simulations 17
2.5.1 Objects.py 18
2.5.2 Main.py 18
2.5.3 Ui.py 19
2.6 Friction Simulation 19
2.6.1 Friction.py 20
2.6.2 Object.py 20
2.6.3 Main.py 20
2.7 Rebound simulation 20
2.7.1 Vectors.py 22
2.7.2 Collisions.py 23
2.7.3 Main.py 23
2.8 String simulation 23
2.8.1 Static objects 24
2.8.2 Camera 24
2.8.3 Strings.py 24

3. Technical Solution 25
3.1 Key algorithms of the parts of the program 26
3.1 Menu 26
3.1.1 Config_manager.py 27
3.2.2 Main_menu.py 29
3.2 General Simulations structure 30
3.2.1 Objects.py 30
3.2.2 Ui.py 33
3.2.3 Main.py 36
3.3 Friction simulation 41
3.3.1 Friction.py 41
3.3.2 Main.py 42
3.4 Rebound simulation 44
3.4.1 Collisions.py 44
3.4.2 Main.py 47
3.5 String simulation 48
3.5.1 Strings.py 48
3.5.2 Main.py 50
4. Testing 51
4.1 Menu 51
4.2 Friction Simulation 52
4.3 Rebound simulation 53
4.4 String simulation 53
5. Evaluation 55
5.1 Feedback 56
5.2 Issues with the solution 56
5.3 Things I would like to do differently 56
6. Appendix 57
6.0 File organization 57
8.1 Menu 59
6.1.1 Launcher.py 59
6.1.2 main_menu.py 59
6.1.3 config_manager.py 66
6.2 Simulations 68
6.2.0 Mymouse.py 68
6.2.1 Friction simulation 68

6.2.1.1 Friction.py 68
6.2.1.2 Main.py 70
6.2.1.3 Objects.py 75
6.2.1.4 Ui.py 78
7.2.2 Rebound simulation 82
6.2.2.1 Collisions.py 82
6.2.2.2 Main.py 87
6.2.2.3 Objects.py 93
6.2.2.4 Ui.py 97
6.2.2.5 Vectors.py 101
6.2.3 String simulation 102
6.2.3.1 Strings.py 102
6.2.3.2 Main.py 104
6.2.3.3 Objects.py 110
6.2.3.4 Ui.py 113
6.2.4 Config 117
6.2.4.1 Config.json 117
7. References 118

1. Analysis

1.1 Problem Background

Being a Further Math student is already challenging by itself, but it is even more difficult without any visual and interactive representations of questions. I was studying Further Math in Year 12 and I know how hard it can be to get your head around the question, so I decided to help my Further Math tutor with representing questions in an interactable and dynamic way.

After our first dialogue I understood that: Mr Robinson, Further Mathematics tutor that requires a physics simulation program to improve his teaching methods and make students life easier. The suggested program will be used as an interactive visualization tool, allowing students to observe and manipulate physical systems in real time, therefore helping to build a bridge between mathematical equations and real life comprehension.

Therefore, program will be based on these topics of Further Math Mechanics topics:

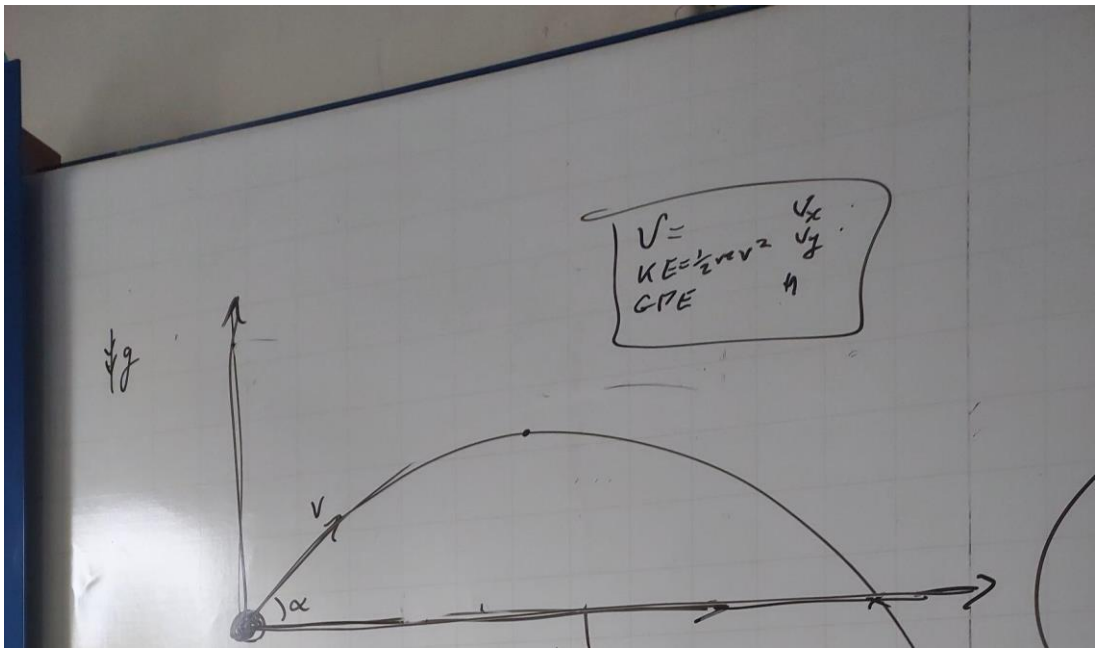
-Momentum and collisions

-Work, Energy and Power

As these parts of the Mechanics course are the hardest to understand.

1.2 Description of the current system

Now Mr Robinson draws diagrams for questions on the board that help students understand what happens in a complex Further Math Mechanics question. This method represents static diagrams that don't encourage students to connect mathematical equations with physical phenomena. This method lacks impact that is needed to convey real life physics. I believe that if students can get proper real time simulation, they will start getting better grades and understanding of perplexing tasks.



1.3 Prospective user(s)

Mr Robinson is the main prospective user as I am developing the program specifically for him. Additionally, he walks through the complex A Level topics and gives them proper Further Math questions.

But as this program is also for students, meaning it could be shared with them by Mr Robinson via School Website (Firefly) or Email, so they can use it at home and have a little play around the physics.

Main use case is displaying on the big board in the centre of the class via projector or on a computer screen.

1.4 Proposed solution

Program that starts on the user`s computer that produces interactable physics engine. All school computers have only 60 Hz refresh rate so the lowest limit for the frame rate should be 60 per second, as this program should be optimised enough to sustain 60 FPS at a school computer. As it will be used at school at school computer the program needs to be preinstalled and the all the libraries should be default, therefore I will be using Python with PyGame.

From interview with Mr Robinson, I understood that I must create three main simulations as these are the most problematic questions for students. As well as smooth graphics he wants to be able to stop, slow down and restart the simulation and change inputs for the objects and environments.

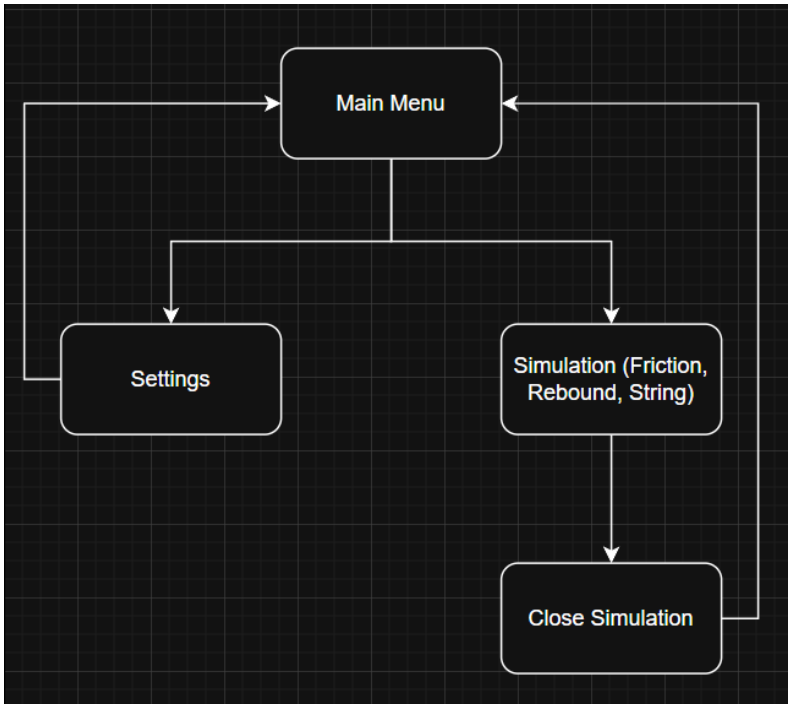
Simulations:

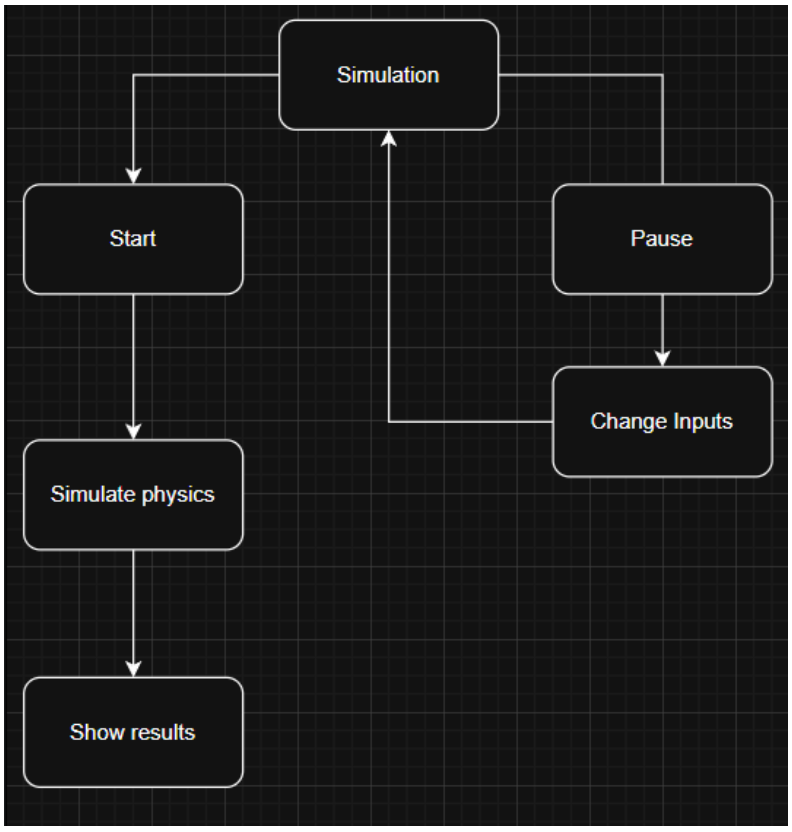
1. Friction Simulation

2. Rebounds Simulation
3. String Simulation

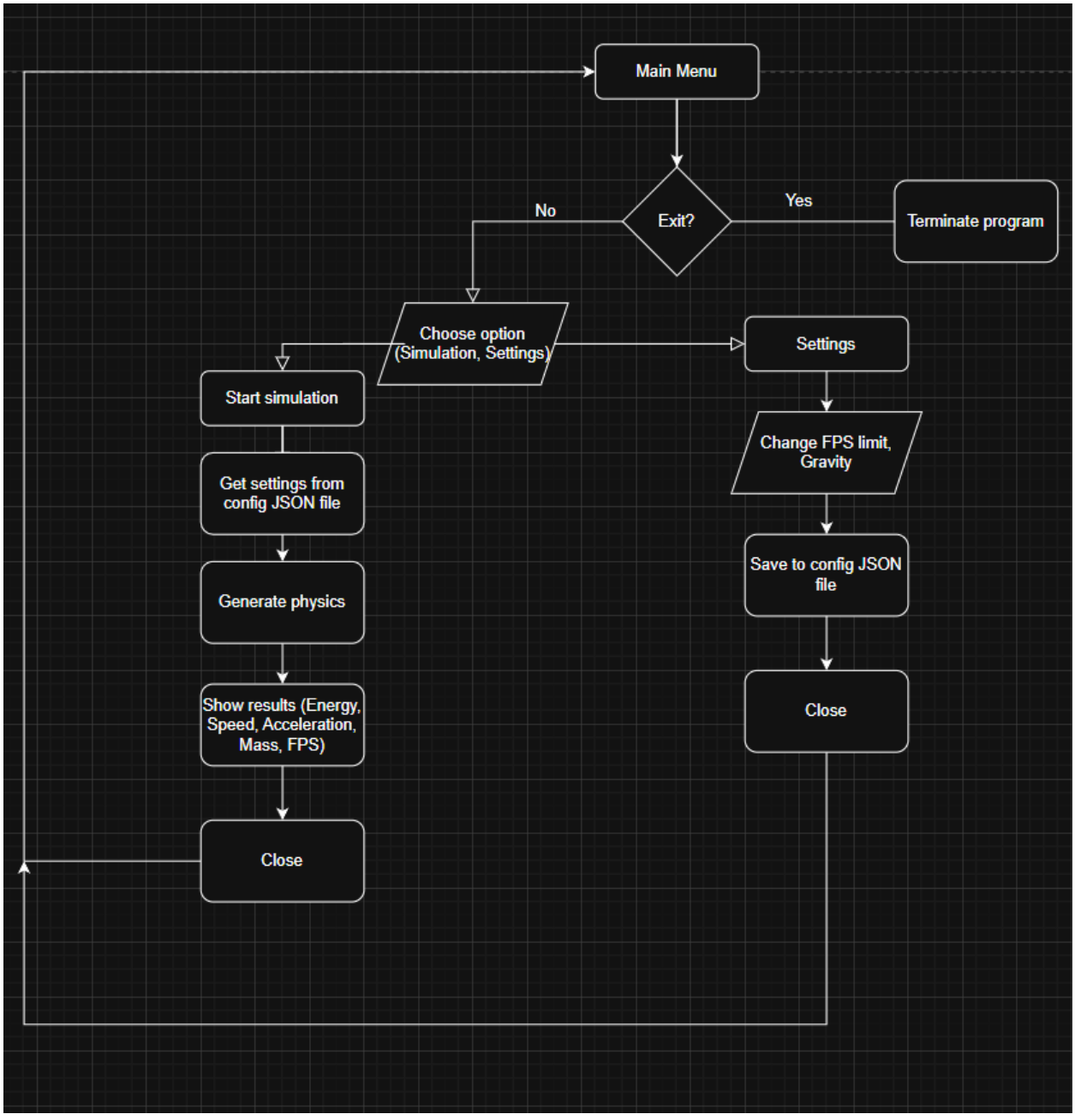
All these simulations must have understandable UI for both teacher and students.

1.5 Process diagram





1.6 Data flow diagram



1.7 Acceptable limitations

After discussion with Mr Robinson, we came up with these limitations, as this is essentially a simple educational program for A Level Further Math Mechanics course:

Simple pygame graphics

Moving objects are circles with centre of mass in the centre of the circle

Program window resolution is 1280x720 pixels

No air resistance

No static friction for friction simulation

No mass for string for string simulation

Maximum of three particles for rebound simulation

Maximum of three strings for string simulation

1.8 Objectives

1. Create correct render of events and mathematically correct physics that are rendered smoothly and calculated correctly regardless of FPS using separate PHYSICS_FPS.

2. Create intuitive UI so the user can change most of the inputs and interact with the simulation.

    1. Adding objects

    2. Deleting objects

    3. Changing parameters of the objects

    4. Change parameters of the simulation

    5. Change pace of the time

3. Create three separate simulations with proper rendering and physics calculations with separating visual and physical calculation

    1. Friction simulation

        1. Create moving of the object along the plane that is rotated by a certain angle with acceleration calculated using components of the weight of the object and friction coefficient the is decelerating the object.

    2. Rebounds simulation

1. Create moving of the objects (max of 3) in the space with correct rebounds from surfaces and other moving objects with changeable elasticity of collisions.

2. Bonus objective is to create a rebounds between objects.

3. String simulation

1. Create moving of the object that is hanging from the string with changeable natural length and modulus of elasticity via calculating the tension that is produced by the string.

4. Correct calculation of gravity, friction, speed, acceleration, energy and etc. via SUVAT equations and other math formulas.

5. Displaying program calculations results and simulation state with time scaling:

1. Friction simulation: velocity, acceleration of the object, plane angle, friction coefficient and mass of the object

2. Rebound simulation: for every moving object it`s velocity, energy, height from the ground and mass

3. String simulation: for every string it`s tension, extension, elastic potential energy; for moving object velocity, acceleration and mass

6. Create pause/resume, time scale feature for every simulation.

7. Create saveable parameters that are shared for all three simulations: FPS, PHYSICS_FPS, GRAVITY.

8. Trail of objects for rebound and string simulation so the user can see how object was travelling in space.

9. Create correct separation of logical and physical calculations of positions of objects in space.

10. Create clear visuals so the user understands what happens to the objects on the screen.

11. Create menu from where user can change FPS and gravity for the simulation and start simulations.

1.9 Evidence of analysis

Log of my interview with Mr Robinson:

Q: How accurate does the physics need to be?

Ans: “It needs to match A-Level specification exactly - so coefficient of restitution formula, μF for friction, Hooke's Law. But we can simplify air resistance isn't in Further Maths spec, so ignore it. Focus on getting the core mechanics perfect rather than adding lots of elements.”

Q: What about usability for students?

Ans: “Keep it simple. Big buttons, clear labels. Instructions and settings menu on the side of the screen.”

Q: What features would make this useful in your classroom?

Ans: "Ability to pause and step through frame-by-frame would be essential - I need to explain what's happening at key moments. Also, displaying the actual force vectors and energy values on screen, not just showing the motion. Students need to see the numbers matching their calculations."

Q: What specific topics do your students struggle with most in mechanics?

Ans: “Definitely friction on slopes and collision problems. Students can plug numbers into formulas but don't understand why objects slide at certain angles or how energy is lost in collisions. They also struggle with understanding where object is travelling so trail and arrow with direction of motion would help enormously.”

Books:

See references 3. And 4.

Mathematical formulas that I will use to calculate physics include:

Energy calculation formulas:
KE = 0.5mv^2 where m=mass, v=velocity

GPE = mgh where m=mass, g=gravity, h=height

EPE = 0.5kx^2 where k=spring constant, x=extension of the string

F=ma where m=mass, a=acceleration

Conservation of linear momentum

Tension in string F=kx

$$\sum m_i(v_i)_1 = \sum m_i(v_i)_2$$

SUVAT equations

$$v = u + at$$
$$v^2 = u^2 + 2as$$
$$s = ut + \frac{1}{2}at^2$$
$$s = vt - \frac{1}{2}at^2$$
$$s = \frac{1}{2}(u+v)t$$

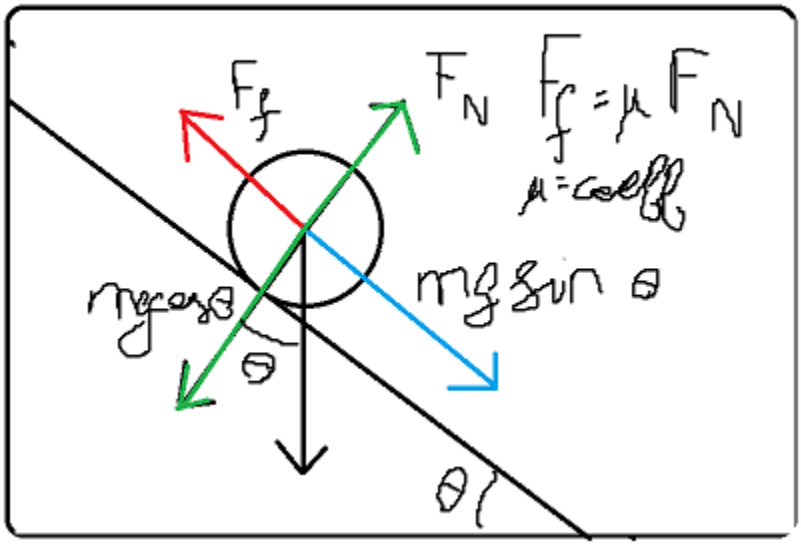
s – displacement
u – initial velocity
v – final velocity
a – acceleration
t - time

Dot product and vector calculations formulas like scalar multiply or vector addition and etc.

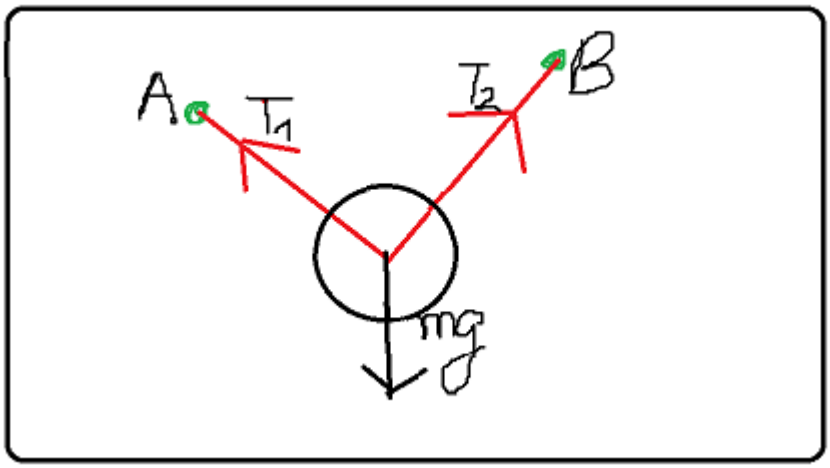
Prototypes of physics calculation

ss

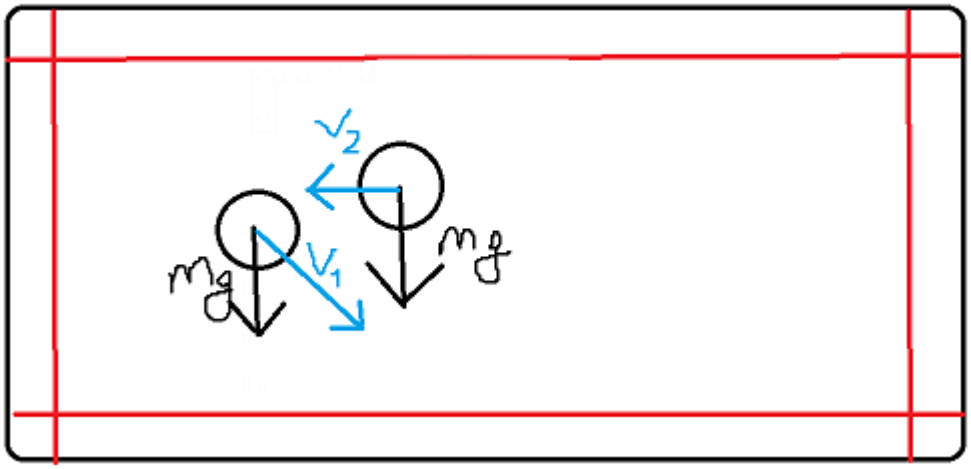
friction_sim



string_sim



rebound_sim



2. Design

2.1 Overview

Main objective is to create three separate physics simulations with comprehensive UI and changeable parameters and a menu where user can choose to alter general settings of the simulations, like FPS, Gravity or enter one of the three simulations.

In simulation HUD should be displayed in the left side of the screen.

When right clicking the object, settings window should appear in the left side of the screen where user can change parameters of the object and can see the instructions for the specific simulation.

2.2 General system for all simulations

HUD: Should be displayed in the left side of the screen with all relevant information for a specific simulation.

Force driven moving for objects: Object->Force->Acceleration->Velocity->Position

Fixed timestep: timestep is calculated using delta time to maintain the proper work of the simulation and correct physics calculations.

Coordinate System is divided into logical (meters) and visual (pixels) to maintain proper calculations of physics as math formulas mostly use meters for correct calculations.

Frames Per Second are dived into Render FPS and Physics FPS this is an industrial approach that is used by physics engines universally.

Direction vector – an arrow from the centre of the object that show where object is travelling

Moving object – a circular particle with a given radius that is affected by environment

Static object – a rectangular object with a given width and height

The simulations will be created in the way they could explain these questions. (see references 3.)

148

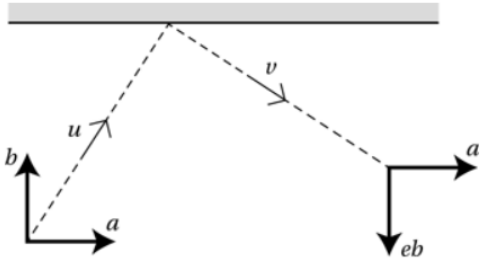
Section 2: Oblique impacts and the impulse–momentum triangle

So far, you have considered direct impacts with objects moving along their line of centres or hitting walls at right angles. You will now look at **oblique impacts**.

Consider a particle travelling with velocity $\mathbf{u} = a\mathbf{i} + b\mathbf{j}$ that hits a wall that is parallel to the vector $\mathbf{i}$.

Since the impulse the wall exerts on the particle is perpendicular to the wall, the $\mathbf{i}$-component of the velocity will not be affected by the collision with the wall.

You can find the $\mathbf{j}$-component of the velocity after the collision using Newton's Experimental Law, exactly as you did in Chapter 3 where the particle hit the wall perpendicularly.



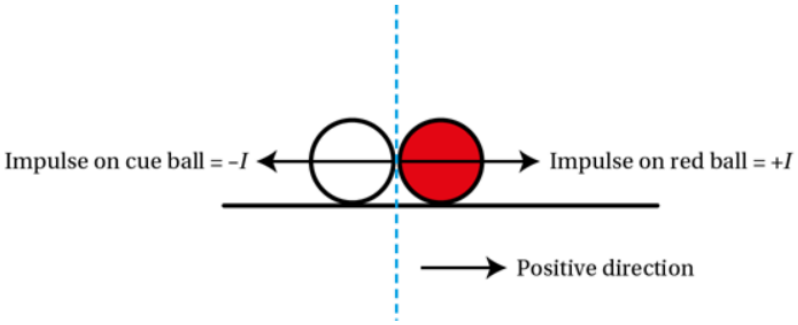
Key point 6.1

When a particle moving with speed $\mathbf{u} = a\mathbf{i} + b\mathbf{j}$ collides with a smooth, flat surface parallel to the vector $\mathbf{i}$, it rebounds with velocity $\mathbf{v} = a\mathbf{i} - e b\mathbf{j}$, where e is the coefficient of restitution between the particle and the surface.

57

Section 2: Collisions and the principle of conservation of momentum

In a game of snooker, if the cue ball hits a stationary red ball it exerts a force on it, causing it to move along the line of the collision. From Newton's third law, there is an equal and opposite reaction on the cue ball. As the time for which the collision force acts is the same for both balls – the time for which the two balls are in contact – then the impulse, I, $(\text{force} \times \text{time})$ on the two balls is also the same but in opposite directions.



The total impulse on the two balls along the line of the collision is $-I + I = 0$.

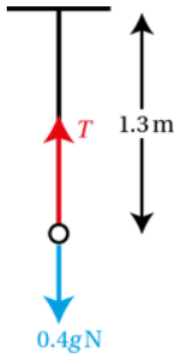
As the total impulse is zero then there is no change in the total momentum of the two balls. This means that linear momentum is conserved in a collision between two objects that are free to move. This is called **the principle of conservation of linear momentum**.

Consider two objects, A and B, of mass m_1 and m_2, moving with velocities u_1 and u_2 in a straight line. After 58

WORKED EXAMPLE 5.5

11

A light elastic string is attached to a fixed point and hangs vertically, in equilibrium, with an object of mass 400 grams attached to its lower end. The string has natural length $0.8\,\text{m}$ and the object is resting $1.3\,\text{m}$ below the point of suspension. Find the stiffness of the string as a multiple of g, the acceleration due to gravity. Hence find the modulus of elasticity of the string.



Draw a diagram showing the forces acting on the particle.

$400\,\text{grams} = 0.4\,\text{kg}$

Convert grams to kilograms.

$T - 0.4g = 0 \Rightarrow T = 0.4g$

Since the string is in equilibrium there is zero resultant force.

$T = kx \Rightarrow k = \frac{T}{x}$

Rearrange the formula for Hooke's law to make k the subject. x is the difference between the stretched and unstretched lengths of the string.

$k = \frac{0.4g}{0.5} \Rightarrow k = 0.8\,g\,\text{N m}^{-1}$

Leave your answer for k as a multiple of g, the acceleration due to gravity, as required.

$k = \frac{\lambda}{l} \Rightarrow \lambda = kl$

Convert from 'stiffness' to modulus of elasticity.

$\lambda = 0.8g \times 0.8\text{N}$

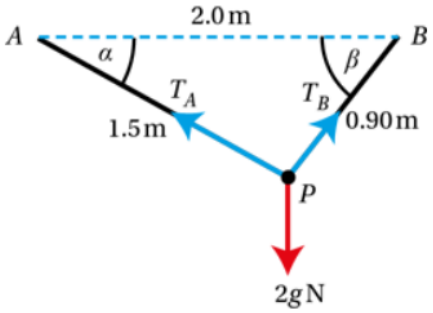
$\Rightarrow \lambda = 0.64g\,\text{N}$

Section 4: Problem solving involving work, energy and power

More complex problems may combine the work–energy principle and the principle of conservation of energy. You may be working with any of the propulsive or resistive forces you have met in Chapter 1 and this chapter.

WORKED EXAMPLE 5.16

An object, P, of mass $2.0\,\text{kg}$, is attached to the ends of two light elastic strings with the same natural length, $0.75\,\text{m}$, but different moduli of elasticity. One of the strings is attached to a point A and the other is attached to point B on the same horizontal level as A, such that the distance AB is $2.0\,\text{m}$. P hangs in equilibrium. The distance AP is $1.5\,\text{m}$ and BP is $0.90\,\text{m}$. Calculate the modulus of elasticity of each of the strings.



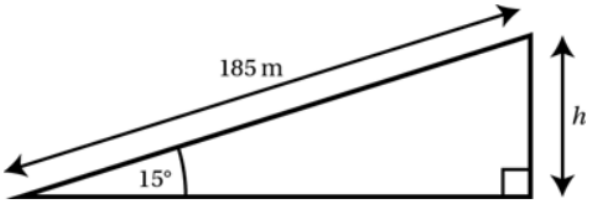
Draw a clearly labelled diagram.

$$\alpha = \cos^{-1}\frac{1.5^2 + 2^2 - 0.9^2}{2 \times 2 \times 1.5}$$
$$\approx 24.95^\circ$$
$$\beta = \cos^{-1}\frac{0.9^2 + 2^2 - 1.5^2}{\cdots}$$

Use the cosine rule to find angles $PAB\,(\alpha)$ and $PBA\,(\beta)$. Work to four significant figures at this stage.

WORKED EXAMPLE 5.13

Irina walks up a steady slope inclined at 15° to the horizontal. Irina's mass is $55\,\text{kg}$ and she walks 185 metres along the slope. Find the work Irina does against gravity as she walks up the slope, giving your answer in kilojoules (kJ).



$$\text{Work done} = \text{force} \times \cos\theta \times \text{distance}$$

Use the definition of work done by a force acting at an angle to the direction of motion.

$$\theta = 75^\circ$$

The slope makes an angle of 15° with the horizontal but 75° with the vertical.

$$\Rightarrow \text{work done} = 55g \times \cos 75^\circ \times 185$$
$$= 25.8\,\text{kJ}\ (3\,\text{s.f.})$$

Irina does work against gravity. Her weight is $55g$ newtons acting vertically downwards. The component of her weight parallel to the slope is $55g \times \cos 75^\circ$.

2.3 IPSO Chart

Simulation	Input	Process	Storage	Output
Friction	• Right-click (edit parameters)	• Decompose gravity into	• Ball object (position,	• Rendered plane and object

	• Left-click (Drag-and-drop objects) • Keyboard (F=pause, R=reset)	components • Calculate normal force N = mg cos(θ) • Determine friction F_friction = μN • Net force F_net = mg sin(θ) - F_friction • Update physics -> find new position • Update visual position	velocity, mass, radius, velocity) • Plane angle • Friction coefficient • Config file (JSON)	• Direction vector of moving object • HUD text (velocity, distance)
Rebound	• Right-click (edit parameters) • Left-click (Drag-and-drop objects) • Keyboard (F=pause, R=reset, A=add object, S=delete object)	• Detect circle-circle collisions • Detect circle-wall collisions • Calculate impulse J = -(1+e) v_rel_n / (1/m1 + 1/m2) • Update physics -> find new position • Update visual position	• Ball objects (position, velocity, mass, radius) • Wall objects (edges, elasticity) • Trail points (set of coordinates) • Elasticity coefficients	• Rendered balls and walls • Edge normals • Direction vector of moving object • HUD (energy, speed) • Trail points
String	• Right-click (edit parameters) • Left-click (Drag-and-drop objects) • Keyboard (F=pause, R=reset) • Mousewheel input for changing the zoom on to the simulation plane.	• Calculate extension • Calculate tension F = kx (if x > 0, else string is slack) • Calculate angle to anchor • Resolve tension into (F_x, F_y) • Update physics -> find new position • Update visual position	• Anchor position of the string (fixed) • Particle state (position, velocity) • String properties (Natural length, Elasticity Modulus) • Camera offset (pixels) • Trail points	• Rendered balls and strings • String line (red if extended, blue if slack) • Direction vector of moving object • Anchor (green dot) • HUD (extension, tension, period) • Trail points

2.4 Menu

I want to create a simple menu where you can choose simulation and change general settings of the simulations – FPS and Gravity.

6. Appendix2.4.1 Settings

FPS for simulations can be changed.

Gravity for simulations can be changed.

When settings are saved, program is restarted to ensure that settings are applied correctly.

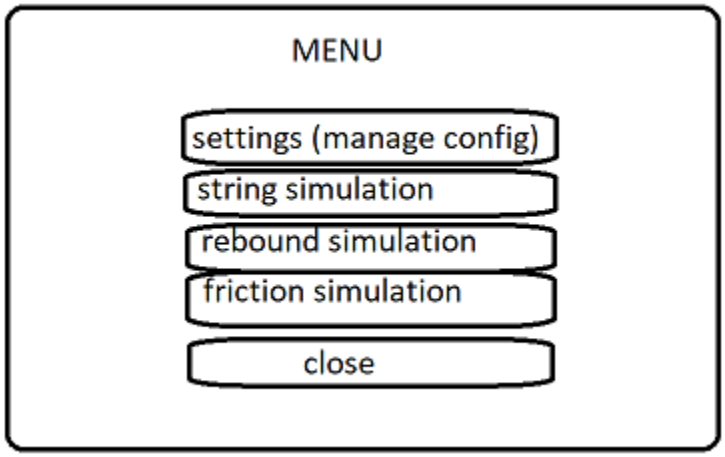
When simulation is chosen, menu closes. Every simulation should have a button to return to Menu.

Refresh rate for menu is 60 FPS.

2.4.2 Launcher.py

Core of the program that starts the menu from where the simulations and settings can be entered.

2.4.3 Main_menu.py



Has buttons of simulations, settings and exit from the program. Settings are opened in the left side of the menu where FPS and Gravity for simulations are entered, when user saves changes program is restarted via subprocesses to ensure that settings are applied correctly. Simulations are started via subprocesses to not mess up the pygame window logic.

Restart: subprocess.run([sys.executable] + sys.argv)

Simulation start: subprocess.run([sys.executable, str(simulation)]) where simulation is absolute path to file of the simulation.

2.4.4 Config_manager.py

Manages the config.json file that saves information about screen size, physics for simulations and colors in tuple(r,g,b). It loads, saves and exports data from config.json.

2.5 Simulations

All simulations are rendered with crucial parameter PIXELS_PER_METER to establish correct physics calculation to render on the screen.

X is horizontal, Y is vertical parameters of the simulation.

All simulations should be paused and resumed with F key.

All simulations should be reset with R key.

All simulations should have changeable time scale to slow down or speed up the simulation.

All simulations should have a settings window opened with right click on the object where user can change parameters and a HUD with all needed for that simulation information about moving object.

Friction simulation and Rebound simulation will calculate the position of the moving object using SUVAT equation: s=(v+0.5a*t)*t as the acceleration in those simulations is constant.

String simulation will calculate the position of the moving object using more general equations:

v=a*t, s=v*t

Time for position calculation in all simulations is delta time between frames (1/PHYSICS_FPS).

By default all simulations have moving object present with 0.25 meters radius.

Working areas:

friction simulation -> a plane where object slides

rebound simulation -> area between walls that are on the sides of the screen

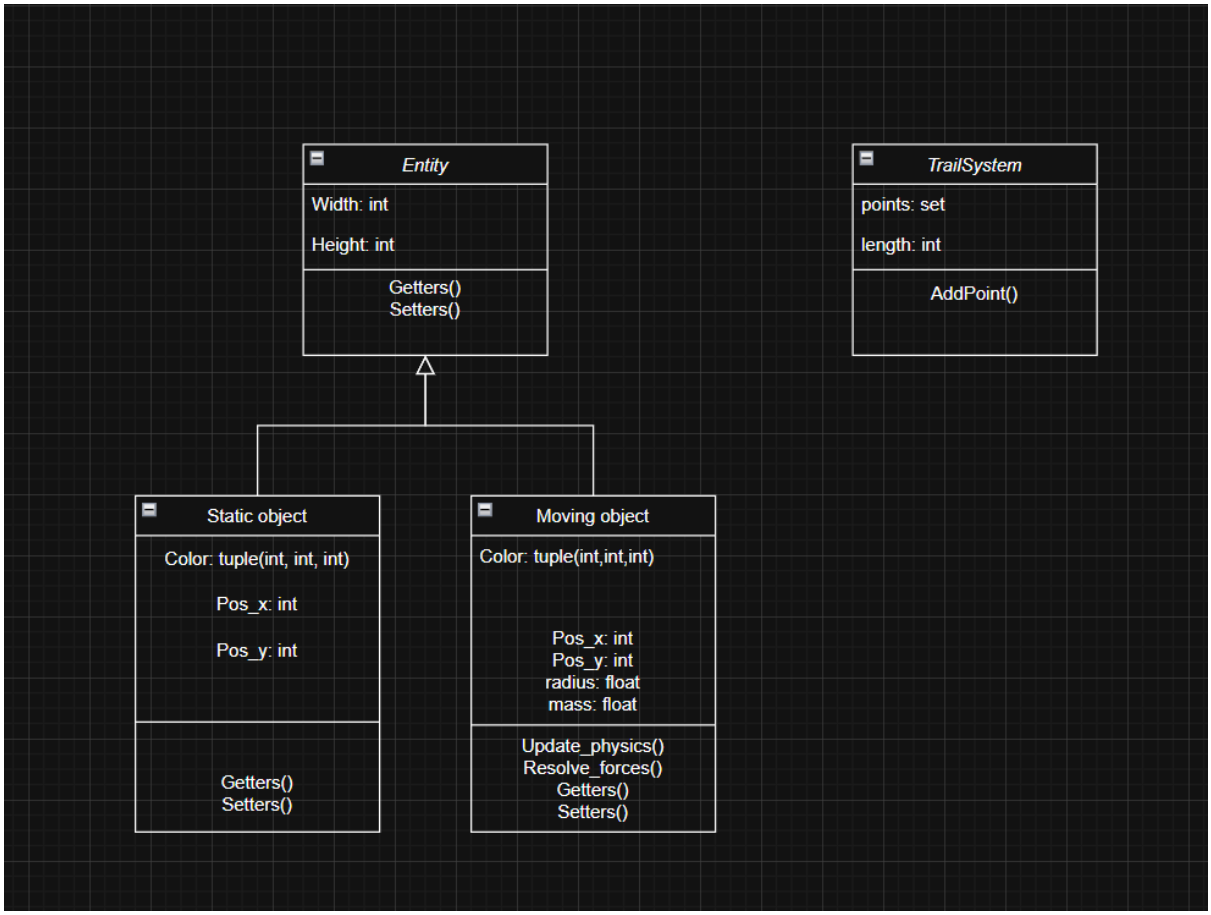
Trail of moving object in rebound and string simulations is created to keep track of movement of an object in space, it is processed by adding a point into set of points at every frame of the simulation.

In all simulations should be a exit button that exits the simulation and opens the main menu.

As I want to create drag-and-drop and settings window opening feature for particles (moving object and string) and strings they need to have a click_hitbox, which then can be used to process if user has clicked on an object. Click_hitbox will be a largest inscribed square into circle as moving object and string anchor are circles.

Left click -> move object. Right click -> open settings.

2.5.1 Objects.py



All moving objects in simulation have division of position into logical and actual on the screen. They have sync_to_visual methods, update_visual_position to synchronize the logical and actual position on the screen.

LOGICAL = ACTUAL/PIXELS_PER_METER

2.5.2 Main.py

In all simulations there will be a close button, default objects that are present and a main loop that handles the pygame window (drawing, refreshing).

Main loop has a default pygame event loop that processes user inputs and a physics loop that processes physics for the objects in the simulation.

Mymouse.py

A simple file that gives sensible names to pygame functions for mouse press and mouse position on the screen. Also it has a draggable_object that stores the object that was clicked by user to move it:

pygame.mouse.get_pressed(num_buttons=3)[0] where index is a button pressed – lmbn is 0, mwb is 1 and rmb is 2.

2.5.3 Ui.py

This file in all simulations is responsible for creating a window on the right side of the screen when user clicks on an object which parameters can be altered or if nothing was clicked it opens a settings menu for current simulation. It also provides necessary instructions for current simulation.

2.6 Friction Simulation

Friction simulation should have a plane with changeable angle of inclination, surface friction and a moving object that moves on the plane due to gravity with given velocity, acceleration (or deceleration if friction is greater than gravity component).

I am not going to simulate the direct contact with the surface, instead I will just simulate the direction of movement of an object above the rotated plane under it.

Plane can be turned from -90 to 90 degrees as only for these values' friction is appliable. Essentially if there is no normal reaction to the plane there will be no friction.

To ensure that object is always visible on a plane I will set it to the start of the plane when it reaches the end of the plane.

HUD:

- Current FPS
- Physics FPS
- Distance travelled
- Velocity X, Velocity Y
- Acceleration X, Acceleration Y
- Mass of the object
- Friction coefficient
- Angle of the plane

User inputs:

- Moving object: mass, velocity along the plane.
- Simulation: time scale, angle of the plane, coefficient of friction of the plane.

Physics:

Decompose weight of the object into components: parallel to the plane, perpendicular to the plane.

Find normal reaction -> weight_perpendicular*cos(angle)

Find force parallel to the plane that creates acceleration for the object downwards-> weight_parallel*sin(angle)

Find friction force parallel to the plane that creates deceleration for the object -> coeff_of_friction*normal_reaction

Friction force is only applied if the object is moving.

Object velocity is rounded to zero when it decreases to small values(<0.1) to prevent infinite movement.

2.6.1 Friction.py

Has friction_manager class that calculates the physics for the simulation.

2.6.2 Object.py

Static object has _compute_edges method to find corners and edges of the rotated rectangle

https://stackoverflow.com/questions/41898990/find-corners-of-a-rotated-rectangle-given-its-center-point-and-rotation

2.6.3 Main.py

Also has helper function to place object directly on the plane and functions to ensure that objects always stays on the plane.

Get_closest_point_on_line

Set_object_on_plane

Check_border_collision

Set_object_before_plane this function is called when object hits the border to be moved to opposite border.

2.7 Rebound simulation

Rebound simulation will have 4 walls at the sides of screen with an inputted elasticity coefficient by user; moving object that is affected by gravity and inputted horizontal and vertical velocity. Impulse of collisions of objects (maximum of 3) between each other will be calculated and applied to objects to simulate real life behaviour. Objects can be added and deleted by user.

HUD:

- Current FPS
- Physics FPS
- **For every object:**
  - Distance travelled
  - Velocity X, Velocity Y
  - Mass of the object
  - Height in meters from the ground
  - Kinetical Energy
  - Gravitational Potential Energy

User inputs:

- Moving object: mass, velocity x, velocity y, radius

- Simulation: time scale, mode (1D or 2D, 1D=no gravity), elasticity of borders, elasticity of collisions between moving objects

Physics:

Moving objects are affected by gravity (downwards acceleration).

Collisions between objects (circular particles)

Checking:

1. Check for collision: dist < r1 + r2 (dist = distance between centers of the circles)

Resolving:

1. Prevent duplicate resolution by tracking already resolved collisions
2. Calculate collisions normal: (c2-c1) / dist
3. Calculated relative velocity of objects along the normal: v_rel_along_normal =dot_product((v1 - v2) normal)
4. Calculate Impulse of the collision with velocity along the normal and elasticity coefficient: J = -(1 + e) * v_rel_along_normal / (1/m1 + 1/m2)
5. Apply impulse to both objects:

   New_v1 = v1 + (J/m1)*normal

   New_v2 = v2 + (J/m2)*normal

Collisions with surfaces:

Checking:

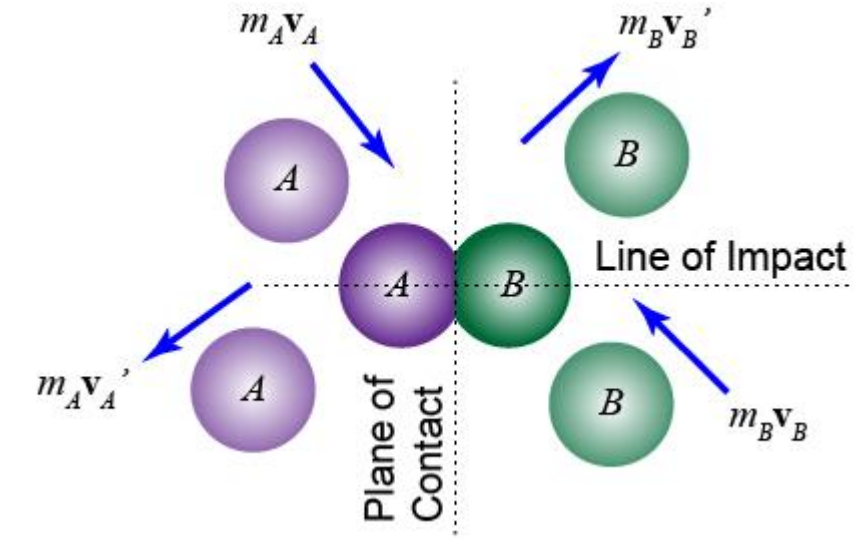
1. Check if the object has hit the wall via dot product of velocity and normal

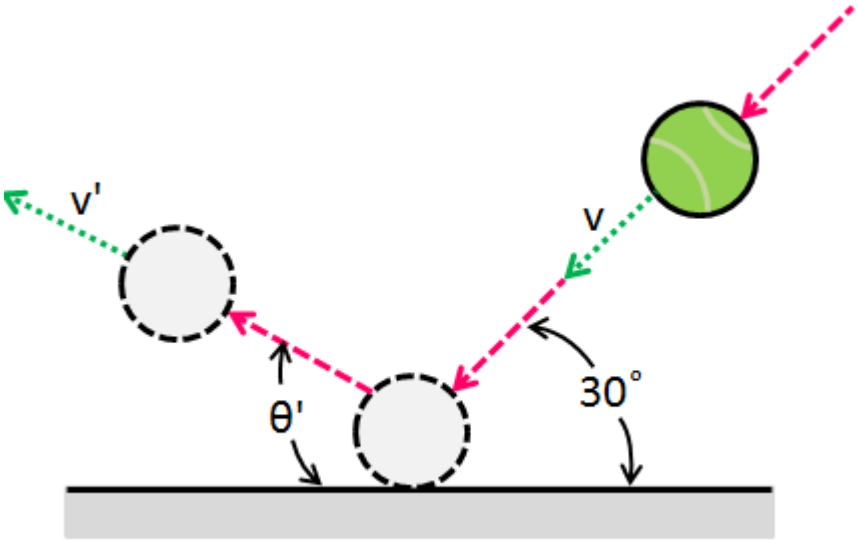
Resolving:

1. Calculate Impulse with normal of the wall and elasticity coefficient
2. Apply impulse to the moving object

References where I got the general idea of resolving forces for rebound simulation:

https://www.physicsforums.com/threads/calculating-impulse-due-to-rigid-body-collision-with-friction.186335/
https://www.damtp.cam.ac.uk/user/reh10/lectures/ia-dyn-chapter5.pdf
https://www.gamedev.net/forums/topic/465248-calculating-impulse-due-to-rigid-body-collision-with-friction/
https://en.wikipedia.org/wiki/Impulse_(physics)





Picture references:
See references 1. And 2.

2.7.1 Vectors.py

This file has necessary math operations to calculate vectors:

Dot_product

Vector_add add one vector to another

Vector_subtract subtracts one vector from another

Scalar_multiply multiplies each vector value by a value

Vector_length calculates length of the vector

Distance_between_points calculates distance between points

Normalize _vector normalizes given vector

2.7.2 Collisions.py

Have such functions as get_line_normal and distance_point_to_line.

For collisions it has methods for checking and resolving collisions:

Check_circle_collisions and check_wall_collisions

Resolve_circle_collisions and resolve_wall_collisions

2.7.3 Main.py

Physics loop that is in the main loop of main.py has 5 steps:

1. Check wall collisions

2. Apply gravity if mode is not 2D (1D=no gravity)

3. Update moving objects position

4. Check circle collisions (collisions of moving objects)

5. Check if the moving object is not going to rebound by calculating if the rebound is bigger than a threshold (2 centimetres) if not -> stop the object.

2.8 String simulation

String simulation will have a single moving object attached to a strings (maximum of 3). Tension is calculated using Hooke`s law and T=kx where x is extension in the string and k (modulus of elasticity divided by natural length) is spring constant. Tensions of the strings are combined, and then general force is resolved by moving object. String is assumed to have 0 mass so there is no need to simulate it`s movement. Strings can be added and deleted by user.

HUD:

- Current FPS
- Physics FPS
- **For moving object:**
  - Distance travelled
  - Velocity X, Velocity Y
  - Acceleration X, Acceleration Y
  - Mass of the object
- **For every string:**
  - Tension
  - Extension
    EPE (Elastic potential energy)

User inputs:

- Moving object: mass, velocity x, velocity y, radius
- Simulation: time scale
- String: natural length, elasticity modulus
- Mouse grab on right click: all objects are translated in reversed direction of user mouse movement.
- Mouse wheel: PIXELS_PER_METER is changed by 5, creating zoom feature.

Physics:

Moving object is attached to string and affected by gravity.

Tension in the string is calculated using Hooke`s law F=kx

x = extension of the string, calculated by taking distance between moving object and string anchor and subtracting natural length of the string: sqrt((x1-x2)**2 + (y1-y2)) – natural_length. If extension is negative then tension is 0 as string becomes slack.

K = spring constant and calculated by dividing modulus of elasticity by natural length of the string. All of these parameters are given by user.

2.8.1 Static objects

This simulation does not have any static objects as there is only moving object hanging from the strings.

2.8.2 Camera

It has new variables and logical that allows user to move the simulations by holding rmb (camera pan) and zoom out/zoom in into simulation by changing the PIXELS_PER_METER variable using the mouse wheel (+5 when scrolled up, -5 when scrolled down).

Positions of strings, trail and moving object are than adjusted by the position of the camera (camera_offset).

2.8.3 Strings.py

Class string that has anchor position coordinates, logical position of anchor coordinates, modulus of elasticity, natural length (length when string is slack), current extension, and a hitbox that is needed to check if the user has clicked on the string. It has relevant methods that allow the program to calculate the tension, extension of the string, and elastic potential energy. All of this information is then displayed for each string, so the user at any given moment could stop the simulation and see what values each string has. Strings can be position via drag-and-drop using lmb. Settings menu is opened by clicking rmb on the string. String can be added by clicking A (maximum of 3 strings) and deleted by clicking S on the string. The colour of the string becomes red when extension is bigger than zero.

3. Technical Solution

File Name	Purpose	Appendix Number
Config.json	This file stores the parameters for the simulation.	6.2.4.1
Launcher.py	This file launches the main menu, therefore the root file of the project	6.1.1
Main_menu.py	This file constructs the main menu of the program where user can press buttons to either: Start simulation Open settings and change them Exit program When settings are saved program is restarted	6.1.2
Config_manager.py	This file manages the config by storing and exporting values from config.json file	6.1.3
Mymouse.py	This file has draggable object that allows, the program to keep track of which object the user is holding	6.2.0
Friction simulation		6.2.1
Friction.py	This file has friction manager that calculates physics for the friction between plane and a moving object	6.2.1.1
Main.py	This file has main pygame loop where user inputs and physics of the object on the screen are processed, also has initial instances for the default objects in the simulation	6.2.1.2
Objects.py	This file has objects classes for simulation.	6.2.1.3
Ui.py	This file the UI that is called when user clicks rmb on the object.	6.2.1.4
Rebound Simulation		6.2.2
Collisions.py	This file has functions for calculation the collisions between object-object and object-wall	6.2.2.1
Vectors.py	This file has functions for mathematical operations with	6.2.2.5

	vectors that is used by collisions.py	
Ui.py	This file the UI that is called when user clicks rmb on the object.	6.2.2.4
Main.py	This file has main pygame loop where user inputs and physics of the object on the screen are processed, also has initial instances for the default objects in the simulation	6.2.2.2
Objects.py	This file has objects classes for simulation.	6.2.2.3
String simulation		6.2.3
Strings.py		6.2.3.1
Main.py	This file has main pygame loop where user inputs and physics of the object on the screen are processed, also has initial instances for the default objects in the simulation	6.2.3.2
Objects.py	This file has objects classes for simulation.	6.2.3.3
Ui.py	This file the UI that is called when user clicks rmb on the object.	6.2.3.4

3.1 Key algorithms of the parts of the program

To see whole code for this parts of the program use the table above.

3.1 Menu

This part of the program creates a menu from where user can start simulation and change settings for the simulation.

3.1.1 Config_manager.py

Has a fixed screen size and it loads the config from the json file.

```
class ConfigManager:
    def __init__(self, config_path=None):
        if config_path is None:
            # default to config.json in parent directory
            current_file = pathlib.Path(__file__).resolve()
            config_path = current_file.parent / "config.json"

        self.config_path = pathlib.Path(config_path)
        self._config = {}
        self.load()

        # fixed screen size
        self.SCREEN_WIDTH = 1280
        self.SCREEN_HEIGHT = 720
```

Saving values to json file

```
def save(self):
    try:
        with open(self.config_path, 'w') as f:
            json.dump(self._config, f, indent=2)
        print(f"Configuration saved to {self.config_path}")
        return True
    except Exception as e:
        print(f"Error saving config: {e}")
        return False
```

Loading values from json file

```
def load(self):
    try:
        with open(self.config_path, 'r') as f:
            self._config = json.load(f)
        print(f"Configuration loaded from {self.config_path}")
    except FileNotFoundError:
        print(f"Config file not found: {self.config_path}")
        print("Using default configuration")
        self._create_default_config()
    except json.JSONDecodeError as e:
        print(f"Error parsing config file: {e}")
        print("Using default configuration")
        self._create_default_config()
```

Methods to get/set values of the json file

```
    def get(self, *keys, default=None):
        value = self._config
        for key in keys:
            if isinstance(value, dict) and key in value:
                value = value[key]
            else:
                return default
        return value

    def set(self, *keys, value):
        if len(keys) == 0:
            return False

        config = self._config
        for key in keys[:-1]:
            if key not in config:
                config[key] = {}
            config = config[key]

        config[keys[-1]] = value
        return True
```

All getters are done this way, it allows the value to be accessed like attribute

```python
@property
def FPS(self):
    return self.get("display", "fps")
```

All setters are done this way, it allows the value to be set like attribute

```python
@PHYSICS_FPS.setter
def PHYSICS_FPS(self, value):
    self.set("display", "physics_fps", value=value)
```

3.2.2 Main_menu.py

Restarting the menu and simulations start are done via subprocess to not mess up the pygame window logic.

```python
def main():
    while True:
        menu = MainMenu()
        restart, simulation = menu.run()

        if restart:
            # restart with new settings
            print("Restarting application with new settings...")
            try:
                subprocess.run([sys.executable] + sys.argv)
            except Exception as e:
                print(f"Error restarting application: {e}")
            break
        elif simulation:
            # run simulation
            print(f"Launching simulation: {simulation}")
            try:
                subprocess.run([sys.executable, str(simulation)])
            except Exception as e:
                print(f"Error running simulation: {e}")
            # continue loop to restart menu after simulation
        else:
            # normal exit
            break
```

3.2 General Simulations structure

3.2.1 Objects.py

Entity class with height, width being in pixels for pygame

```
class Entity(pygame.sprite.Sprite):

    def __init__(self, height, width):
        pygame.sprite.Sprite.__init__(self)
        self.height = height * cfg.PIXELS_PER_METER
        self.width = width * cfg.PIXELS_PER_METER

    def get_height(self): return self.height
    def get_width(self):  return self.width
```

Static object for rebound and friction simulations.

```
class StaticObject(Entity):
    def __init__(self, height, width, color, pos_x, pos_y, elasticity):
        super().__init__(height, width)

        self.color = color
        self.elasticity = elasticity
        self.pos_x = pos_x
        self.pos_y = pos_y

        # needed for pygames sprite groups
        self.image = pygame.Surface((int(self.width), int(self.height)), pygame.SRCALPHA)
        self.image.fill((0, 0, 0, 0))
        self.rect = self.image.get_rect(center=(pos_x, pos_y))

        self.edges = self._compute_edges()
```

Moving object for every simulation where physics are updated

```
class MovingObject(Entity):
    def __init__(self, radius, color, pos_x, pos_y, elasticity, mass):
        diameter = radius * 2 # meters
        super().__init__(diameter, diameter) # height, width that are stored in pixels

        self.mass   = mass
        self.radius = radius # metres

        self.pos_x = pos_x
        self.pos_y = pos_y

        self.color = color
        self.elasticity = elasticity

        self.velocity_x = 0.0
        self.velocity_y = 0.0
        self.acceleration_x = 0.0
        self.acceleration_y = 0.0

        self.gforce = [0.0, mass * cfg.GRAVITY]


        # image needed for pygame sprites
        self.image = pygame.Surface((int(self.width), int(self.height)), pygame.SRCALPHA)
        self.image.fill((0, 0, 0, 0))
        self.rect = self.image.get_rect(center=(pos_x, pos_y))

        # logical position in meters (used by the physics integrator)
        self.logical_x = pos_x / cfg.PIXELS_PER_METER
        self.logical_y = pos_y / cfg.PIXELS_PER_METER

        self.click_hitbox = self._make_hitbox()

        self.distance_travelled = 0.0

    # -- Hitbox --
```

Friction and rebound simulations (constant acceleration)

```python
# -- Physics --

def resolve_forces(self, *forces):
    fx, fy = 0.0, 0.0
    for f in forces:
        fx += f[0]
        fy += f[1]

    self.acceleration_x = fx / self.mass
    self.acceleration_y = fy / self.mass

def update_physics(self, dt):
    prev_x, prev_y  = self.logical_x, self.logical_y

    self.logical_x += (self.velocity_x + 0.5 * self.acceleration_x * dt) * dt # suvat equation: s=v*t+0.5*a*t^2 -> s=(v+0.5at)t
    self.logical_y += (self.velocity_y + 0.5 * self.acceleration_y * dt) * dt

    self.velocity_x += self.acceleration_x * dt
    self.velocity_y += self.acceleration_y * dt

    self.pos_x = self.logical_x * cfg.PIXELS_PER_METER
    self.pos_y = self.logical_y * cfg.PIXELS_PER_METER

    dx = self.logical_x - prev_x # meters
    dy = self.logical_y - prev_y # meters
    self.distance_travelled += math.sqrt(dx * dx + dy * dy) # meters

    self.set_center((self.pos_x, self.pos_y))
```

String simulations (non-constant acceleration)

```python
    # -- Physics --

    def resolve_forces(self, *forces):
        fx, fy = 0.0, 0.0
        for f in forces:
            fx += f[0]
            fy += f[1]
        self.acceleration_x = fx / self.mass
        self.acceleration_y = fy / self.mass

    def update_physics(self, dt):
        # -- Simple physics --

        # previous position to calculate change
        prev_x, prev_y = self.logical_x, self.logical_y

        self.velocity_x += self.acceleration_x * dt
        self.velocity_y += self.acceleration_y * dt

        self.logical_x += (self.velocity_x) * dt
        self.logical_y += (self.velocity_y) * dt

        self.pos_x = self.logical_x * cfg.PIXELS_PER_METER + self.camera_offset_x # + camera offset to maintain consistency of the physics
        self.pos_y = self.logical_y * cfg.PIXELS_PER_METER + self.camera_offset_y # + camera offset to maintain consistency of the physics

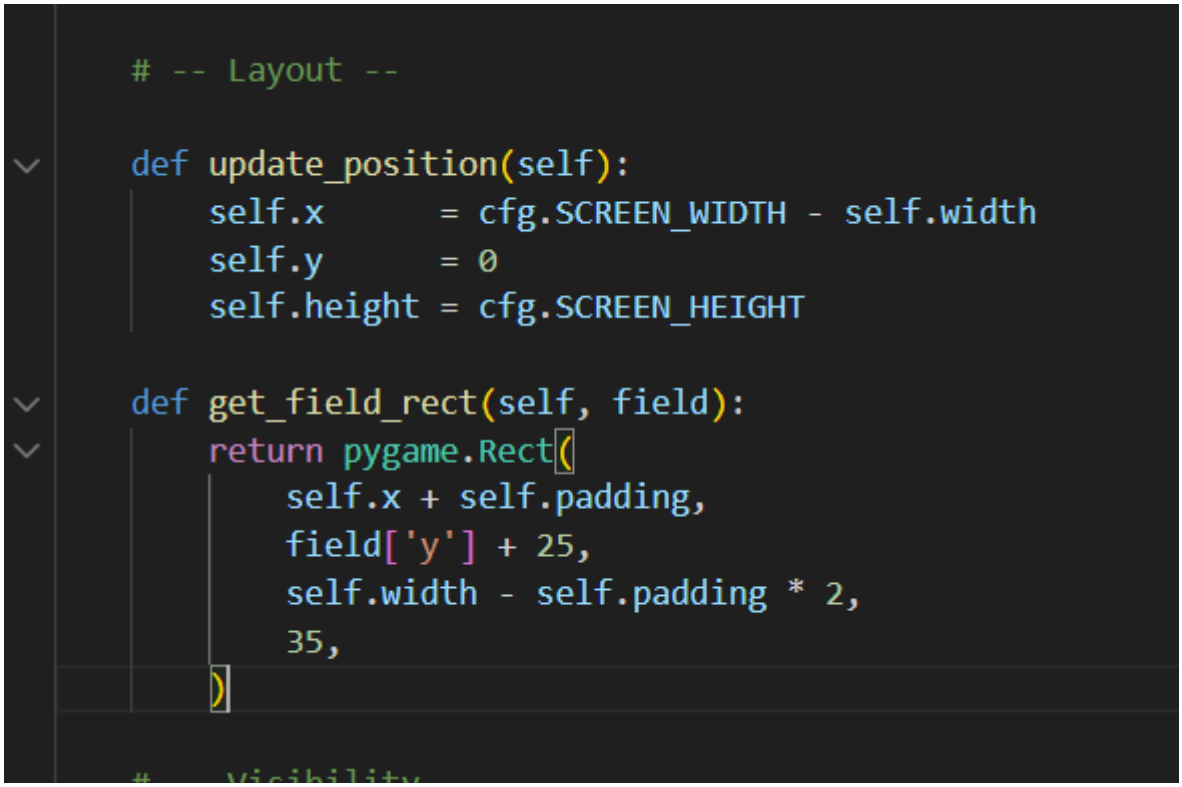
        dx = self.logical_x - prev_x # meters
        dy = self.logical_y - prev_y # meters

        self.distance_travelled += math.sqrt(dx * dx + dy * dy) # meters

        self.set_center((self.pos_x, self.pos_y))
```

3.2.2 Ui.py

Ui.py in all simulations are done in similar way:



```
    # -- Layout --

    def update_position(self):
        self.x      = cfg.SCREEN_WIDTH - self.width
        self.y      = 0
        self.height = cfg.SCREEN_HEIGHT

    def get_field_rect(self, field):
        return pygame.Rect(
            self.x + self.padding,
            field['y'] + 25,
            self.width - self.padding * 2,
            35,
        )

    #   Visibility
```

```
    # -- Visibility --

    def show(self, target_object=None):

        from objects import MovingObject

        self.target_object = target_object
        self.visible = True
        self.fields = []
        self.active_field = None
        self.update_position()

        if target_object is None:
            self.menu_mode = "settings"
            self._setup_settings_fields()
        elif isinstance(target_object, MovingObject):
            self.menu_mode = "moving_object"
            self._setup_moving_object_fields()
        else:
            # If it's a StaticObject (plane), show settings instead
            self.menu_mode = "settings"
            self._setup_settings_fields()

    def hide(self):
        self.visible = False
        self.target_object = None
        self.active_field = None
```

```
    # -- Event handling --

    def handle_event(self, event):

        if not self.visible:
            return False

        if event.type == pygame.MOUSEBUTTONDOWN:
            self.active_field = None
            for field in self.fields:
                if field['editable'] and self.get_field_rect(field).collidepoint(event.pos):
                    self.active_field = field
                    return True

        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                self.hide()
                return "hide"

            if self.active_field:
                if event.key == pygame.K_RETURN:
                    self.apply_changes()
                    self.active_field = None
                    return "apply_changes"
                elif event.key == pygame.K_BACKSPACE:
                    self.active_field['value'] = self.active_field['value'][:-1]
                elif event.unicode in '0123456789.-':
                    self.active_field['value'] += event.unicode
                return True

        return False
```

```
        # Bottom instructions
        instructions = [
            "Press ENTER to save changes",
            "Press ESC to close menu",
            "",
            "Controls:",
            "RMB Click object - Edit properties",
            "RMB Click empty space - Settings",
            "F - Pause/Resume",
            "R - Reset simulation",
        ]
        inst_y = self.height - 220
        for i, line in enumerate(instructions):
            surf = self.input_font.render(line, True, self.text_color)
            screen.blit(surf,
                        (self.x + (self.width - surf.get_width()) // 2,
                         inst_y + i * 25))
```

```
    # -- Apply --

    def apply_changes(self):
        try:
            if self.menu_mode == "settings":
                for field in self.fields:
                    value = float(field['value'])

                    if field['name'] == 'time_scale' and value > 0:
                        cfg.TIME_SCALE = value
                        print(f"Time scale -> {value}")

                    elif field['name'] == 'plane_angle':
                        # Clamp value
                        if value > 90: value = 90
                        elif value < -90: value = -90
                        cfg.ANGLE = value
                        print(f"Plane angle -> {value}")

                    elif field['name'] == 'friction_coefficient':
                        # Clamp value
                        if value < 0: value = 0
                        elif value > 1: value = 1
                        cfg.FRICTION_COEFFICIENT = value
                        print(f"Friction coefficient -> {value}")

            elif self.menu_mode == "moving_object":
                obj = self.target_object
                for field in self.fields:
                    value = float(field['value'])

                    if field['name'] == 'mass' and value > 0:
                        obj.mass    = value
                        obj.gforce  = [0, value * cfg.GRAVITY]

                    elif field['name'] == 'velocity_magnitude':
                        obj.set_velocity_along_plane(value, cfg.ANGLE)

            print(f"Changes applied to {self.menu_mode}")
```

3.2.3 Main.py

All mains have similar structure:

Close button class to return to menu

```
# -- Close Button --
class CloseButton:

    def __init__(self, x, y, width, height):
        self.rect = pygame.Rect(x, y, width, height)
        self.hovered = False
        self.font = pygame.font.Font(None, 24)

        self.bg_color = [98, 114, 164]
        self.hover_color = [139, 157, 195]
        self.text_color = [248, 248, 242]

    def handle_event(self, event):
        if event.type == pygame.MOUSEMOTION:
            self.hovered = self.rect.collidepoint(event.pos)
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.hovered:
                return True
        return False

    def draw(self, screen):
        color = self.hover_color if self.hovered else self.bg_color
        pygame.draw.rect(screen, color, self.rect, border_radius=6)
        pygame.draw.rect(screen, self.text_color, self.rect, 2, border_radius=6)

        text_surf = self.font.render("Close (Return to Menu)", True, self.text_color)
        text_rect = text_surf.get_rect(center=self.rect.center)
        screen.blit(text_surf, text_rect)
```

Main loop which draws the window and calculates physics, at the start of each loop time elapsed since last iteration is calculated:

```
# -- Main loop --

run = True
while run:

    # -- Timing --
    current_time = time.perf_counter()
    dt = current_time - last_time
    last_time = current_time

    if simulation_started and not simulation_paused:
        accumulator += dt * cfg.TIME_SCALE
    else:
        accumulator = 0.0
```

Then there is event loop for user inputs in this kind of structure:

```
# -- Events --
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        run = False

    # Close button takes priority
    if close_button.handle_event(event):
        print("Returning to main menu...")
        run = False
        continue
```

Drag-and-drop on example of friction simulation

```
    # -- Drag-and-drop --
    if mouse.is_lmb_clicked() and simulation_paused:
        mouse_pos = mouse.get_mouse_pos()

        if mouse.draggable_object is None:
            # Try to pick up an object under the cursor
            if moving_object.click_hitbox.collidepoint(mouse_pos):
                mouse.draggable_object = moving_object
        else:
            # Move the held object to the cursor, then snap it onto the plane
            mouse.draggable_object.set_center(mouse_pos)
            border = check_border_collision(*mouse_pos, plane)
            if border != "none":
                set_object_before_plane(border, plane, mouse.draggable_object)
            else:
                set_object_on_plane(plane, mouse.draggable_object)

    elif not mouse.is_lmb_clicked() and simulation_paused:
        # Drop: sync logical position if an object was being dragged
        if mouse.draggable_object is not None:
            mouse.draggable_object.sync_logical_to_visual()
            mouse.draggable_object.set_velocity_x(0)
            mouse.draggable_object.set_velocity_y(0)
        mouse.draggable_object = None
```

Physics loop inside main loop on example of friction simulation

```
# -- Physics --
physics_steps = 0
while accumulator >= physics_timestep and physics_steps < 1:
    if simulation_started and not simulation_paused:
        x, y   = moving_object.get_center()
        border = check_border_collision(x, y, plane)
        if border != "none":
            set_object_before_plane(border, plane, moving_object)

        # Clamp near-zero motion to a full stop
        if (abs(moving_object.get_acceleration_x()) < 0.1
                and abs(moving_object.get_acceleration_y()) < 0.1
                and abs(moving_object.get_velocity_x())     < 0.1
                and abs(moving_object.get_velocity_y())     < 0.1):
            moving_object.set_velocity_x(0)
            moving_object.set_velocity_y(0)

        force_x, force_y = friction_manager.calculate_forces()
        moving_object.resolve_forces((force_x, force_y))
        moving_object.update_physics(physics_timestep)

    accumulator   -= physics_timestep
    physics_steps += 1
```

And then render of objects, direction of motion of the object, HUD, UI on the screen:

```
# -- Rendering --
screen.fill(cfg.COLORS["WHITE"])

plane.draw(screen)
moving_object.draw(screen)

# Plane edges
for static_obj in edges_list:
    for edge in static_obj.edges:
        pygame.draw.line(screen, (255, 0, 255), edge[0], edge[1], 2)

# Draw click hitboxes (debug)
pygame.draw.rect(screen, cfg.COLORS["GREEN"], moving_object.click_hitbox, 1)

# Velocity arrow
center = moving_object.get_center()
pygame.draw.circle(screen, (0, 0, 255), (int(center[0]), int(center[1])), 5)

vel_x, vel_y = moving_object.get_velocity_x(), moving_object.get_velocity_y()
pygame.draw.line(
    screen, (0, 0, 255),
    center,
    (center[0] + vel_x * 10, center[1] + vel_y * 10),
    2,
)
```

```
    # Edit menu overlay
    edit_menu.draw(screen)

    # Close button
    close_button.draw(screen)

    # -- HUD --
    font      = pygame.font.Font(None, 24)
    sim_state = ("RUNNING" if simulation_started and not simulation_paused
                 else "PAUSED")
    hud_lines = [
        f"Graphics FPS: {current_fps}/{cfg.FPS}",
        f"Physics FPS: {cfg.PHYSICS_FPS}",
        f"Distance travelled: {moving_object.distance_travelled:.3f} m",
        f"Velocity X: {moving_object.get_velocity_x():.2f} m/s",
        f"Velocity Y: {moving_object.get_velocity_y():.2f} m/s",
        f"Acceleration X: {moving_object.acceleration_x:.2f} m/s²",
        f"Acceleration Y: {moving_object.acceleration_y:.2f} m/s²",
        f"Mass: {moving_object.mass} kg",
        f"Plane angle: {cfg.ANGLE}°",
        f"Friction coefficient: {cfg.FRICTION_COEFFICIENT}",
        f"Time Scale: {cfg.TIME_SCALE}x",
        f"Simulation: {sim_state}",
    ]
    for i, line in enumerate(hud_lines):
        surf = font.render(line, True, cfg.COLORS["GREEN"])
        screen.blit(surf, (10, 300 + i * 25))
```

And and fps counter to see the performance of the simulation:

```
370
371             # -- FPS counter --
372             fps_counter += 1
373             if time.time() - fps_timer >= 1.0:
374                 current_fps = fps_counter
375                 fps_counter = 0
376                 fps_timer   = time.time()
377
378             pygame.display.flip()
379             clock.tick(cfg.FPS)
380
381         pygame.quit()
382
383     if __name__ == "__main__":
384         main()
```

3.3 Friction simulation

3.3.1 Friction.py

This file manages physics for the simulation and calculates acceleration that has to be applied to the moving object.

```
class FrictionManager:

    # -- Init --

    def __init__(self, plane, plane_angle, moving_object, friction_coefficient):
        self.plane = plane
        self.plane_angle = plane_angle   # degrees
        self.moving_object = moving_object
        self.friction_coefficient = friction_coefficient

        self.angle_rad = math.radians(self.plane_angle)
```

```
    # -- Utility --

    def _gravity_components(self):
        weight = self.moving_object.mass * cfg.GRAVITY
        return (
            weight * math.sin(self.angle_rad),  # pulls object down the slope
            weight * math.cos(self.angle_rad),  # presses object into slope
        )
```

```
    def get_velocity_component_along_plane(self, vel_x, vel_y):
        plane_dir_x = math.cos(self.angle_rad)
        plane_dir_y = math.sin(self.angle_rad)
        return vel_x * plane_dir_x + vel_y * plane_dir_y
```

```python
def calculate_forces(self):
    gravity_parallel, gravity_perp = self._gravity_components()
    normal_force = gravity_perp
    max_friction = self.friction_coefficient * normal_force

    vel_x = self.moving_object.get_velocity_x()
    vel_y = self.moving_object.get_velocity_y()
    velocity_along_plane = self.get_velocity_component_along_plane(vel_x, vel_y)

    velocity_threshold = 0.1  # m/s — below this considered stopped

    # determine friction force
    if abs(velocity_along_plane) > velocity_threshold:
        # object is moving — friction opposes motion
        friction_force = -math.copysign(max_friction, velocity_along_plane)

    elif abs(velocity_along_plane) <= velocity_threshold and gravity_parallel != 0.0:
        # object is stopped — friction opposes gravity
        if abs(gravity_parallel) <= max_friction:
            # friction can hold object in place
            self.moving_object.set_velocity_x(0.0)
            self.moving_object.set_velocity_y(0.0)
            friction_force = -gravity_parallel  # Cancel gravity exactly
        else:
            # gravity overcomes friction — object will slide
            friction_force = -math.copysign(max_friction, gravity_parallel)

    else:
        # flat plane, no velocity, no gravity component
        return 0.0, 0.0

    # calculate net force
    net_force_along_plane = gravity_parallel + friction_force

    # prevent oscillation when nearly stopped
    if abs(velocity_along_plane) < velocity_threshold:
        if gravity_parallel != 0.0 and (
            math.copysign(1.0, net_force_along_plane) !=
            math.copysign(1.0, gravity_parallel)
        ):
            net_force_along_plane = 0.0

    # convert to world coordinates
    force_x = net_force_along_plane * math.cos(self.angle_rad)
    force_y = net_force_along_plane * math.sin(self.angle_rad)
    return force_x, force_y
```

3.3.2 Main.py

Physics application in main pygame loop

```
# -- Physics --
physics_steps = 0
while accumulator >= physics_timestep and physics_steps < 1:
    if simulation_started and not simulation_paused:
        x, y   = moving_object.get_center()
        border = check_border_collision(x, y, plane)
        if border != "none":
            set_object_before_plane(border, plane, moving_object)

        # Clamp near-zero motion to a full stop
        if (abs(moving_object.get_acceleration_x()) < 0.1
                and abs(moving_object.get_acceleration_y()) < 0.1
                and abs(moving_object.get_velocity_x())     < 0.1
                and abs(moving_object.get_velocity_y())     < 0.1):
            moving_object.set_velocity_x(0)
            moving_object.set_velocity_y(0)

        force_x, force_y = friction_manager.calculate_forces()
        moving_object.resolve_forces((force_x, force_y))
        moving_object.update_physics(physics_timestep)

    accumulator   -= physics_timestep
    physics_steps += 1
```

3.4 Rebound simulation

3.4.1 Collisions.py

Geometry helpers functions to help calculate the physics

```
# -- Geometry helpers --

def get_line_normal(point1, point2):
    dx = point2[0] - point1[0]
    dy = point2[1] - point1[1]

    normal = (-dy, dx)

    length = math.hypot(normal[0], normal[1])
    if length == 0:
        return (0, 0)
    return (normal[0] / length, normal[1] / length)


def distance_point_to_line(point, p1, p2):
    ax = point[0] - p1[0]
    ay = point[1] - p1[1]
    cx = p2[0] - p1[0]
    cy = p2[1] - p1[1]

    length_sq = cx * cx + cy * cy
    if length_sq == 0:
        return vec.distance_between_points(point, p1), p1

    t = max(0.0, min(1.0, (ax * cx + ay * cy) / length_sq))

    closest = (p1[0] + t * cx, p1[1] + t * cy)
    return vec.distance_between_points(point, closest), closest
```

Circle-circle collisions and resolving the collision

```python
# -- Circle-circle collisions --

def check_circle_collisions(moving_objects):
    collisions = []
    moving_list = list(moving_objects)

    for i in range(len(moving_list)):
        for j in range(i + 1, len(moving_list)):
            obj1 = moving_list[i]
            obj2 = moving_list[j]

            c1 = obj1.get_center()
            c2 = obj2.get_center()
            r1 = obj1.get_width() / 2
            r2 = obj2.get_width() / 2

            dist = vec.distance_between_points(c1, c2)

            if dist < r1 + r2:
                normal = ((c2[0] - c1[0]) / dist,
                          (c2[1] - c1[1]) / dist)

                collisions.append((obj1, obj2, normal))

    return collisions
```

Resolved collisions are stored in a set to prevent resolving them again.

```python
def resolve_circle_collisions(collisions, elasticity):
    resolved_pairs = set()

    for obj1, obj2, normal in collisions:
        pair_id = tuple(sorted([id(obj1), id(obj2)]))
        if pair_id in resolved_pairs:
            continue
        resolved_pairs.add(pair_id)

        m1 = obj1.get_mass()
        m2 = obj2.get_mass()
        v1 = (obj1.get_velocity_x(), obj1.get_velocity_y())
        v2 = (obj2.get_velocity_x(), obj2.get_velocity_y())

        # velocity impulse (only if approaching)
        v_rel_along_normal = vec.dot_product(vec.vector_subtract(v1, v2), normal)
        if v_rel_along_normal <= 0:
            continue

        # J = -(1+e) * v_rel_n / (1/m1 + 1/m2)
        J = -(1 + elasticity) * v_rel_along_normal / (1 / m1 + 1 / m2)
        new_v1 = vec.vector_add(v1, vec.scalar_multiply(normal, J / m1))
        new_v2 = vec.vector_subtract(v2, vec.scalar_multiply(normal, J / m2))

        obj1.set_velocity_x(new_v1[0])
        obj1.set_velocity_y(new_v1[1])
        obj2.set_velocity_x(new_v2[0])
        obj2.set_velocity_y(new_v2[1])
```

Circle-wall collisions and resolving them

```python
# -- Circle-wall collisions --

def check_wall_collisions(moving_objects, static_objects):
    collisions = []

    for moving_obj in moving_objects:
        center = moving_obj.get_center()
        radius = moving_obj.get_width() / 2

        for static_obj in static_objects:
            for p1, p2 in static_obj.edges:
                dist, closest = distance_point_to_line(center, p1, p2)

                if dist > radius:
                    continue

                normal = get_line_normal(p1, p2)
                to_circle = (center[0] - closest[0], center[1] - closest[1])

                # flip normal if it points away from the circle
                if vec.dot_product(normal, to_circle) < 0:
                    normal = (-normal[0], -normal[1])

                collisions.append((moving_obj, static_obj, normal))

    return collisions
```

```python
def resolve_wall_collisions(collisions):
    for moving_obj, static_obj, normal in collisions:
        velocity = (moving_obj.get_velocity_x(), moving_obj.get_velocity_y())
        v_dot_n = vec.dot_product(velocity, normal)
        elasticity = static_obj.get_elasticity()

        # only process collision if moving into wall
        if v_dot_n >= 0:
            continue  # already separating, no impulse needed

        # apply impulse: Δv = -(1+e) * (v·n) * n
        J = vec.scalar_multiply(normal, -(1 + elasticity) * v_dot_n)
        new_velocity = vec.vector_add(velocity, J)

        moving_obj.set_velocity_x(new_velocity[0])
        moving_obj.set_velocity_y(new_velocity[1])
```

3.4.2 Main.py

Physics loop in main pygame loop that checks for mode of the simulation, checks and resolves the collisions with collisions.py and checks if there is no rebound to stop the simulation.

```
        # -- Physics (fixed timestep) --
        physics_steps = 0
        while accumulator >= physics_timestep and physics_steps < 10:

            # Step 1: detect and resolve wall contacts (only for non-stopped objects)
            wall_collisions = collisions.check_wall_collisions(moving_objects, edges_list)
            if wall_collisions:
                collisions.resolve_wall_collisions(wall_collisions)

            # Step 2: apply forces (only to non-stopped objects)
            for obj in moving_objects:
                obj.acceleration_x = 0.0
                obj.acceleration_y = 0.0

                # Only apply gravity in 2D mode
                if cfg.COLLISION_MODE == "2D":
                    obj.resolve_forces(obj.get_gforce())

            # Step 3: integrate positions and velocities (only for non-stopped objects)
            for obj in moving_objects:
                obj.update_physics(physics_timestep)

            # Step 4: ball-ball collisions (after new positions, only active objects)
            circle_collisions = collisions.check_circle_collisions(moving_objects)
            if circle_collisions:
                collisions.resolve_circle_collisions(circle_collisions, elasticity=cfg.COLLISION_ELASTICITY)

            # Step 5: check total energy and stop objects with insufficient energy
            if cfg.COLLISION_MODE == "2D":
                ground_y = GROUND_SURFACE_Y  # Top surface of ground edge

                all_objects_stopped = True
                for obj in moving_objects:
                    # default threshold: 0.02m = 2cm equivalent height
                    if obj.check_should_stop(ground_y, threshold_height=0.02):
                        # stop the object if no rebound
                        obj.velocity_x = 0
                        obj.velocity_y = 0

                    else:
                        all_objects_stopped = False

                if all_objects_stopped:
                    simulation_started = not simulation_started
                    simulation_paused = not simulation_paused

            accumulator   -= physics_timestep
            physics_steps += 1
```

3.5 String simulation

3.5.1 Strings.py

Strings class that creates string object on the screen and calculates physics for the string that is then applied to moving object.

```python
class String:

    def __init__(self, start_x, start_y, elasticity_modulus, natural_length, particle):
        self.start_x = start_x
        self.start_y = start_y
        self.elasticity_modulus  = elasticity_modulus
        self.natural_length = natural_length
        self.particle = particle

        self.elasticity_coefficient = elasticity_modulus / natural_length

        # logical anchor position in meters
        self.logical_start_x = start_x / cfg.PIXELS_PER_METER
        self.logical_start_y = start_y / cfg.PIXELS_PER_METER

        # camera offset for panning / zooming
        self.camera_offset_x = 0.0
        self.camera_offset_y = 0.0

        # small visible radius for the anchor dot and its click hitbox (10cm)
        self.radius = 0.1  # metres
        self.click_hitbox = self._make_hitbox()
```

```python
    # -- Physics --

    def calculate_extension(self):
        px, py   = self.particle.get_logical_position()
        distance = math.hypot(self.logical_start_x - px, self.logical_start_y - py)
        return distance - self.natural_length

    def calculate_angle(self):
        px, py = self.particle.get_logical_position()
        return math.atan2(self.logical_start_y - py, self.logical_start_x - px)

    def calculate_tension(self):
        extension = self.calculate_extension()
        if extension <= 0:
            return (0.0, 0.0)

        tension = self.elasticity_coefficient * extension
        angle   = self.calculate_angle()
        return (math.cos(angle) * tension, math.sin(angle) * tension)

    def calcualte_elastic_potential_energy(self):
        # EPE = λx^2 / 2l where λ is modulus of elasticity, l is natural length, x is extension
        return (self.elasticity_modulus*(self.calculate_extension()**2))/(2*self.natural_length)

    # -- Drawing --

    def draw(self, surface):
        color = cfg.COLORS["RED"] if self.calculate_extension() > 0 else cfg.COLORS["BLUE"]
        pygame.draw.circle(surface, cfg.COLORS["GREEN"],
                           (int(self.start_x), int(self.start_y)),
                           int(self.radius * cfg.PIXELS_PER_METER))
        pygame.draw.aaline(surface, color,
                           (self.start_x, self.start_y),
                           self.particle.get_center())
```

3.5.2 Main.py

Physics in main pygame loop that sums app the tensions from all strings

```
# -- Physics --
physics_steps = 0
while accumulator >= physics_timestep and physics_steps < 10:
    if simulation_started and not simulation_paused:
        forces = [s.calculate_tension() for s in string_objects]
        forces.append(moving_object.get_gforce())
        moving_object.resolve_forces(*forces)
        moving_object.update_physics(physics_timestep)
    accumulator   -= physics_timestep
    physics_steps += 1
```

4. Testing

4.1 Menu

Test number	Description	Timing	Expected Outcome	Pass/Fail
1	Settings button creates the settings window in the left side of the screen	0:07	A window appears in the left side of the screen with input boxes for Graphics FPS, Physics FPS and Gravity. As well as buttons to save or close the settigns window,	Pass
2	Values are entered into input boxes and then saved into config.json. Program is then restarted.	0:17-0:37	Values are written into config.json and program restarts successfully.	Pass
3.	All simulations are started and then closed with Close button in the simulation. Menu is opened after simulation was closed.	0:37-0:52	All simulations are started and closed correctly and then closed with Close button in the simulation. Menu is opened right after the simulation was closed.	Pass
4.	Simulation is closed via Exit button in the Menu	0:54	Simulation is successfully closed after the press of the button	Pass

4.2 Friction Simulation

Test number	Description	Timing	Expected Outcome	Pass/Fail
1	Settings windows of simulation and moving object are opened with RMB and closed with ESC.	Throughout the video	Settings windows are opened with RMB and closed with ESC successfully.	Pass
2	Friction coefficient and plane angle is changed in settings menu of the simulation,	0:18	Values are changed successfully.	Pass
3	Velocity and mass of moving object is changed.	0:40	Values are changed successfully.	Pass
4	Object moves smoothly on the plane with deceleration due to friction.	Throughout the video	Object moves successfully.	Pass
5	Time scale is changed to slow down the simulation	1:04	Time scale is correctly applied that results in slowing down the simulation.	Pass
6	Object appears on opposite side of the plane when it reaches the end of the plane.	0:38	Object appears on other side successfully.	Pass
7	User moves the object on the plane using LMB.	0:12	Object is clamped to move directly on the plane correctly.	Pass
8	Simulation is paused and resumed.	Throughout the video	Simulation is paused and resumed successfully.	Pass
9	Simulation data is displayed in the HUD	Throughout the video	Data is displayed successfully	Pass

4.3 Rebound simulation

Test number	Description	Timing	Expected Outcome	Pass/Fail
1	Settings windows of simulation and moving object are opened with RMB and closed with ESC.	Throughout the video	Settings windows are opened with RMB and closed with ESC successfully.	Pass
2	Settings values (mass, speed, radius) are changed and applied	1:05	Values are applied correctly	Pass
3	Moving objects rebound from surfaces and from each other correctly. When elasticity of collisions is 1 the perfectly elastic collision is processed correctly: moving object travels exact same distance vertically without diverging. When elasticity of collisions is <1 (0.6) object loses momentum and it results in a full stop of the object as there is no rebounds.	-	Objects rebound and stop correctly	Pass
4	Speed acceleration, energy and etc. is displayed in the HUD	Throughout the video	Data is displayed	Pass
5	Mode of the simulation is changed. 2D = apply gravity. 1D = no gravity.	1:15	Mode is altered correctly	Pass
6	Simulation is paused and resumed	Throughout the video	Simulation is paused and resumed correctly.	Pass
7	Trail of the object is created.	Throughout the video	Trail of the object is created successfully.	Pass
8	Time scale is applied	0:13	Time scale is applied correctly	Pass

4.4 String simulation

Test number	Description	Timing	Expected Outcome	Pass/Fail

1	Settings windows of simulation and moving object are opened with RMB and closed with ESC.	Throughout the video	Settings windows are opened with RMB and closed with ESC successfully.	Pass
2	Strings are deleted and added	1:06	Strings are deleted and added successfully	Pass
3	Data about strings and moving object is displayed in HUD	Throughout the video	All data is displayed	Pass
4	Trail is created	Throughout the video	Trail is created successfully	Pass
5	Time scale is applied	1:25	Time scale is applied correctly	Pass
6	Strings are displayed correctly and flagged with red colour whenever they are starting to extend.	Throughout the video		
7	Camera pan and zoom are working properly without distorting the physics of the running simulation	0:05	Correct work of camera pan and zoom	Pass
8	Velocity and mass of moving object is changed.	0:30	Values are changed successfully.	Pass
9	Modulus of elasticity and natural length of the string is changed.	0:40	Values are changed successfully.	Pass

Videos:

1. Menu test: https://youtu.be/olUHWqYW2Ec

2. Friction test: https://youtu.be/-Kj08BYeuRg

3 Rebound test: https://youtu.be/JfIs2MEUrqs

4. String test: https://youtu.be/ZFSTIRAqCzs

5. Evaluation

Objective	Current state	Improvement
Create correct render of events and mathematically correct physics that are rendered smoothly and calculated correctly regardless of FPS using separate PHYSICS_FPS.	Program appears to run smoothly and the FPS is separated that gives the program fluidity and sustainability as the visual lag does not affect physics in any way.	Could make constraints: PHYSICS_FPS should always be bigger than GRAPHICS_FPS
2. Create intuitive UI so the user can change most of the inputs and interact with the simulation.	UI is intuitive with instructions shown for the user on the screen. All of the features listed in that objective were successfully added.	Could make the UI to be calculated in more proficient way by calculating the percentage of the screen taken and not be constant for current screen resolution (1280x720)
3. Create three separate simulations with proper rendering and physics calculations with separating visual and physical calculation	All of the three simulations were made successfully with a proper calculation and displaying of the physics according to Mr Robinson.	Could add more physics simulations for other A Level Further Math areas.
4. Correct calculation of gravity, friction, speed, acceleration, energy and etc. via SUVAT equations and other math formulas.	All of the values are correctly calculated and applied to the moving object.	No improvement needed
5. Displaying program calculations results and simulation state with time scaling	All necessary data is displayed in the HUD	Again, could make it to be more fluid and dynamic by calculating the percentage of the screen taken and not to be constant for current screen resolution (1280x720)
6. Create pause/resume, time scale feature for every simulation.	Features were successfully created and implemented.	No improvement needed
7. Create saveable parameters that are shared for all three simulations: FPS, PHYSICS_FPS, GRAVITY.	Values inputted by user are saved into config.json file from where they are exported back to the program.	Could make more values changeable and also store more values in config,json
8. Trail of objects for rebound and string simulation so the user can see how object was travelling in space.	Trails were successfully added and displayed	No improvement needed

9. Create correct separation of logical and physical calculations of positions of objects in space.	Logic is implemented and used throughout the program	No improvement needed
10. Create clear visuals so the user understands what happens to the objects on the screen.	Visuals are clear and users can distinguish objects, read instructions, see direction of object and if tension is applied to the string.	Could make more indications for objects that are displayed only in debug mode.
11. Create menu from where user can change FPS and gravity for the simulation and start simulations.	Menu it intuitive and clear to user what to click.	Could add more values to be changed.

5.1 Feedback

Project turned out looking good and understandable for first time users. Mr Robinson has complemented the appearance and amount of effort invested into this project, he said that he will be using this project as a showcase of physics for some relevant problems as well as an example for his students of what knowledge from Further Math course can create.

Mr Robinson also said that program turned out to be robust and optimized as the framerate does not drop significantly and the program maintains steady run.

This project showed me an area for improvement of my understanding of how I want my projects be created from idea to solution.

5.2 Issues with the solution

The structure of the code is kind of poor as I was rushing to complete it, it works, but it looks messy and there is a lot of redundant code. If I were able to remake the project I would start by thinking what parts of the code are most usable so I can take them out and share them with rest of the program. The other big problem is with the research, it is hard to find any relevant or usable information about math and applications of math equations in programming physics, so I spend lots of time just trying to find information for this project.

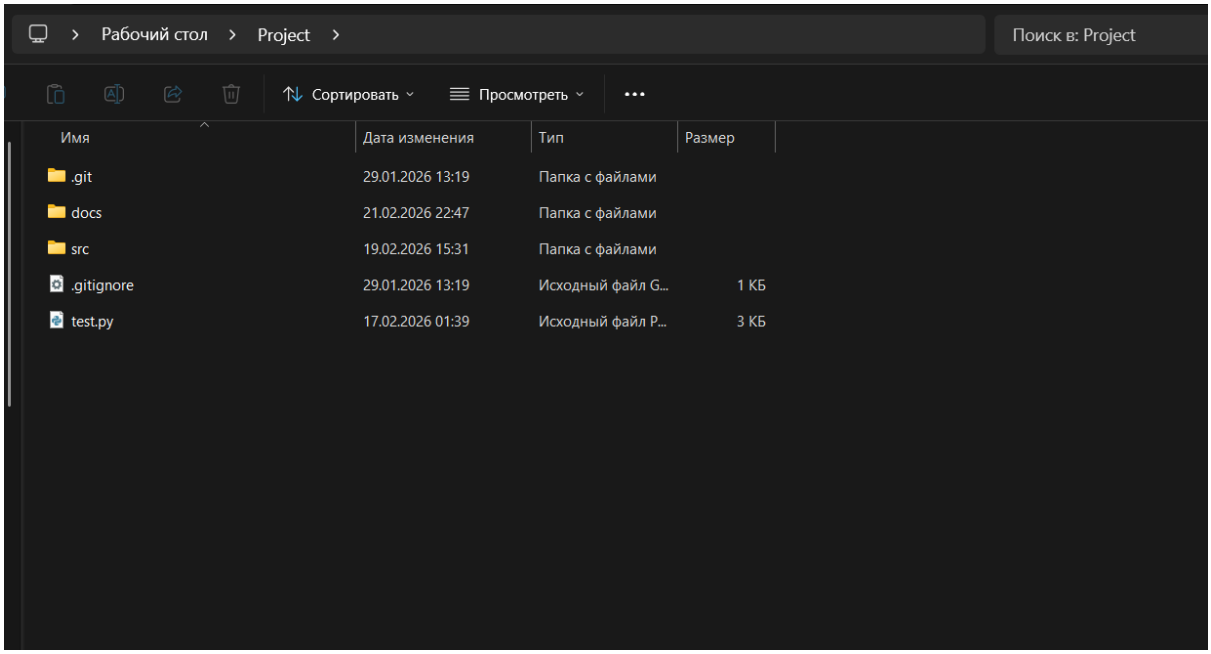
5.3 Things I would like to do differently

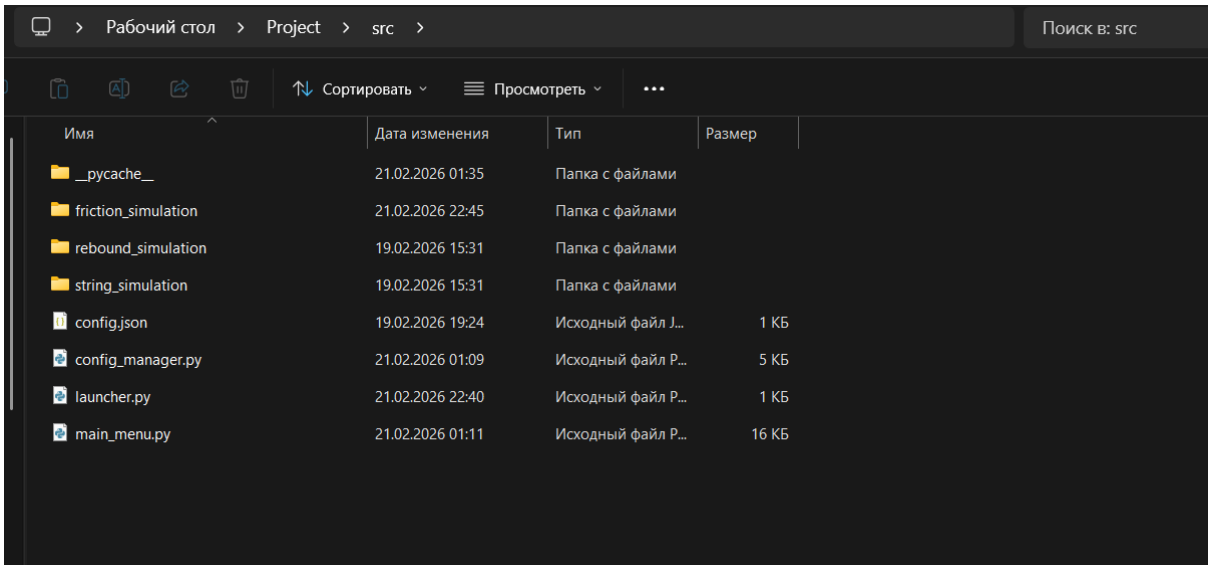
If I had more time to do this project, I would most definitely restructure the code, so it looks much cleaner and easier to understand. Modularization of the program is alright for now but if I would like to improve it, it would have to be improved to maintain the easier to edit way of coding.

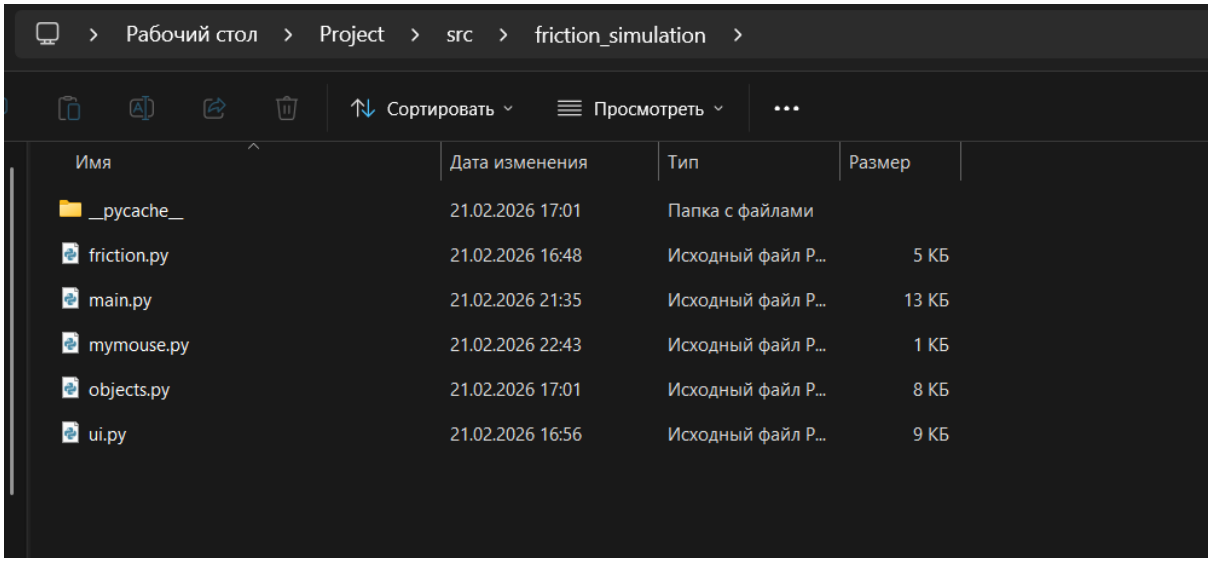
I would like to use Git for version control as it has many more options and is just faster and better than OneDrive where I had stored all previous versions.

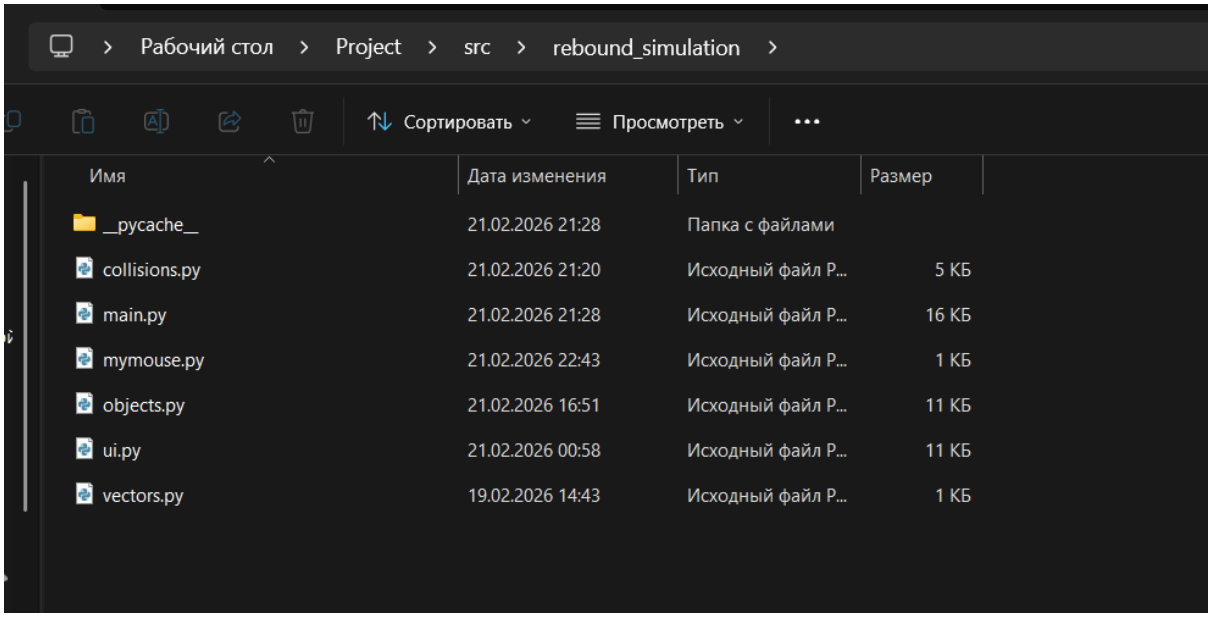
6. Appendix

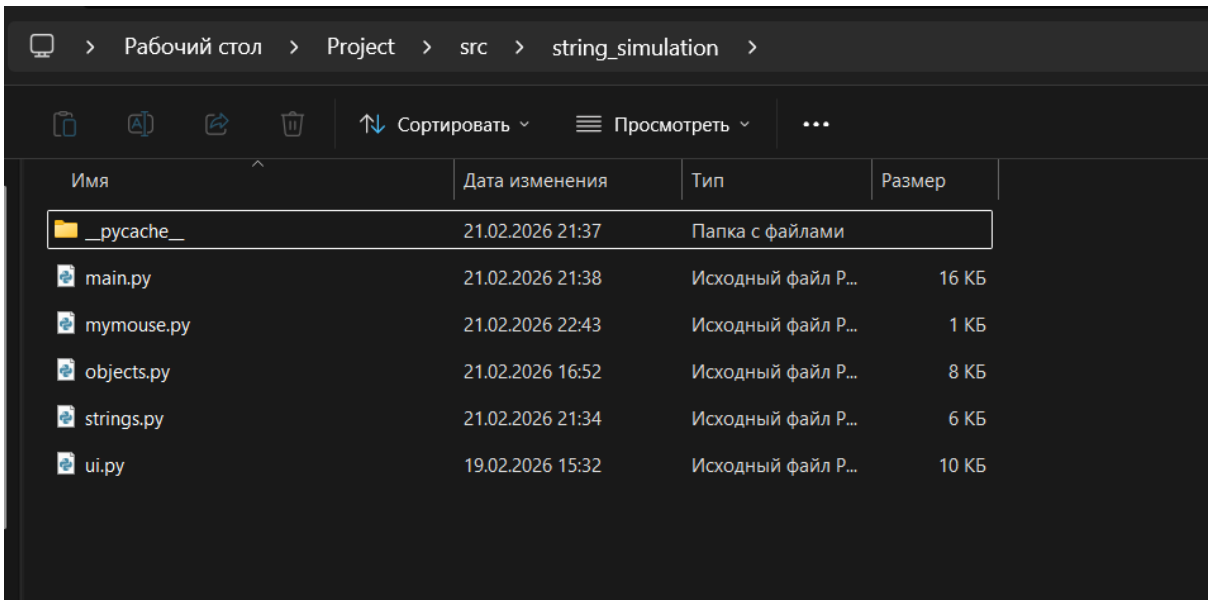
6.0 File organization






Рабочий стол › Project › src › friction_simulation ›
Сортировать
Просмотреть
Имя
Дата изменения
Тип
Размер
__pycache__ 21.02.2026 17:01 Папка с файлами
friction.py 21.02.2026 16:48 Исходный файл P... 5 КБ
main.py 21.02.2026 21:35 Исходный файл P... 13 КБ
mymouse.py 21.02.2026 22:43 Исходный файл P... 1 КБ
objects.py 21.02.2026 17:01 Исходный файл P... 8 КБ
ui.py 21.02.2026 16:56 Исходный файл P... 9 КБ


Рабочий стол › Project › src › rebound_simulation ›
Сортировать
Просмотреть
Имя
Дата изменения
Тип
Размер
__pycache__ 21.02.2026 21:28 Папка с файлами
collisions.py 21.02.2026 21:20 Исходный файл P... 5 КБ
main.py 21.02.2026 21:28 Исходный файл P... 16 КБ
mymouse.py 21.02.2026 22:43 Исходный файл P... 1 КБ
objects.py 21.02.2026 16:51 Исходный файл P... 11 КБ
ui.py 21.02.2026 00:58 Исходный файл P... 11 КБ
vectors.py 19.02.2026 14:43 Исходный файл P... 1 КБ


Рабочий стол › Project › src › string_simulation ›
Сортировать
Просмотреть
Имя
Дата изменения
Тип
Размер
__pycache__ 21.02.2026 21:37 Папка с файлами
main.py 21.02.2026 21:38 Исходный файл P... 16 КБ
mymouse.py 21.02.2026 22:43 Исходный файл P... 1 КБ
objects.py 21.02.2026 16:52 Исходный файл P... 8 КБ
strings.py 21.02.2026 21:34 Исходный файл P... 6 КБ
ui.py 19.02.2026 15:32 Исходный файл P... 10 КБ

8.1 Menu

6.1.1 Launcher.py

```
import sys
import pathlib

# add parent directory to path for imports
current_dir = pathlib.Path(__file__).resolve().parent
sys.path.insert(0, str(current_dir))

from main_menu import main as main_menu_main

if __name__ == "__main__":
    print("=" * 60)
    print("Physics Simulations Suite")
    print("=" * 60)
    print()

    main_menu_main()

    print()
    print("Thank you for using Physics Simulations!")

```

6.1.2 main_menu.py

```
import pygame
import sys
import subprocess
import pathlib
from config_manager import get_config_manager

class Button:
    def __init__(self, x, y, width, height, text, callback):
        self.rect = pygame.Rect(x, y, width, height)
        self.text = text
        self.callback = callback
        self.hovered = False

        self.bg_color = [98, 114, 164]
        self.hover_color = [139, 157, 195]
        self.text_color = [248, 248, 242]

        self.font = pygame.font.Font(None, 32)

    def handle_event(self, event):
        if event.type == pygame.MOUSEMOTION:
            self.hovered = self.rect.collidepoint(event.pos)
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.hovered:
                self.callback()
                return True
        return False

    def draw(self, screen):
        color = self.hover_color if self.hovered else self.bg_color
        pygame.draw.rect(screen, color, self.rect, border_radius=8)
        pygame.draw.rect(screen, self.text_color, self.rect, 2, border_radius=8)

        text_surf = self.font.render(self.text, True, self.text_color)
```

```
35.         text_rect = text_surf.get_rect(center=self.rect.center)
36.         screen.blit(text_surf, text_rect)
37.
38. class SettingsPanel:
39.
40.     def __init__(self, menu=None):
41.         self.cfg = get_config_manager()
42.         self.visible = False
43.         self.menu = menu  # reference to main menu for restart
44.
45.         self.width = 330
46.         self.padding = 20
47.
48.         self.title_font = pygame.font.Font(None, 36)
49.         self.label_font = pygame.font.Font(None, 24)
50.         self.input_font = pygame.font.Font(None, 22)
51.
52.         self.bg_color = [40, 42, 54]
53.         self.text_color = [248, 248, 242]
54.         self.input_bg = [68, 71, 90]
55.         self.input_active = [98, 114, 164]
56.         self.border_color = [98, 114, 164]
57.
58.         self.fields = []
59.         self.active_field = None
60.         self.close_button = None
61.         self.save_button = None
62.
63.         self.update_position()
64.
65.     def update_position(self):
66.         screen_w = self.cfg.SCREEN_WIDTH
67.         screen_h = self.cfg.SCREEN_HEIGHT
68.
69.         self.x = screen_w - self.width
70.         self.y = 0
71.         self.height = screen_h
72.
73.         # setup fields
74.         self.fields = []
75.         y = 100
76.         spacing = 80
77.
78.         sections = [
79.             ("Display", [
80.                 ("fps", "Graphics FPS", self.cfg.FPS),
81.                 ("physics_fps", "Physics FPS", self.cfg.PHYSICS_FPS),
82.             ]),
83.             ("Physics", [
84.                 ("gravity", "Gravity (m/s²)", self.cfg.GRAVITY),
85.             ]),
86.         ]
87.
88.         for section_name, section_fields in sections:
89.             self.fields.append({
90.                 'type': 'header',
91.                 'label': section_name,
92.                 'y': y,
93.             })
94.             y += 40
95.
96.             for name, label, value in section_fields:
97.                 self.fields.append({
98.                     'type': 'input',
99.                     'name': name,
100.                    'label': label,
101.                    'value': str(value),
102.                    'y': y,
103.                })
```

```
                y += spacing

        # buttons
        button_y = self.height - 120
        button_w = (self.width - 3 * self.padding) // 2

        self.save_button = Button(
            self.x + self.padding,
            button_y,
            button_w,
            50,
            "Save",
            self.save_settings
        )

        self.close_button = Button(
            self.x + self.padding * 2 + button_w,
            button_y,
            button_w,
            50,
            "Close",
            self.hide
        )

    def show(self):
        self.visible = True
        self.cfg.load()  # reload from disk
        self.update_position()

    def hide(self):
        self.visible = False
        self.active_field = None

    def save_settings(self):
        try:
            for field in self.fields:
                if field['type'] != 'input':
                    continue

                name = field['name']
                value_str = field['value']

                # parse value
                if '.' in value_str:
                    value = float(value_str)
                else:
                    value = int(value_str)

                # map field names to config paths
                if name in ['fps', 'physics_fps']:
                    self.cfg.set("display", name, value=int(value))
                elif name in ['gravity']:
                    value = 0 if value < 0 else value  # clamp negative to 0
                    self.cfg.set("physics", name, value=value)

            if self.cfg.save():
                print("Settings saved successfully!")
                print("Restarting application to apply changes...")
                self.hide()
                # trigger restart of the application
                if self.menu:
                    self.menu.restart_after_save = True
                    self.menu.running = False

        except Exception as e:
            print(e)

    def get_field_rect(self, field):
        return pygame.Rect(
```

```
173.             self.x + self.padding,
174.             field['y'] + 30,
175.             self.width - self.padding * 2,
176.             35,
177.         )
178.
179.     def handle_event(self, event):
180.         if not self.visible:
181.             return False
182.
183.         # button events
184.         if self.save_button.handle_event(event) or self.close_button.handle_event(event):
185.             return True
186.
187.         # field events
188.         if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
189.             self.active_field = None
190.             for field in self.fields:
191.                 if field['type'] == 'input':
192.                     if self.get_field_rect(field).collidepoint(event.pos):
193.                         self.active_field = field
194.                         return True
195.
196.         elif event.type == pygame.KEYDOWN:
197.             if event.key == pygame.K_ESCAPE:
198.                 self.hide()
199.                 return True
200.
201.             if self.active_field:
202.                 if event.key == pygame.K_RETURN:
203.                     self.save_settings()
204.                 elif event.key == pygame.K_BACKSPACE:
205.                     self.active_field['value'] = self.active_field['value'][:-1]
206.                 elif event.unicode and event.unicode in '0123456789.-':
207.                     self.active_field['value'] += event.unicode
208.                 return True
209.
210.         return False
211.
212.     def draw(self, screen):
213.         if not self.visible:
214.             return
215.
216.         # panel background
217.         panel_rect = pygame.Rect(self.x, self.y, self.width, self.height)
218.         pygame.draw.rect(screen, self.bg_color, panel_rect)
219.         pygame.draw.rect(screen, self.border_color, panel_rect, 3)
220.
221.         # title
222.         title_surf = self.title_font.render("Settings", True, self.text_color)
223.         screen.blit(title_surf,
224.                     (self.x + (self.width - title_surf.get_width()) // 2, 30))
225.
226.         # display current resolution (non-editable info)
227.         resolution_text = f"Resolution: {self.cfg.SCREEN_WIDTH}x{self.cfg.SCREEN_HEIGHT}"
228.         resolution_surf = self.input_font.render(resolution_text, True, self.text_color)
229.         resolution_x = self.x + (self.width - resolution_surf.get_width()) // 2
230.         screen.blit(resolution_surf, (resolution_x, 70))
231.
232.         # fields
233.         for field in self.fields:
234.             if field['type'] == 'header':
235.                 # section header
236.                 header_surf = self.label_font.render(field['label'], True,
self.border_color)
237.                 screen.blit(header_surf, (self.x + self.padding, field['y']))
238.
239.             elif field['type'] == 'input':
240.                 # label
```

```
                label_surf = self.label_font.render(field['label'], True, self.text_color)
                screen.blit(label_surf, (self.x + self.padding, field['y']))

                # input box
                field_rect = self.get_field_rect(field)

                # background color
                bg = self.input_active if field is self.active_field else self.input_bg

                pygame.draw.rect(screen, bg, field_rect, border_radius=4)
                pygame.draw.rect(screen, self.border_color, field_rect, 2, border_radius=4)

                # text
                text_surf = self.input_font.render(field['value'], True, self.text_color)
                text_x = field_rect.x + 10
                text_y = field_rect.y + (field_rect.height - text_surf.get_height()) // 2
                screen.blit(text_surf, (text_x, text_y))

                # cursor
                if field is self.active_field:
                    cursor_x = text_x + text_surf.get_width() + 2
                    pygame.draw.line(screen, self.text_color,
                                     (cursor_x, field_rect.y + 5),
                                     (cursor_x, field_rect.y + field_rect.height - 5), 2)

        # save, close buttons
        self.save_button.draw(screen)
        self.close_button.draw(screen)

class MainMenu:
    def __init__(self):
        pygame.init()

        self.cfg = get_config_manager()
        self.screen = pygame.display.set_mode((self.cfg.SCREEN_WIDTH,
self.cfg.SCREEN_HEIGHT))
        pygame.display.set_caption("Physics Simulations - Main Menu")

        self.clock = pygame.time.Clock()
        self.running = True
        self.restart_after_sim = False
        self.restart_after_save = False
        self.pending_simulation = None

        # fonts
        self.title_font = pygame.font.Font(None, 72)
        self.subtitle_font = pygame.font.Font(None, 32)

        # colors
        self.bg_color = self.cfg.COLORS.get("WHITE", (255, 255, 255))
        self.text_color = self.cfg.COLORS.get("BLACK", (0, 0, 0))
        self.accent_color = [98, 114, 164]

        # get parent directory for simulations
        self.project_root = pathlib.Path(__file__).resolve().parent

        # settings panel (pass self for restart capability)
        self.settings_panel = SettingsPanel(menu=self)

        # create buttons
        self.buttons = []
        self.create_buttons()

    def create_buttons(self):
        center_x = self.cfg.SCREEN_WIDTH // 2
        start_y = 250
        button_width = 400
        button_height = 60
        spacing = 80
```

```
309.
310.         simulations = [
311.             ("Friction Simulation", "friction_simulation"),
312.             ("Rebound Simulation", "rebound_simulation"),
313.             ("String Simulation", "string_simulation"),
314.         ]
315.
316.         for i, (name, folder) in enumerate(simulations):
317.             y = start_y + i * spacing
318.             button = Button(
319.                 center_x - button_width // 2,
320.                 y,
321.                 button_width,
322.                 button_height,
323.                 name,
324.                 lambda f=folder: self.launch_simulation(f)
325.             )
326.             self.buttons.append(button)
327.
328.         # settings button
329.         settings_button = Button(
330.             center_x - button_width // 2,
331.             start_y + len(simulations) * spacing,
332.             button_width,
333.             button_height,
334.             "Settings",
335.             self.settings_panel.show
336.         )
337.         self.buttons.append(settings_button)
338.
339.         # exit button
340.         exit_button = Button(
341.             center_x - button_width // 2,
342.             start_y + (len(simulations) + 1) * spacing,
343.             button_width,
344.             button_height,
345.             "Exit",
346.             self.quit
347.         )
348.         self.buttons.append(exit_button)
349.
350.     def launch_simulation(self, sim_folder):
351.         sim_path = self.project_root / sim_folder / "main.py"
352.
353.         if not sim_path.exists():
354.             print(f"Simulation not found: {sim_path}")
355.             return
356.
357.         print(f"Launching {sim_folder}...")
358.
359.         # set flag to restart after simulation
360.         self.restart_after_sim = True
361.         self.running = False
362.
363.         # store the simulation path
364.         self.pending_simulation = sim_path
365.
366.     def quit(self):
367.         self.running = False
368.
369.     def handle_events(self):
370.         for event in pygame.event.get():
371.             if event.type == pygame.QUIT:
372.                 self.running = False
373.
374.             # settings panel has priority
375.             if self.settings_panel.handle_event(event):
376.                 continue
377.
```

```
            # button events
            for button in self.buttons:
                button.handle_event(event)

            # keyboard shortcuts
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    self.running = False

    def draw(self):
        self.screen.fill(self.bg_color)

        # title
        title_surf = self.title_font.render("Physics Simulations", True, self.text_color)
        title_rect = title_surf.get_rect(center=(self.cfg.SCREEN_WIDTH // 2, 100))
        self.screen.blit(title_surf, title_rect)

        # subtitle
        subtitle_surf = self.subtitle_font.render(
            "Select a simulation to begin", True, self.accent_color
        )
        subtitle_rect = subtitle_surf.get_rect(center=(self.cfg.SCREEN_WIDTH // 2, 160))
        self.screen.blit(subtitle_surf, subtitle_rect)

        # buttons
        for button in self.buttons:
            button.draw(self.screen)

        # settings panel
        self.settings_panel.draw(self.screen)

        pygame.display.flip()

    def run(self):
        while self.running:
            dt = self.clock.tick(60)
            self.handle_events()
            self.draw()

        pygame.quit()

        # return flags for proper restart handling
        return self.restart_after_save, self.pending_simulation

def main():
    while True:
        menu = MainMenu()
        restart, simulation = menu.run()

        if restart:
            # restart with new settings
            print("Restarting application with new settings...")
            try:
                subprocess.run([sys.executable] + sys.argv)
            except Exception as e:
                print(f"Error restarting application: {e}")
            break
        elif simulation:
            # run simulation
            print(f"Launching simulation: {simulation}")
            try:
                subprocess.run([sys.executable, str(simulation)])
            except Exception as e:
                print(f"Error running simulation: {e}")
            # continue loop to restart menu after simulation
        else:
            # normal exit
            break

```

```
447. if __name__ == "__main__":
448.     main()
449.
```

6.1.3 config_manager.py

```
  1. import json
  2. import pathlib
  3.
  4. class ConfigManager:
  5.     def __init__(self, config_path=None):
  6.         if config_path is None:
  7.             # default to config.json in parent directory
  8.             current_file = pathlib.Path(__file__).resolve()
  9.             config_path = current_file.parent / "config.json"
 10.
 11.         self.config_path = pathlib.Path(config_path)
 12.         self._config = {}
 13.         self.load()
 14.
 15.         # fixed screen size
 16.         self.SCREEN_WIDTH = 1280
 17.         self.SCREEN_HEIGHT = 720
 18.
 19.     def load(self):
 20.         try:
 21.             with open(self.config_path, 'r') as f:
 22.                 self._config = json.load(f)
 23.             print(f"Configuration loaded from {self.config_path}")
 24.         except FileNotFoundError:
 25.             print(f"Config file not found: {self.config_path}")
 26.             print("Using default configuration")
 27.             self._create_default_config()
 28.         except json.JSONDecodeError as e:
 29.             print(f"Error parsing config file: {e}")
 30.             print("Using default configuration")
 31.             self._create_default_config()
 32.
 33.     def save(self):
 34.         try:
 35.             with open(self.config_path, 'w') as f:
 36.                 json.dump(self._config, f, indent=2)
 37.             print(f"Configuration saved to {self.config_path}")
 38.             return True
 39.         except Exception as e:
 40.             print(f"Error saving config: {e}")
 41.             return False
 42.
 43.     def _create_default_config(self):
 44.         self._config = {
 45.             "display": {
 46.                 "fps": 120,
 47.                 "physics_fps": 240
 48.             },
 49.             "physics": {
 50.                 "gravity": 9.81,
 51.                 "pixels_per_meter": 100,
 52.                 "time_scale": 1.0
 53.             },
 54.             "colors": {
 55.                 "WHITE": [255, 255, 255],
 56.                 "BLACK": [0, 0, 0],
 57.                 "RED": [255, 0, 0],
 58.                 "GREEN": [0, 255, 0],
 59.                 "BLUE": [0, 0, 255],
 60.                 "GRAY": [128, 128, 128],
 61.                 "DARK_GRAY": [64, 64, 64]
```

```
62.             },
63.         }
64.         self.save()
65.
66.     def get(self, *keys, default=None):
67.         value = self._config
68.         for key in keys:
69.             if isinstance(value, dict) and key in value:
70.                 value = value[key]
71.             else:
72.                 return default
73.         return value
74.
75.     def set(self, *keys, value):
76.         if len(keys) == 0:
77.             return False
78.
79.         config = self._config
80.         for key in keys[:-1]:
81.             if key not in config:
82.                 config[key] = {}
83.             config = config[key]
84.
85.         config[keys[-1]] = value
86.         return True
87.
88.     @property
89.     def FPS(self):
90.         return self.get("display", "fps")
91.
92.     @FPS.setter
93.     def FPS(self, value):
94.         self.set("display", "fps", value=value)
95.
96.     @property
97.     def PHYSICS_FPS(self):
98.         return self.get("display", "physics_fps")
99.
100.    @PHYSICS_FPS.setter
101.    def PHYSICS_FPS(self, value):
102.        self.set("display", "physics_fps")
103.
104.    @property
105.    def GRAVITY(self):
106.        return self.get("physics", "gravity")
107.
108.    @GRAVITY.setter
109.    def GRAVITY(self, value):
110.        self.set("physics", "gravity")
111.
112.    @property
113.    def PIXELS_PER_METER(self):
114.        return self.get("physics", "pixels_per_meter")
115.
116.    @PIXELS_PER_METER.setter
117.    def PIXELS_PER_METER(self, value):
118.        self.set("physics", "pixels_per_meter")
119.
120.    @property
121.    def TIME_SCALE(self):
122.        return self.get("physics", "time_scale")
123.
124.    @TIME_SCALE.setter
125.    def TIME_SCALE(self, value):
126.        self.set("physics", "time_scale", value=value)
127.
128.    @property
129.    def COLORS(self):
130.        colors = self.get("colors", default={})
```

```
131.         # convert lists to tuples for pygame compatibility
132.         return {k: tuple(v) if isinstance(v, list) else v for k, v in colors.items()}
133.
134. # create a global instance
135. _config_manager = ConfigManager()
136.
137. # export values from json
138. SCREEN_WIDTH = _config_manager.SCREEN_WIDTH
139. SCREEN_HEIGHT = _config_manager.SCREEN_HEIGHT
140. FPS = _config_manager.FPS
141. PHYSICS_FPS = _config_manager.PHYSICS_FPS
142. GRAVITY = _config_manager.GRAVITY
143. PIXELS_PER_METER = _config_manager.PIXELS_PER_METER
144. TIME_SCALE = _config_manager.TIME_SCALE
145. COLORS = _config_manager.COLORS
146.
147. def get_config_manager():
148.     return _config_manager
149.
```

6.2 Simulations

6.2.0 Mymouse.py

```
 1. import pygame
 2.
 3. class Mouse:
 4.
 5.     def __init__(self):
 6.         self.draggable_object    = None
 7.         self.draggable_object_id = None
 8.
 9.     # -- Button state --
10.
11.     def is_lmb_clicked(self):
12.         return pygame.mouse.get_pressed(num_buttons=3)[0]
13.
14.     def is_rmb_clicked(self):
15.         return pygame.mouse.get_pressed(num_buttons=3)[2]
16.
17.     def is_mwb_clicked(self):
18.         return pygame.mouse.get_pressed(num_buttons=3)[1]
19.
20.     # -- Position --
21.
22.     def get_mouse_pos(self):
23.         return pygame.mouse.get_pos()
24.
```

6.2.1 Friction simulation

6.2.1.1 Friction.py

```
1. import math
2. import sys
3. import pathlib
4. # -- Source files --
5. current_dir = pathlib.Path(__file__).resolve()
6. parent_dir  = current_dir.parent.parent
7.
8. sys.path.insert(0, str(parent_dir))
```

```
 9.
10. # Use config_manager
11. from config_manager import get_config_manager
12. cfg = get_config_manager()
13.
14. class FrictionManager:
15.
16.     # -- Init --
17.
18.     def __init__(self, plane, plane_angle, moving_object, friction_coefficient):
19.         self.plane  = plane
20.         self.plane_angle    = plane_angle   # degrees
21.         self.moving_object  = moving_object
22.         self.friction_coefficient = friction_coefficient
23.
24.         self.angle_rad = math.radians(self.plane_angle)
25.
26.     # -- Utility --
27.
28.     def _gravity_components(self):
29.         weight = self.moving_object.mass * cfg.GRAVITY
30.         return (
31.             weight * math.sin(self.angle_rad),  # pulls object down the slope
32.             weight * math.cos(self.angle_rad),  # presses object into slope
33.         )
34.
35.     def get_velocity_component_along_plane(self, vel_x, vel_y):
36.         plane_dir_x = math.cos(self.angle_rad)
37.         plane_dir_y = math.sin(self.angle_rad)
38.         return vel_x * plane_dir_x + vel_y * plane_dir_y
39.
40.     def calculate_forces(self):
41.         gravity_parallel, gravity_perp = self._gravity_components()
42.         normal_force = gravity_perp
43.         max_friction = self.friction_coefficient * normal_force
44.
45.         vel_x = self.moving_object.get_velocity_x()
46.         vel_y = self.moving_object.get_velocity_y()
47.         velocity_along_plane = self.get_velocity_component_along_plane(vel_x, vel_y)
48.
49.         velocity_threshold = 0.1  # m/s — below this considered stopped
50.
51.         # determine friction force
52.         if abs(velocity_along_plane) > velocity_threshold:
53.             # object is moving — friction opposes motion
54.             friction_force = -math.copysign(max_friction, velocity_along_plane)
55.
56.         elif abs(velocity_along_plane) <= velocity_threshold and gravity_parallel != 0.0:
57.             # object is stopped — friction opposes gravity
58.             if abs(gravity_parallel) <= max_friction:
59.                 # friction can hold object in place
60.                 self.moving_object.set_velocity_x(0.0)
61.                 self.moving_object.set_velocity_y(0.0)
62.                 friction_force = -gravity_parallel  # Cancel gravity exactly
63.             else:
64.                 # gravity overcomes friction — object will slide
65.                 friction_force = -math.copysign(max_friction, gravity_parallel)
66.
67.         else:
68.             # flat plane, no velocity, no gravity component
69.             return 0.0, 0.0
70.
71.         # calculate net force
72.         net_force_along_plane = gravity_parallel + friction_force
73.
74.         # prevent oscillation when nearly stopped
75.         if abs(velocity_along_plane) < velocity_threshold:
76.             if gravity_parallel != 0.0 and (
77.                 math.copysign(1.0, net_force_along_plane) !=
```

```
78.                 math.copysign(1.0, gravity_parallel)
79.             ):
80.                 net_force_along_plane = 0.0
81.
82.         # convert to world coordinates
83.         force_x = net_force_along_plane * math.cos(self.angle_rad)
84.         force_y = net_force_along_plane * math.sin(self.angle_rad)
85.         return force_x, force_y
86.
```

6.2.1.2 Main.py

```
 1. # -- Libraries --
 2. import pygame
 3. import pygame.gfxdraw
 4. import sys
 5. import math
 6. import time
 7. import pathlib
 8.
 9. # -- Source files --
10. current_dir = pathlib.Path(__file__).resolve()
11. parent_dir  = current_dir.parent.parent
12.
13. sys.path.insert(0, str(parent_dir))
14.
15. # Use config_manager
16. from config_manager import get_config_manager
17. cfg = get_config_manager()
18.
19. import objects
20. import mymouse
21. import friction
22. import ui
23.
24. pygame.init()
25.
26. # -- Helpers --
27.
28. def get_closest_point_on_line(point, line, offset=30):
29.     A = list(line[0])
30.     B = list(line[1])
31.     v = [A[0] - B[0], A[1] - B[1]]
32.     u = [point[0] - A[0], point[1] - A[1]]
33.
34.     dot_uv  = v[0] * u[0] + v[1] * u[1]
35.     v_squared = v[0] ** 2 + v[1] ** 2
36.    t = dot_uv / v_squared
37.
38.     return [
39.         (A[0] + t * v[0]) + math.sin(math.radians(cfg.ANGLE)) * offset,
40.         (A[1] + t * v[1]) - math.cos(math.radians(cfg.ANGLE)) * offset,
41.     ]
42.
43. def set_object_on_plane(plane, moving_object, mouse=None):
44.     point    = mouse.get_mouse_pos() if mouse is not None else moving_object.get_center()
45.     top_edge  = plane.get_edges()[0]
46.     point_on_line  = get_closest_point_on_line(point, top_edge,
47.                                                offset=moving_object.get_height() / 2)
48.     moving_object.set_center(point_on_line)
49.
50. def check_border_collision(x, y, plane):
51.     line  = plane.get_edges()[0]
52.     A, B = line
53.     v  = [B[0] - A[0], B[1] - A[1]]
```

```
    u   = [x - A[0],    y - A[1]]
    v_squared = v[0] ** 2 + v[1] ** 2

    if v_squared == 0:
        return "none"

    t = (v[0] * u[0] + v[1] * u[1]) / v_squared

    if t < 0:
        return "left"
    if t > 1:
        return "right"
    return "none"

def set_object_before_plane(border, plane, moving_object):
    if border == "none":
        return

    corners = plane.get_rotated_corners()
    line    = (corners[0], corners[1])

    # Flip: "left" overflow -> place at right end (corners[0]) and vice-versa
    ref_point = corners[1] if border == "left" else corners[0]
    point_on_line = get_closest_point_on_line(ref_point, line,
                                              offset=moving_object.get_height() / 2)
    moving_object.set_center(point_on_line)

# -- Close Button --
class CloseButton:

    def __init__(self, x, y, width, height):
        self.rect = pygame.Rect(x, y, width, height)
        self.hovered = False
        self.font = pygame.font.Font(None, 24)

        self.bg_color = [98, 114, 164]
        self.hover_color = [139, 157, 195]
        self.text_color = [248, 248, 242]

    def handle_event(self, event):
        if event.type == pygame.MOUSEMOTION:
            self.hovered = self.rect.collidepoint(event.pos)
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.hovered:
                return True
        return False

    def draw(self, screen):
        color = self.hover_color if self.hovered else self.bg_color
        pygame.draw.rect(screen, color, self.rect, border_radius=6)
        pygame.draw.rect(screen, self.text_color, self.rect, 2, border_radius=6)

        text_surf = self.font.render("Close (Return to Menu)", True, self.text_color)
        text_rect = text_surf.get_rect(center=self.rect.center)
        screen.blit(text_surf, text_rect)

# -- Main --

def main():

    # -- Window --
    screen = pygame.display.set_mode((cfg.SCREEN_WIDTH, cfg.SCREEN_HEIGHT))
    pygame.display.set_caption("Friction Simulation")
    clock = pygame.time.Clock()

    # -- Elasticity settings --
    cfg.FRICTION_COEFFICIENT = 0.3 # Friction between object and a plane
    cfg.ANGLE = 0.0 # Angle of a plane

```

```
123.     # -- Scene objects --
124.     plane = objects.StaticObject(
125.         height=0.5,
126.         width=cfg.SCREEN_HEIGHT / cfg.PIXELS_PER_METER,
127.         color=cfg.COLORS["BLACK"],
128.         pos_x=cfg.SCREEN_WIDTH  / 2,
129.         pos_y=cfg.SCREEN_HEIGHT / 2,
130.     )
131.     plane.rotate_by(cfg.ANGLE)
132.
133.     diameter  = 0.5   # metres
134.     moving_object  = objects.MovingObject(
135.         color=cfg.COLORS["WHITE"],
136.         pos_x=cfg.SCREEN_WIDTH  / 2,
137.         pos_y=cfg.SCREEN_HEIGHT / 2 - diameter * cfg.PIXELS_PER_METER,
138.         mass=10,
139.         radius=diameter/2,
140.     )
141.
142.     static_objects = pygame.sprite.Group()
143.     moving_objects = pygame.sprite.Group()
144.     static_objects.add(plane)
145.     moving_objects.add(moving_object)
146.
147.     edit_list  = [moving_object, plane]
148.     edges_list = [plane]
149.
150.     # -- Subsystems --
151.     friction_manager = friction.FrictionManager(
152.         plane=plane,
153.         plane_angle=cfg.ANGLE,
154.         moving_object=moving_object,
155.         friction_coefficient=cfg.FRICTION_COEFFICIENT,
156.     )
157.
158.     mouse     = mymouse.Mouse()
159.     edit_menu = ui.EditMenu()
160.
161.     # Close button
162.     close_button = CloseButton(10, 10, 220, 40)
163.
164.     # -- Simulation state --
165.     simulation_started = False
166.     simulation_paused = True
167.
168.     physics_timestep = 1.0 / cfg.PHYSICS_FPS
169.     accumulator = 0.0
170.     last_time = time.perf_counter()
171.
172.     fps_counter = 0
173.     fps_timer = time.time()
174.     current_fps = 0
175.
176.     # -- Local helpers --
177.
178.     def reset_simulation():
179.         nonlocal simulation_started, simulation_paused
180.         simulation_started = False
181.         simulation_paused  = True
182.         moving_object.set_velocity_x(0)
183.         moving_object.set_velocity_y(0)
184.         moving_object.distance_travelled = 0
185.         print("Simulation reset")
186.
187.     # -- Main loop --
188.
189.     run = True
190.     while run:
191.
```

```
192.         # -- Timing --
193.         current_time = time.perf_counter()
194.         dt = current_time - last_time
195.         last_time = current_time
196.
197.         if simulation_started and not simulation_paused:
198.             accumulator += dt * cfg.TIME_SCALE
199.         else:
200.             accumulator = 0.0
201.
202.         # -- Events --
203.         for event in pygame.event.get():
204.             if event.type == pygame.QUIT:
205.                 run = False
206.
207.             # Close button takes priority
208.             if close_button.handle_event(event):
209.                 print("Returning to main menu...")
210.                 run = False
211.                 continue
212.
213.             # Edit menu takes priority
214.             menu_result = edit_menu.handle_event(event)
215.
216.             if menu_result == "apply_changes":
217.                 # Reset simulation when settings change (includes plane parameters)
218.                 if edit_menu.menu_mode == "settings":
219.                     print("Updating plane from settings...")
220.                     plane.rotation_angle = 0
221.                     plane.rotate_by(cfg.ANGLE)
222.
223.                     friction_manager.plane_angle = cfg.ANGLE
224.                     friction_manager.angle_rad   = math.radians(cfg.ANGLE)
225.                     friction_manager.friction_coefficient = cfg.FRICTION_COEFFICIENT
226.
227.                     set_object_on_plane(plane, moving_object)
228.                     reset_simulation()
229.                     print("Plane updated, simulation reset")
230.
231.                 accumulator = 0.0
232.                 continue
233.
234.             if menu_result:
235.                 continue
236.
237.             # -- Right-click -> open edit menu --
238.             if event.type == pygame.MOUSEBUTTONDOWN and event.button == 3:
239.                 mouse_pos     = pygame.mouse.get_pos()
240.
241.                 # if moving object is clicked -> open menu for moving object
242.                 if moving_object.click_hitbox.collidepoint(mouse_pos):
243.                     edit_menu.show(moving_object)
244.                 # if empty space is clicked -> open general menu for simulation
245.                 else:
246.                     edit_menu.show()
247.
248.             # -- Keyboard --
249.             if event.type == pygame.KEYDOWN:
250.                 if event.key == pygame.K_ESCAPE:
251.                     edit_menu.hide()
252.
253.                 if event.key == pygame.K_f:
254.                     simulation_paused  = not simulation_paused
255.                     simulation_started = not simulation_started
256.
257.                 if event.key == pygame.K_r:
258.                     reset_simulation()
259.
260.         # -- Drag-and-drop --
```

```
261.         if mouse.is_lmb_clicked() and simulation_paused:
262.             mouse_pos = mouse.get_mouse_pos()
263.
264.             if mouse.draggable_object is None:
265.                 # Try to pick up an object under the cursor
266.                 if moving_object.click_hitbox.collidepoint(mouse_pos):
267.                     mouse.draggable_object = moving_object
268.             else:
269.                 # Move the held object to the cursor, then snap it onto the plane
270.                 mouse.draggable_object.set_center(mouse_pos)
271.                 border = check_border_collision(*mouse_pos, plane)
272.                 if border != "none":
273.                     set_object_before_plane(border, plane, mouse.draggable_object)
274.                 else:
275.                     set_object_on_plane(plane, mouse.draggable_object)
276.
277.         elif not mouse.is_lmb_clicked() and simulation_paused:
278.             # Drop: sync logical position if an object was being dragged
279.             if mouse.draggable_object is not None:
280.                 mouse.draggable_object.sync_logical_to_visual()
281.                 mouse.draggable_object.set_velocity_x(0)
282.                 mouse.draggable_object.set_velocity_y(0)
283.             mouse.draggable_object = None
284.
285.         # -- Physics --
286.         physics_steps = 0
287.         while accumulator >= physics_timestep and physics_steps < 1:
288.             if simulation_started and not simulation_paused:
289.                 x, y   = moving_object.get_center()
290.                 border = check_border_collision(x, y, plane)
291.                 if border != "none":
292.                     set_object_before_plane(border, plane, moving_object)
293.
294.                 # Clamp near-zero motion to a full stop
295.                 if (abs(moving_object.get_acceleration_x()) < 0.1
296.                         and abs(moving_object.get_acceleration_y()) < 0.1
297.                         and abs(moving_object.get_velocity_x())     < 0.1
298.                         and abs(moving_object.get_velocity_y())     < 0.1):
299.                     moving_object.set_velocity_x(0)
300.                     moving_object.set_velocity_y(0)
301.
302.                 force_x, force_y = friction_manager.calculate_forces()
303.                 moving_object.resolve_forces((force_x, force_y))
304.                 moving_object.update_physics(physics_timestep)
305.
306.             accumulator   -= physics_timestep
307.             physics_steps += 1
308.
309.         # -- Rendering --
310.         screen.fill(cfg.COLORS["WHITE"])
311.
312.         plane.draw(screen)
313.         moving_object.draw(screen)
314.
315.         # Plane edges
316.         for static_obj in edges_list:
317.             for edge in static_obj.edges:
318.                 pygame.draw.line(screen, (255, 0, 255), edge[0], edge[1], 2)
319.
320.         # Draw click hitboxes (debug)
321.         pygame.draw.rect(screen, cfg.COLORS["GREEN"], moving_object.click_hitbox, 1)
322.
323.         # Velocity arrow
324.         center = moving_object.get_center()
325.         pygame.draw.circle(screen, (0, 0, 255), (int(center[0]), int(center[1])), 5)
326.
327.         vel_x, vel_y = moving_object.get_velocity_x(), moving_object.get_velocity_y()
328.         pygame.draw.line(
329.             screen, (0, 0, 255),
```

```
330.             center,
331.             (center[0] + vel_x * 10, center[1] + vel_y * 10),
332.             2,
333.         )
334.
335.         # Edit menu overlay
336.         edit_menu.draw(screen)
337.
338.         # Close button
339.         close_button.draw(screen)
340.
341.         # -- HUD --
342.         font      = pygame.font.Font(None, 24)
343.         sim_state = ("RUNNING" if simulation_started and not simulation_paused
344.                      else "PAUSED")
345.         hud_lines = [
346.             f"Graphics FPS: {current_fps}/{cfg.FPS}",
347.             f"Physics FPS: {cfg.PHYSICS_FPS}",
348.             f"Distance travelled: {moving_object.distance_travelled:.3f} m",
349.             f"Velocity X: {moving_object.get_velocity_x():.2f} m/s",
350.             f"Velocity Y: {moving_object.get_velocity_y():.2f} m/s",
351.             f"Acceleration X: {moving_object.acceleration_x:.2f} m/s²",
352.             f"Acceleration Y: {moving_object.acceleration_y:.2f} m/s²",
353.             f"Mass: {moving_object.mass} kg",
354.             f"Plane angle: {cfg.ANGLE}°",
355.             f"Friction coefficient: {cfg.FRICTION_COEFFICIENT}",
356.             f"Simulation: {sim_state}",
357.         ]
358.         for i, line in enumerate(hud_lines):
359.             surf = font.render(line, True, cfg.COLORS["GREEN"])
360.             screen.blit(surf, (10, 300 + i * 25))
361.
362.
363.         # -- FPS counter --
364.         fps_counter += 1
365.         if time.time() - fps_timer >= 1.0:
366.             current_fps = fps_counter
367.             fps_counter = 0
368.             fps_timer   = time.time()
369.
370.         pygame.display.flip()
371.         clock.tick(cfg.FPS)
372.
373.     pygame.quit()
374.
375. if __name__ == "__main__":
376.     main()
377.
```

6.2.1.3 Objects.py

```
 1. import pygame
 2. import sys
 3. import pathlib
 4. # -- Source files --
 5. current_dir = pathlib.Path(__file__).resolve()
 6. parent_dir  = current_dir.parent.parent
 7.
 8. sys.path.insert(0, str(parent_dir))
 9.
10. # use config_manager
11. from config_manager import get_config_manager
12. cfg = get_config_manager()
13. import math
14.
15. class Entity(pygame.sprite.Sprite):
16.     def __init__(self, height, width):
```

```
17.         pygame.sprite.Sprite.__init__(self)
18.         self.height = height * cfg.PIXELS_PER_METER
19.         self.width  = width  * cfg.PIXELS_PER_METER
20.
21.     def get_height(self): return self.height
22.     def get_width(self):  return self.width
23.
24. class StaticObject(Entity):
25.     def __init__(self, height, width, color, pos_x, pos_y):
26.         super().__init__(height, width)
27.
28.         self.pos_x = pos_x
29.         self.pos_y = pos_y
30.         self.color = color
31.         self.rotation_angle = 0
32.
33.         # pygame sprite requires image + rect even when drawing manually
34.         self.image = pygame.Surface((int(self.width), int(self.height)), pygame.SRCALPHA)
35.         self.image.fill((0, 0, 0, 0))
36.         self.rect  = self.image.get_rect(center=(pos_x, pos_y))
37.
38.         self.rotated_corners = None
39.         self.edges = self._compute_edges()
40.
41.     # -- Geometry --
42.
43.     def _compute_edges(self):
44.         hw = self.width  / 2
45.         hh = self.height / 2
46.
47.         corners_local = [(-hw, -hh), (hw, -hh), (hw, hh), (-hw, hh)]
48.
49.         a = math.radians(self.rotation_angle)
50.         cos = math.cos(a)
51.         sin = math.sin(a)
52.
53.         self.rotated_corners = [
54.             (self.pos_x + x * cos - y * sin,
55.              self.pos_y + x * sin + y * cos)
56.             for x, y in corners_local
57.         ]
58.
59.         return [(self.rotated_corners[i], self.rotated_corners[(i + 1) % 4])
60.                 for i in range(4)]
61.
62.     def get_edges(self):           return self.edges
63.     def get_rotated_corners(self): return self.rotated_corners
64.
65.     def rotate_by(self, angle):
66.         self.rotation_angle = (self.rotation_angle + angle) % 360
67.         self.edges = self._compute_edges()
68.         self.rect.center = (self.pos_x, self.pos_y)
69.
70.     # -- Position --
71.
72.     def get_center(self): return (self.pos_x, self.pos_y)
73.
74.     def set_center(self, position):
75.         self.pos_x, self.pos_y = position
76.         self.rect.center = position
77.         self.edges = self._compute_edges()  # rebuilds rotated_corners
78.
79.     # -- Drawing --
80.
81.     def draw(self, screen):
82.         if self.rotated_corners:
83.             pygame.draw.polygon(screen, self.color,
84.                                 [(int(x), int(y)) for x, y in self.rotated_corners])
85.
```

```
 86. class MovingObject(Entity):
 87.
 88.     def __init__(self, radius, color, pos_x, pos_y, mass):
 89.         diameter = radius * 2
 90.         super().__init__(diameter, diameter)
 91.
 92.         self.mass   = mass
 93.         self.radius = radius
 94.         self.color  = color
 95.
 96.         self.velocity_x     = 0.0
 97.         self.velocity_y     = 0.0
 98.         self.acceleration_x = 0.0
 99.         self.acceleration_y = 0.0
100.
101.         self.gforce = [0.0, mass * cfg.GRAVITY]
102.
103.         self.logical_x = pos_x / cfg.PIXELS_PER_METER
104.         self.logical_y = pos_y / cfg.PIXELS_PER_METER
105.         self.pos_x     = float(pos_x)
106.         self.pos_y     = float(pos_y)
107.
108.         self.distance_travelled = 0.0
109.
110.         # for pygame sprite
111.         self.image = pygame.Surface((int(self.width), int(self.height)), pygame.SRCALPHA)
112.         self.image.fill((0, 0, 0, 0))
113.         self.rect = self.image.get_rect(center=(pos_x, pos_y))
114.
115.         self.click_hitbox = self._make_hitbox()
116.
117.     # -- Hitbox --
118.
119.     def _make_hitbox(self):
120.         r_px = self.radius * cfg.PIXELS_PER_METER
121.         side = r_px * math.sqrt(2) # largest square inscribed in circle
122.         return pygame.Rect(
123.             self.pos_x - side / 2,
124.             self.pos_y - side / 2,
125.             side, side,
126.         )
127.
128.     # -- Position helpers --
129.
130.     def get_center(self): return (self.pos_x, self.pos_y)
131.
132.     def set_center(self, position):
133.         self.pos_x, self.pos_y = float(position[0]), float(position[1])
134.         self.rect.center       = (self.pos_x, self.pos_y)
135.         self.logical_x         = self.pos_x / cfg.PIXELS_PER_METER
136.         self.logical_y         = self.pos_y / cfg.PIXELS_PER_METER
137.         self.click_hitbox      = self._make_hitbox()
138.
139.     def get_logical_position(self): return (self.logical_x, self.logical_y)
140.
141.     def sync_logical_to_visual(self):
142.         self.logical_x = self.pos_x / cfg.PIXELS_PER_METER
143.         self.logical_y = self.pos_y / cfg.PIXELS_PER_METER
144.
145.     # -- Physics --
146.
147.     def resolve_forces(self, *forces):
148.         fx, fy = 0.0, 0.0
149.         for f in forces:
150.             fx += f[0]
151.             fy += f[1]
152.         self.acceleration_x = fx / self.mass
153.         self.acceleration_y = fy / self.mass
154.
```

```
155.     def update_physics(self, dt):
156.         prev_x, prev_y  = self.logical_x, self.logical_y
157.
158.         self.logical_x += (self.velocity_x + 0.5 * self.acceleration_x * dt) * dt # suvat
equation: s=v*t+0.5*a*t^2
159.         self.logical_y += (self.velocity_y + 0.5 * self.acceleration_y * dt) * dt
160.
161.         self.velocity_x += self.acceleration_x * dt
162.         self.velocity_y += self.acceleration_y * dt
163.
164.         self.pos_x = self.logical_x * cfg.PIXELS_PER_METER
165.         self.pos_y = self.logical_y * cfg.PIXELS_PER_METER
166.
167.         dx = self.logical_x - prev_x # meters
168.         dy = self.logical_y - prev_y # meters
169.         self.distance_travelled += math.sqrt(dx * dx + dy * dy) # meters
170.
171.         self.set_center((self.pos_x, self.pos_y))
172.
173.     def set_velocity_along_plane(self, magnitude, angle_degrees):
174.         angle_rad = math.radians(angle_degrees)
175.         self.velocity_x = magnitude * math.cos(angle_rad)
176.         self.velocity_y = magnitude * math.sin(angle_rad)
177.
178.     # -- Getters / setters --
179.
180.     def get_mass(self): return self.mass
181.     def get_velocity_x(self): return self.velocity_x
182.     def get_velocity_y(self): return self.velocity_y
183.     def get_acceleration_x(self): return self.acceleration_x
184.     def get_acceleration_y(self): return self.acceleration_y
185.     def set_velocity_x(self, v): self.velocity_x = v
186.     def set_velocity_y(self, v): self.velocity_y = v
187.     def get_gforce(self): return self.gforce
188.     def get_pos_x(self): return self.pos_x
189.     def get_pos_y(self): return self.pos_y
190.     def get_diameter(self): return self.radius * 2
191.
192.     # -- Drawing --
193.
194.     def draw(self, screen):
195.         r_px = int(self.radius * cfg.PIXELS_PER_METER)
196.         cx = int(self.pos_x)
197.         cy = int(self.pos_y)
198.         pygame.draw.circle(screen, cfg.COLORS["BLACK"], (cx, cy), r_px, 3)
199.         pygame.draw.circle(screen, cfg.COLORS["BLACK"], (cx, cy), r_px * 0.75, 3)
200.
```

6.2.1.4 Ui.py

```
 1. import pygame
 2. import math
 3. import sys
 4. import pathlib
 5.
 6. # -- Source files --
 7. current_dir = pathlib.Path(__file__).resolve()
 8. parent_dir  = current_dir.parent.parent
 9.
10. sys.path.insert(0, str(parent_dir))
11.
12. # Use config_manager
13. from config_manager import get_config_manager
14. cfg = get_config_manager()
15.
16. class EditMenu:
17.
```

```
18.     # -- Init --
19.
20.     def __init__(self):
21.         self.width   = 300
22.         self.padding = 20
23.         self.visible = False
24.
25.         self.target_object = None
26.         self.menu_mode = None   # "settings" | "moving_object" | "plane"
27.
28.         # Fonts
29.         self.title_font = pygame.font.Font(None, 32)
30.         self.label_font = pygame.font.Font(None, 24)
31.         self.input_font = pygame.font.Font(None, 22)
32.
33.         # Colour palette
34.         self.bg_color      = (40,  42,  54)
35.         self.text_color    = (248, 248, 242)
36.         self.input_bg      = (68,  71,  90)
37.         self.input_active  = (98,  114, 164)
38.         self.border_color  = (98,  114, 164)
39.
40.         # Input fields
41.         self.fields       = []
42.         self.active_field = None
43.
44.         self.update_position()
45.
46.     # -- Layout --
47.
48.     def update_position(self):
49.         self.x      = cfg.SCREEN_WIDTH - self.width
50.         self.y      = 0
51.         self.height = cfg.SCREEN_HEIGHT
52.
53.     def get_field_rect(self, field):
54.         return pygame.Rect(
55.             self.x + self.padding,
56.             field['y'] + 25,
57.             self.width - self.padding * 2,
58.             35,
59.         )
60.
61.     # -- Visibility --
62.
63.     def show(self, target_object=None):
64.
65.         from objects import MovingObject
66.
67.         self.target_object = target_object
68.         self.visible = True
69.         self.fields = []
70.         self.active_field = None
71.         self.update_position()
72.
73.         if target_object is None:
74.             self.menu_mode = "settings"
75.             self._setup_settings_fields()
76.         elif isinstance(target_object, MovingObject):
77.             self.menu_mode = "moving_object"
78.             self._setup_moving_object_fields()
79.         else:
80.             # If it's a StaticObject (plane), show settings instead
81.             self.menu_mode = "settings"
82.             self._setup_settings_fields()
83.
84.     def hide(self):
85.         self.visible = False
86.         self.target_object = None
```

```
87.         self.active_field = None
88.
89.     # -- Field builders --
90.
91.     def _setup_settings_fields(self):
92.         y = 100
93.         spacing = 80
94.
95.         for name, label, value in [
96.             ('time_scale', 'Time Scale', cfg.TIME_SCALE),
97.             ('plane_angle', 'Plane Angle (deg)', cfg.ANGLE),
98.             ('friction_coefficient', 'Friction Coefficient', cfg.FRICTION_COEFFICIENT),
99.         ]:
100.             self.fields.append({
101.                 'name': name, 'label': label,
102.                 'value': str(round(value, 3)),
103.                 'y': y, 'editable': True,
104.             })
105.             y += spacing
106.
107.     def _setup_moving_object_fields(self):
108.         obj     = self.target_object
109.         y       = 100
110.         spacing = 80
111.
112.         vel_magnitude = math.hypot(obj.get_velocity_x(), obj.get_velocity_y())
113.
114.         for name, label, value in [
115.             ('mass', 'Mass (kg)', obj.get_mass()),
116.             ('velocity_magnitude', 'Velocity (m/s)', vel_magnitude),
117.         ]:
118.             self.fields.append({
119.                 'name': name, 'label': label,
120.                 'value': str(round(value, 3)),
121.                 'y': y, 'editable': True,
122.             })
123.             y += spacing
124.
125.     # -- Event handling --
126.
127.     def handle_event(self, event):
128.
129.         if not self.visible:
130.             return False
131.
132.         if event.type == pygame.MOUSEBUTTONDOWN:
133.             self.active_field = None
134.             for field in self.fields:
135.                 if field['editable'] and
self.get_field_rect(field).collidepoint(event.pos):
136.                     self.active_field = field
137.                     return True
138.
139.         elif event.type == pygame.KEYDOWN:
140.             if event.key == pygame.K_ESCAPE:
141.                 self.hide()
142.                 return "hide"
143.
144.             if self.active_field:
145.                 if event.key == pygame.K_RETURN:
146.                     self.apply_changes()
147.                     self.active_field = None
148.                     return "apply_changes"
149.                 elif event.key == pygame.K_BACKSPACE:
150.                     self.active_field['value'] = self.active_field['value'][:-1]
151.                 elif event.unicode in '0123456789.-':
152.                     self.active_field['value'] += event.unicode
153.                 return True
154.
```

```
155.         return False
156.
157.     # -- Apply --
158.
159.     def apply_changes(self):
160.         try:
161.             if self.menu_mode == "settings":
162.                 for field in self.fields:
163.                     value = float(field['value'])
164.
165.                     if field['name'] == 'time_scale' and value > 0:
166.                         cfg.TIME_SCALE = value
167.                         print(f"Time scale -> {value}")
168.
169.                     elif field['name'] == 'plane_angle':
170.                         # Clamp value
171.                         if value > 90: value = 90
172.                         elif value < -90: value = -90
173.                         cfg.ANGLE = value
174.                         print(f"Plane angle -> {value}")
175.
176.                     elif field['name'] == 'friction_coefficient':
177.                         # Clamp value
178.                         if value < 0: value = 0
179.                         elif value > 1: value = 1
180.                         cfg.FRICTION_COEFFICIENT = value
181.                         print(f"Friction coefficient -> {value}")
182.
183.             elif self.menu_mode == "moving_object":
184.                 obj = self.target_object
185.                 for field in self.fields:
186.                     value = float(field['value'])
187.
188.                     if field['name'] == 'mass' and value > 0:
189.                         obj.mass    = value
190.                         obj.gforce  = [0, value * cfg.GRAVITY]
191.
192.                     elif field['name'] == 'velocity_magnitude':
193.                         obj.set_velocity_along_plane(value, cfg.ANGLE)
194.
195.             print(f"Changes applied to {self.menu_mode}")
196.
197.         except ValueError as exc:
198.             print(f"Error applying changes: {exc}")
199.
200.     # -- Drawing --
201.
202.     def draw(self, screen):
203.         if not self.visible:
204.             return
205.
206.         # Panel background
207.         pygame.draw.rect(screen, self.bg_color,
208.                          (self.x, self.y, self.width, self.height))
209.         pygame.draw.line(screen, self.border_color,
210.                          (self.x, self.y), (self.x, self.height), 3)
211.
212.         # Title
213.         titles = {
214.             "settings":      "Settings",
215.             "moving_object": "Edit Moving Object",
216.         }
217.         title_surf = self.title_font.render(
218.             titles.get(self.menu_mode, "Menu"), True, self.text_color
219.         )
220.         screen.blit(title_surf,
221.                     (self.x + (self.width - title_surf.get_width()) // 2, 30))
222.
223.         # Input fields
```

```
224.         for field in self.fields:
225.             label_surf = self.label_font.render(field['label'], True, self.text_color)
226.             screen.blit(label_surf, (self.x + self.padding, field['y']))
227.
228.             if not field['editable']:
229.                 continue
230.
231.             field_rect = self.get_field_rect(field)
232.             bg         = self.input_active if field is self.active_field else self.input_bg
233.             pygame.draw.rect(screen, bg,               field_rect, border_radius=4)
234.             pygame.draw.rect(screen, self.border_color, field_rect, 2, border_radius=4)
235.
236.             text_surf = self.input_font.render(field['value'], True, self.text_color)
237.             text_x    = field_rect.x + 10
238.             text_y    = field_rect.y + (field_rect.height - text_surf.get_height()) // 2
239.             screen.blit(text_surf, (text_x, text_y))
240.
241.             # Blinking cursor for active field
242.             if field is self.active_field:
243.                 cursor_x = text_x + text_surf.get_width() + 2
244.                 pygame.draw.line(screen, self.text_color,
245.                                  (cursor_x, field_rect.y + 5),
246.                                  (cursor_x, field_rect.y + field_rect.height - 5), 2)
247.
248.         # Bottom instructions
249.         instructions = [
250.             "Press ENTER to save changes",
251.             "Press ESC to close menu",
252.             "",
253.             "Controls:",
254.             "RMB Click object - Edit properties",
255.             "RMB Click empty space - Settings",
256.             "F - Pause/Resume",
257.             "R - Reset simulation",
258.         ]
259.         inst_y = self.height - 220
260.         for i, line in enumerate(instructions):
261.             surf = self.input_font.render(line, True, self.text_color)
262.             screen.blit(surf,
263.                         (self.x + (self.width - surf.get_width()) // 2,
264.                          inst_y + i * 25))
265.
```

7.2.2 Rebound simulation

6.2.2.1 Collisions.py

```
import math

import pathlib

import sys


# -- Source files --

current_dir = pathlib.Path(__file__).resolve()

parent_dir  = current_dir.parent.parent


sys.path.insert(0, str(parent_dir))
```

```
import vectors as vec

# -- Geometry helpers --

def get_line_normal(point1, point2):
    dx = point2[0] - point1[0]
    dy = point2[1] - point1[1]

    normal = (-dy, dx)

    length = math.hypot(normal[0], normal[1])
    if length == 0:
        return (0, 0)
    return (normal[0] / length, normal[1] / length)

def distance_point_to_line(point, p1, p2):
    ax = point[0] - p1[0]
    ay = point[1] - p1[1]
    cx = p2[0] - p1[0]
    cy = p2[1] - p1[1]

    length_sq = cx * cx + cy * cy
    if length_sq == 0:
        return vec.distance_between_points(point, p1), p1

    t = max(0.0, min(1.0, (ax * cx + ay * cy) / length_sq))

    closest = (p1[0] + t * cx, p1[1] + t * cy)
    return vec.distance_between_points(point, closest), closest

# -- Circle–circle collisions --
```

```
def check_circle_collisions(moving_objects):
    collisions = []
    moving_list = list(moving_objects)

    for i in range(len(moving_list)):
        for j in range(i + 1, len(moving_list)):
            obj1 = moving_list[i]
            obj2 = moving_list[j]

            c1 = obj1.get_center()
            c2 = obj2.get_center()
            r1 = obj1.get_width() / 2
            r2 = obj2.get_width() / 2

            dist = vec.distance_between_points(c1, c2)

            if dist < r1 + r2:
                normal = ((c2[0] - c1[0]) / dist,
                          (c2[1] - c1[1]) / dist)

                collisions.append((obj1, obj2, normal))

    return collisions

def resolve_circle_collisions(collisions, elasticity):
    resolved_pairs = set()

    for obj1, obj2, normal in collisions:
        pair_id = tuple(sorted([id(obj1), id(obj2)]))
        if pair_id in resolved_pairs:
```

```
            continue
        resolved_pairs.add(pair_id)

        m1 = obj1.get_mass()
        m2 = obj2.get_mass()
        v1 = (obj1.get_velocity_x(), obj1.get_velocity_y())
        v2 = (obj2.get_velocity_x(), obj2.get_velocity_y())

        # velocity impulse (only if approaching)
        v_rel_along_normal = vec.dot_product(vec.vector_subtract(v1, v2), normal)
        if v_rel_along_normal <= 0:
            continue

        # J = -(1+e) * v_rel_n / (1/m1 + 1/m2)
        J = -(1 + elasticity) * v_rel_along_normal / (1 / m1 + 1 / m2)
        new_v1 = vec.vector_add(v1, vec.scalar_multiply(normal, J / m1))
        new_v2 = vec.vector_subtract(v2, vec.scalar_multiply(normal, J / m2))

        obj1.set_velocity_x(new_v1[0])
        obj1.set_velocity_y(new_v1[1])
        obj2.set_velocity_x(new_v2[0])
        obj2.set_velocity_y(new_v2[1])

# -- Circle–wall collisions --

def check_collisions(moving_objects, static_objects):
    collisions = []

    for moving_obj in moving_objects:
        center = moving_obj.get_center()
        radius = moving_obj.get_width() / 2
```

```
        for static_obj in static_objects:
            for p1, p2 in static_obj.edges:
                dist, closest = distance_point_to_line(center, p1, p2)

                if dist > radius:
                    continue

                normal = get_line_normal(p1, p2)
                to_circle = (center[0] - closest[0], center[1] - closest[1])

                # flip normal if it points away from the circle
                if vec.dot_product(normal, to_circle) < 0:
                    normal = (-normal[0], -normal[1])

                collisions.append((moving_obj, static_obj, normal))

    return collisions

def resolve_wall_collisions(collisions):
    for moving_obj, static_obj, normal in collisions:
        velocity = (moving_obj.get_velocity_x(), moving_obj.get_velocity_y())
        v_dot_n = vec.dot_product(velocity, normal)
        elasticity = static_obj.get_elasticity()

        # only process collision if moving into wall
        if v_dot_n >= 0:
            continue  # already separating, no impulse needed

        # apply impulse: Δv = -(1+e) * (v·n) * n
        J = vec.scalar_multiply(normal, -(1 + elasticity) * v_dot_n)
```

```
    new_velocity = vec.vector_add(velocity, J)


    moving_obj.set_velocity_x(new_velocity[0])
    moving_obj.set_velocity_y(new_velocity[1])
```

6.2.2.2 Main.py

```
# -- Libraries --
import pygame
import pygame.gfxdraw
import sys
import math
import time
import pathlib

# -- Source files --
current_dir = pathlib.Path(__file__).resolve()
parent_dir  = current_dir.parent.parent

sys.path.insert(0, str(parent_dir))

# Use config_manager
from config_manager import get_config_manager
cfg = get_config_manager()

import objects
import mymouse
import collisions
import ui

pygame.init()

# -- Close Button --
class CloseButton:

    def __init__(self, x, y, width, height):
        self.rect = pygame.Rect(x, y, width, height)
        self.hovered = False
        self.font = pygame.font.Font(None, 24)

        self.bg_color = [98, 114, 164]
        self.hover_color = [139, 157, 195]
        self.text_color = [248, 248, 242]

    def handle_event(self, event):
        if event.type == pygame.MOUSEMOTION:
            self.hovered = self.rect.collidepoint(event.pos)
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.hovered:
                return True
        return False

    def draw(self, screen):
        color = self.hover_color if self.hovered else self.bg_color
        pygame.draw.rect(screen, color, self.rect, border_radius=6)
        pygame.draw.rect(screen, self.text_color, self.rect, 2, border_radius=6)

        text_surf = self.font.render("Close (Return to Menu)", True, self.text_color)
        text_rect = text_surf.get_rect(center=self.rect.center)
        screen.blit(text_surf, text_rect)

def main():

```

```
57.     # -- Window --
58.     screen = pygame.display.set_mode((cfg.SCREEN_WIDTH, cfg.SCREEN_HEIGHT))
59.     pygame.display.set_caption("Collision Simulation")
60.     clock  = pygame.time.Clock()
61.
62.     # -- Collision mode --
63.     cfg.COLLISION_MODE = "2D"  # "1D" or "2D"
64.
65.     # 1D track position (center of screen)
66.     TRACK_Y = cfg.SCREEN_HEIGHT / 2
67.
68.     # -- Elasticity settings --
69.     cfg.COLLISION_ELASTICITY = 1.0  # circle-circle collision elasticity
70.     cfg.EDGE_ELASTICITY = 1.0       # wall collision elasticity
71.
72.     # -- Border walls --
73.     edge_size = 1.0   # metres
74.
75.     # calculate actual ground surface position
76.     # edge_bottom center is at cfg.SCREEN_HEIGHT
77.     # edge_bottom height is edge_size meters
78.     # top surface of ground is at center - (height/2)
79.     GROUND_SURFACE_Y = cfg.SCREEN_HEIGHT - (edge_size/2 * cfg.PIXELS_PER_METER )
80.
81.     edge_left   = objects.StaticObject(cfg.SCREEN_HEIGHT / cfg.PIXELS_PER_METER, edge_size,
82.                                        cfg.COLORS["BLACK"],
0,                   cfg.SCREEN_HEIGHT / 2,
83.                                        cfg.EDGE_ELASTICITY)
84.     edge_top    = objects.StaticObject(edge_size, cfg.SCREEN_WIDTH  / cfg.PIXELS_PER_METER,
85.                                        cfg.COLORS["BLACK"], cfg.SCREEN_WIDTH / 2, 0,
86.                                        cfg.EDGE_ELASTICITY)
87.     edge_right  = objects.StaticObject(cfg.SCREEN_HEIGHT / cfg.PIXELS_PER_METER, edge_size,
88.                                        cfg.COLORS["BLACK"],
cfg.SCREEN_WIDTH,     cfg.SCREEN_HEIGHT / 2,
89.                                        cfg.EDGE_ELASTICITY)
90.     edge_bottom = objects.StaticObject(edge_size, cfg.SCREEN_WIDTH  / cfg.PIXELS_PER_METER,
91.                                        cfg.COLORS["BLACK"], cfg.SCREEN_WIDTH / 2,
cfg.SCREEN_HEIGHT,
92.                                        cfg.EDGE_ELASTICITY)
93.
94.     static_objects = pygame.sprite.Group()
95.     for edge in (edge_left, edge_right, edge_top, edge_bottom):
96.         static_objects.add(edge)
97.
98.     edges_list = [edge_left, edge_top, edge_right, edge_bottom]
99.
100.    # -- Default Moving object --
101.    moving_object = objects.MovingObject(radius=0.25,
102.                                         color=cfg.COLORS["WHITE"],
103.                                         pos_x=300,  pos_y=200,
104.                                         elasticity=1.0,
105.                                         mass=10)
106.
107.    moving_objects = pygame.sprite.Group()
108.    moving_objects.add(moving_object)
109.
110.    edit_list = [moving_object]
111.
112.    # -- Subsystems --
113.    mouse     = mymouse.Mouse()
114.    edit_menu = ui.EditMenu()
115.    trail     = objects.TrailSystem(max_points=8000)
116.
117.    # Close button
118.    close_button = CloseButton(5, 5, 220, 40)
119.
120.    # -- Simulation state --
121.    simulation_started = True
122.    simulation_paused  = False
```

```
123.
124.     physics_timestep = 1.0 / cfg.PHYSICS_FPS
125.     accumulator = 0.0
126.     last_time = time.perf_counter()
127.
128.     fps_counter = 0
129.     fps_timer = time.time()
130.     current_fps = 0
131.
132.     # -- Local helpers --
133.
134.     def reset_simulation():
135.         # reset speed
136.         # reset trail
137.         for obj in moving_objects:
138.             obj.reset_speed()
139.             obj.sync_logical_to_visual()
140.         trail.clear()
141.
142.     # -- Main loop --
143.
144.     run = True
145.     while run:
146.
147.         # -- Timing --
148.         current_time = time.perf_counter()
149.         dt = current_time - last_time
150.         last_time = current_time
151.
152.         if simulation_started and not simulation_paused:
153.             accumulator += dt * cfg.TIME_SCALE
154.         else:
155.             accumulator = 0.0
156.
157.         # -- Events --
158.         for event in pygame.event.get():
159.             if event.type == pygame.QUIT:
160.                 run = False
161.
162.             # close button takes priority
163.             if close_button.handle_event(event):
164.                 print("Returning to main menu...")
165.                 run = False
166.                 continue
167.
168.             # edit menu takes priority
169.             menu_result = edit_menu.handle_event(event)
170.
171.             if edit_menu.menu_mode == "settings" and menu_result == "apply_changes":
172.                 old_mode = cfg.get("COLLISION_MODE", "2D")
173.                 accumulator = 0.0
174.
175.                 # update edge elasticity for all walls
176.                 for edge in edges_list:
177.                     edge.elasticity = cfg.EDGE_ELASTICITY
178.
179.                 # update gravity force for all moving objects
180.                 for obj in moving_objects:
181.                     obj.gforce = [0.0, obj.mass * cfg.GRAVITY]
182.
183.                 # handle mode switching
184.                 new_mode = cfg.COLLISION_MODE
185.                 if old_mode != new_mode:
186.                     print(f"Switching from {old_mode} to {new_mode} mode")
187.
188.                     if new_mode == "1D":
189.                         # place all objects on the track
190.                         for obj in moving_objects:
191.                             obj.set_center((obj.pos_x, TRACK_Y))
```

```
                            obj.set_velocity_y(0.0)
                            obj.sync_logical_to_visual()
                        print("Objects placed on 1D track")

                continue

            if menu_result:
                continue

            # -- Right-click -> open edit menu --
            if event.type == pygame.MOUSEBUTTONDOWN and event.button == 3:
                mouse_pos      = pygame.mouse.get_pos()
                object_clicked = False
                for obj in edit_list:
                    if obj.click_hitbox.collidepoint(mouse_pos):
                        edit_menu.show(obj)
                        object_clicked = True
                        break
                if not object_clicked:
                    edit_menu.show()

            # -- Keyboard --
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    edit_menu.hide()
                if event.key == pygame.K_f:
                    simulation_started = not simulation_started
                    simulation_paused = not simulation_paused
                if event.key == pygame.K_r:
                    reset_simulation()

                if event.key == pygame.K_a:

                    if len(moving_objects) >= 3:
                        print("Cannot add more than 3 moving objects")
                        continue

                    mx, my = mouse.get_mouse_pos()
                    new_object = objects.MovingObject(
                        radius=0.25,
                        color=cfg.COLORS["WHITE"],
                        pos_x=mx, pos_y=my,
                        elasticity=1.0,
                        mass=10
                    )
                    moving_objects.add(new_object)
                    edit_list.append(new_object)
                    print(f"Added moving object at ({mx}, {my})")

                if event.key == pygame.K_s:
                    mouse_pos = mouse.get_mouse_pos()
                    object_to_delete = None
                    for obj in edit_list:
                        if obj.click_hitbox.collidepoint(mouse_pos):
                            object_to_delete = obj
                            break

                    if object_to_delete and len(list(moving_objects)) > 1:
                        moving_objects.remove(object_to_delete)
                        edit_list.remove(object_to_delete)
                        print(f"Deleted moving object")
                    elif object_to_delete and len(list(moving_objects)) == 1:
                        print("Cannot delete the last moving object")
                        continue

        # -- Drag-and-drop --
        if mouse.is_lmb_clicked() and simulation_paused:
            mouse_pos = mouse.get_mouse_pos()
            if mouse.draggable_object is None:
```

```
261.                 for obj in edit_list:
262.                     if obj.click_hitbox.collidepoint(mouse_pos):
263.                         mouse.draggable_object    = obj
264.                         break
265.             else:
266.                 mouse.draggable_object.set_center(mouse_pos)
267.
268.         elif not mouse.is_lmb_clicked() and simulation_paused:
269.             if mouse.draggable_object is not None:
270.                 mouse.draggable_object.sync_logical_to_visual()
271.             mouse.draggable_object    = None
272.             mouse.draggable_object_id = None
273.
274.         # -- Physics (fixed timestep) --
275.         physics_steps = 0
276.         while accumulator >= physics_timestep and physics_steps < 10:
277.
278.             # Step 1: detect and resolve wall contacts (only for non-stopped objects)
279.             wall_collisions = collisions.check_collisions(moving_objects, edges_list)
280.             if wall_collisions:
281.                 collisions.resolve_wall_collisions(wall_collisions)
282.
283.             # Step 2: apply forces (only to non-stopped objects)
284.             for obj in moving_objects:
285.                 obj.acceleration_x = 0.0
286.                 obj.acceleration_y = 0.0
287.
288.                 # Only apply gravity in 2D mode
289.                 if cfg.COLLISION_MODE == "2D":
290.                     obj.resolve_forces(obj.get_gforce())
291.
292.             # Step 3: integrate positions and velocities (only for non-stopped objects)
293.             for obj in moving_objects:
294.                 obj.update_physics(physics_timestep)
295.
296.             # Step 4: ball-ball collisions (after new positions, only active objects)
297.             circle_collisions = collisions.check_circle_collisions(moving_objects)
298.             if circle_collisions:
299.                 collisions.resolve_circle_collisions(circle_collisions,
elasticity=cfg.COLLISION_ELASTICITY)
300.
301.             # Step 5: check total energy and stop objects with insufficient energy
302.             if cfg.COLLISION_MODE == "2D":
303.                 ground_y = GROUND_SURFACE_Y  # Top surface of ground edge
304.
305.                 all_objects_stopped = True
306.                 for obj in moving_objects:
307.                     # default threshold: 0.02m = 2cm equivalent height
308.                     if obj.check_should_stop(ground_y, threshold_height=0.02):
309.                         # stop the object if no rebound
310.                         obj.velocity_x = 0
311.                         obj.velocity_y = 0
312.
313.                     else:
314.                         all_objects_stopped = False
315.
316.                 if all_objects_stopped:
317.                     simulation_started = not simulation_started
318.                     simulation_paused = not simulation_paused
319.
320.             accumulator   -= physics_timestep
321.             physics_steps += 1
322.
323.         # -- Trail --
324.         if not simulation_paused:
325.             for obj in moving_objects:
326.                 trail.add_point(obj.get_center())
327.
328.         # -- Rendering --
```

```
329.         screen.fill(cfg.COLORS["WHITE"])
330.
331.         #static_objects.draw(screen)
332.         trail.draw(screen)
333.
334.         # edge outlines + outward normals (debug)
335.         for static_obj in edges_list:
336.             static_obj.draw(screen)
337.             for p1, p2 in static_obj.edges:
338.                 pygame.draw.line(screen, (255, 0, 255),
339.                                  (int(p1[0]), int(p1[1])),
340.                                  (int(p2[0]), int(p2[1])), 2)
341.                 mx, my = (p1[0] + p2[0]) / 2, (p1[1] + p2[1]) / 2
342.                 nx, ny = collisions.get_line_normal(p1, p2)
343.                 pygame.draw.line(screen, (0, 255, 0),
344.                                  (int(mx), int(my)),
345.                                  (int(mx + nx * 30), int(my + ny * 30)), 2)
346.
347.         # draw moving object overlays: centre dot, velocity arrow, hitbox
348.         for obj in moving_objects:
349.             center       = obj.get_center()
350.             vel_x, vel_y = obj.get_velocity_x(), obj.get_velocity_y()
351.
352.             obj.draw(screen)
353.             pygame.draw.circle(screen, (0, 0, 255),
354.                                (int(center[0]), int(center[1])), 5)
355.             pygame.draw.line(screen, (0, 0, 255),
356.                              center,
357.                              (center[0] + vel_x * 10, center[1] + vel_y * 10), 2)
358.             pygame.draw.rect(screen, cfg.COLORS["GREEN"], obj.click_hitbox, 1)
359.
360.         edit_menu.draw(screen)
361.
362.         # Close button
363.         close_button.draw(screen)
364.
365.         # -- HUD --
366.         font      = pygame.font.Font(None, 24)
367.         sim_state = "PAUSED" if simulation_paused else "RUNNING"
368.
369.         # calculate total mechanical energy (KE + GPE) for all objects
370.
371.         ground_y = GROUND_SURFACE_Y  # use actual ground surface position
372.
373.         hud_moving_object_lines = []
374.
375.         for obj in moving_objects:
376.             hud_moving_object_lines.append("")
377.             hud_moving_object_lines.append(f"Ball velocity: ({obj.get_velocity_x():.2f},
{obj.get_velocity_y():.2f}) m/s")
378.             hud_moving_object_lines.append(f"Ball mass: {obj.mass} kg",)
379.
380.             # total kinetic energy
381.             ke = 0.5 * obj.mass * (obj.velocity_y**2 + obj.velocity_x**2)
382.             # gravitational potential energy (relative to ground)
383.             radius_px = obj.radius * cfg.PIXELS_PER_METER
384.             ball_bottom_y = obj.pos_y + radius_px
385.             height_m = (ground_y - ball_bottom_y) / cfg.PIXELS_PER_METER
386.             gpe = obj.mass * cfg.GRAVITY * height_m
387.
388.             hud_moving_object_lines.append(f"Height: {height_m:.3f} m")
389.             hud_moving_object_lines.append(f"Total KE: {ke:.3f} J")
390.             hud_moving_object_lines.append(f"Total GPE: {gpe:.3f} J")
391.
392.         hud_lines = [
393.             f"Graphics FPS: {current_fps}/{cfg.FPS}",
394.             f"Physics FPS: {cfg.PHYSICS_FPS}",
395.             f"Time Scale: {cfg.TIME_SCALE}x",
396.             f"Simulation: {sim_state}",
```

```
397.         ]
398.         hud_lines += hud_moving_object_lines
399.
400.         for i, text in enumerate(hud_lines):
401.             surf = font.render(text, True, cfg.COLORS["GREEN"])
402.             screen.blit(surf, (edge_size/2*cfg.PIXELS_PER_METER + 10,
edge_size/2*cfg.PIXELS_PER_METER + 10 + i * 25))
403.
404.         # -- FPS counter --
405.         fps_counter += 1
406.         if time.time() - fps_timer >= 1.0:
407.             current_fps = fps_counter
408.             fps_counter = 0
409.             fps_timer   = time.time()
410.
411.         pygame.display.flip()
412.         clock.tick(cfg.FPS)
413.
414.     pygame.quit()
415.
416. if __name__ == "__main__":
417.     main()
418.
```

6.2.2.3 Objects.py

```
 1. import pygame
 2. import pygame.gfxdraw
 3. import pathlib
 4. import sys
 5. import math
 6.
 7. # -- Source files --
 8. current_dir = pathlib.Path(__file__).resolve()
 9. parent_dir  = current_dir.parent.parent
10.
11. sys.path.insert(0, str(parent_dir))
12.
13. # Use config_manager
14. from config_manager import get_config_manager
15. cfg = get_config_manager()
16.
17. class Entity(pygame.sprite.Sprite):
18.
19.     def __init__(self, height, width):
20.         pygame.sprite.Sprite.__init__(self)
21.         self.height = height * cfg.PIXELS_PER_METER
22.         self.width = width * cfg.PIXELS_PER_METER
23.
24.     def get_height(self): return self.height
25.     def get_width(self):  return self.width
26.
27. class StaticObject(Entity):
28.     def __init__(self, height, width, color, pos_x, pos_y, elasticity):
29.         super().__init__(height, width)
30.
31.         self.color = color
32.         self.elasticity = elasticity
33.         self.pos_x = pos_x
34.         self.pos_y = pos_y
35.
36.         # needed for pygames sprite groups
37.         self.image = pygame.Surface((int(self.width), int(self.height)), pygame.SRCALPHA)
38.         self.image.fill((0, 0, 0, 0))
39.         self.rect = self.image.get_rect(center=(pos_x, pos_y))
40.
41.         self.edges = self._compute_edges()
```

```
42.
43.     # -- Geometry --
44.
45.     def _compute_edges(self):
46.         hw = self.width  / 2
47.         hh = self.height / 2
48.
49.         # four corners of the axis-aligned rectangle
50.         corners = [
51.             (self.pos_x - hw, self.pos_y - hh),  # top-left
52.             (self.pos_x + hw, self.pos_y - hh),  # top-right
53.             (self.pos_x + hw, self.pos_y + hh),  # bottom-right
54.             (self.pos_x - hw, self.pos_y + hh)   # bottom-left
55.         ]
56.
57.         # return edges as pairs of consecutive corners
58.         return [(corners[i], corners[(i + 1) % 4])
59.                 for i in range(4)]
60.
61.     def get_edges(self): return self.edges
62.
63.     # -- Position --
64.
65.     def get_center(self): return (self.pos_x, self.pos_y)
66.
67.     def set_center(self, position):
68.         self.pos_x, self.pos_y = position[0], position[1]
69.         self.rect.center = position
70.         self.edges = self._compute_edges()
71.
72.     # -- Misc --
73.
74.     def get_elasticity(self): return self.elasticity
75.
76.     def set_image(self, image):
77.         self.image = pygame.transform.smoothscale(image, (int(self.width),
int(self.height)))
78.         self.edges = self._compute_edges()
79.
80.     def set_color(self, color):
81.         self.color = color
82.         self.image.fill(color)
83.
84.     def draw(self, screen):
85.         pygame.draw.rect(screen, self.color, self.rect)
86.
87. class MovingObject(Entity):
88.     def __init__(self, radius, color, pos_x, pos_y, elasticity, mass):
89.         diameter = radius * 2 # meters
90.         super().__init__(diameter, diameter) # height, width that are stored in pixels
91.
92.         self.mass   = mass
93.         self.radius = radius # metres
94.
95.         self.pos_x = pos_x
96.         self.pos_y = pos_y
97.
98.         self.color = color
99.         self.elasticity = elasticity
100.
101.         self.velocity_x = 0.0
102.         self.velocity_y = 0.0
103.         self.acceleration_x = 0.0
104.         self.acceleration_y = 0.0
105.
106.         self.gforce = [0.0, mass * cfg.GRAVITY]
107.
108.
109.         # image needed for pygame sprites
```

```
110.          self.image = pygame.Surface((int(self.width), int(self.height)), pygame.SRCALPHA)
111.          self.image.fill((0, 0, 0, 0))
112.          self.rect = self.image.get_rect(center=(pos_x, pos_y))
113.
114.          # logical position in meters (used by the physics integrator)
115.          self.logical_x = pos_x / cfg.PIXELS_PER_METER
116.          self.logical_y = pos_y / cfg.PIXELS_PER_METER
117.
118.          self.click_hitbox = self._make_hitbox()
119.
120.          self.distance_travelled = 0.0
121.
122.      # -- Hitbox --
123.
124.      def _make_hitbox(self):
125.          side = self.radius * cfg.PIXELS_PER_METER * math.sqrt(2)
126.          return pygame.Rect(self.pos_x - side / 2, self.pos_y - side / 2, side, side)
127.
128.      def set_click_hitbox(self):
129.          self.click_hitbox = self._make_hitbox()
130.
131.      # -- Position --
132.
133.      def set_center(self, position):
134.          self.pos_x, self.pos_y = position[0], position[1]
135.          self.rect.center       = position
136.          self.click_hitbox      = self._make_hitbox()
137.
138.      def get_logical_position(self): return (self.logical_x, self.logical_y)
139.
140.      def sync_logical_to_visual(self):
141.          self.logical_x = self.pos_x / cfg.PIXELS_PER_METER
142.          self.logical_y = self.pos_y / cfg.PIXELS_PER_METER
143.
144.      # -- Physics --
145.
146.      def resolve_forces(self, *forces):
147.          fx, fy = 0.0, 0.0
148.          for f in forces:
149.              fx += f[0]
150.              fy += f[1]
151.
152.          self.acceleration_x = fx / self.mass
153.          self.acceleration_y = fy / self.mass
154.
155.      def update_physics(self, dt):
156.          prev_x, prev_y  = self.logical_x, self.logical_y
157.
158.          self.logical_x += (self.velocity_x + 0.5 * self.acceleration_x * dt) * dt # suvat
equation: s=v*t+0.5*a*t^2 -> s=(v+0.5at)t
159.          self.logical_y += (self.velocity_y + 0.5 * self.acceleration_y * dt) * dt
160.
161.          self.velocity_x += self.acceleration_x * dt
162.          self.velocity_y += self.acceleration_y * dt
163.
164.          self.pos_x = self.logical_x * cfg.PIXELS_PER_METER
165.          self.pos_y = self.logical_y * cfg.PIXELS_PER_METER
166.
167.          dx = self.logical_x - prev_x # meters
168.          dy = self.logical_y - prev_y # meters
169.          self.distance_travelled += math.sqrt(dx * dx + dy * dy) # meters
170.
171.          self.set_center((self.pos_x, self.pos_y))
172.
173.
174.
175.      def reset_speed(self):
176.          self.velocity_x     = 0.0
177.          self.velocity_y     = 0.0
```

```
178.         self.acceleration_x = 0.0
179.         self.acceleration_y = 0.0
180.
181.     def check_should_stop(self, ground_y, threshold_height=0.02):
182.         # calculate total kinetic energy (only vertical component because calculation is
for rebound)
183.         total_ke_y = 0.5 * self.mass * (self.velocity_y ** 2) #(self.velocity_x ** 2 +
self.velocity_y ** 2)
184.
185.         # calculate gravitational potential energy
186.         radius_px = self.radius * cfg.PIXELS_PER_METER
187.         ball_bottom_y = self.pos_y + radius_px
188.
189.         # height above ground (reference: ground surface)
190.         height_above_ground_px = ground_y - ball_bottom_y
191.         height_above_ground_m = height_above_ground_px / cfg.PIXELS_PER_METER
192.
193.         # GPE (can be negative if ball is below ground due to penetration)
194.         gpe = self.mass * cfg.GRAVITY * height_above_ground_m
195.
196.         # total mechanical energy (ke+gpe)
197.         total_energy = total_ke_y + gpe
198.
199.         # convert total energy to equivalent height
200.         # this makes the threshold mass-independent
201.         if cfg.GRAVITY > 0 and self.mass > 0:
202.             equivalent_height = total_energy / (self.mass * cfg.GRAVITY)
203.         else:
204.             equivalent_height = float('inf')
205.
206.         # stop if total energy is too low
207.         # default: equivalent to bouncing less than 2cm (0.02m)
208.         insufficient_energy = equivalent_height < threshold_height
209.
210.         return insufficient_energy
211.
212.     def calculate_ke(self):
213.         return 0.5 * self.mass * (self.velocity_y ** 2 + self.velocity_x** 2) #
0.5*m*(sqrt(vx^2+vy^))2 -> 0.5m(vx^2+vy^2)
214.
215.     # -- Getters / Setters --
216.
217.     def get_center(self): return (self.pos_x, self.pos_y)
218.     def get_mass(self): return self.mass
219.     def get_velocity_x(self): return self.velocity_x
220.     def get_velocity_y(self): return self.velocity_y
221.     def get_acceleration_x(self): return self.acceleration_x
222.     def get_acceleration_y(self): return self.acceleration_y
223.     def set_velocity_x(self, v): self.velocity_x = v
224.     def set_velocity_y(self, v): self.velocity_y = v
225.     def set_acceleration_x(self, a): self.acceleration_x = a
226.     def set_acceleration_y(self, a): self.acceleration_y = a
227.     def get_gforce(self): return self.gforce
228.
229.     # -- Drawing --
230.
231.     def draw(self, screen):
232.         r_px = int(self.radius * cfg.PIXELS_PER_METER)
233.         cx = int(self.pos_x)
234.         cy = int(self.pos_y)
235.         pygame.draw.circle(screen, cfg.COLORS["BLACK"], (cx, cy), r_px, 3)
236.         pygame.draw.circle(screen, cfg.COLORS["BLACK"], (cx, cy), r_px * 0.75, 3)
237.
238. class TrailSystem:
239.
240.     def __init__(self, max_points=5000):
241.         self.points = set()
242.         self.max_points = max_points
243.         self.color = cfg.COLORS["RED"]
```

```
244.
245.     def add_point(self, position):
246.         self.points.add((int(position[0]), int(position[1])))
247.         if len(self.points) > self.max_points:
248.             self.points.pop()
249.
250.     def clear(self):  self.points.clear()
251.     def length(self): return len(self.points)
252.
253.     def draw(self, screen):
254.         w, h              = screen.get_width(), screen.get_height()
255.         points_to_remove = set()
256.         for point in self.points:
257.             x, y = int(point[0]), int(point[1])
258.
259.             # if trail point is far away of the screen -> do not draw it
260.             if -200 < x < w + 200 and -200 < y < h + 200:
261.                 try:
262.                     pygame.gfxdraw.pixel(screen, x, y, self.color)
263.                 except (OverflowError, ValueError, TypeError):
264.                     points_to_remove.add(point)
265.         self.points -= points_to_remove
266.
```

6.2.2.4 Ui.py

```
 1. import pygame
 2. import pathlib
 3. import sys
 4.
 5. # -- Source files --
 6. current_dir = pathlib.Path(__file__).resolve()
 7. parent_dir  = current_dir.parent.parent
 8.
 9. sys.path.insert(0, str(parent_dir))
10.
11. # Use config_manager
12. from config_manager import get_config_manager
13. cfg = get_config_manager()
14.
15. class EditMenu:
16.
17.     # -- Init --
18.
19.     def __init__(self):
20.         self.width   = 300
21.         self.padding = 20
22.         self.visible = False
23.
24.         self.target_object = None
25.         self.menu_mode     = None   # "settings" | "moving_object"
26.
27.         # Fonts
28.         self.title_font = pygame.font.Font(None, 32)
29.         self.label_font = pygame.font.Font(None, 24)
30.         self.input_font = pygame.font.Font(None, 22)
31.
32.         # Colour palette
33.         self.bg_color = (40,  42,  54)
34.         self.text_color = (248, 248, 242)
35.         self.input_bg = (68,  71,  90)
36.         self.input_active = (98,  114, 164)
```

```
37.         self.border_color = (98,  114, 164)
38.
39.         # Input fields
40.         self.fields = []
41.         self.active_field = None
42.
43.         self.update_position()
44.
45.     # -- Layout --
46.
47.     def update_position(self):
48.         self.x = cfg.SCREEN_WIDTH - self.width
49.         self.y = 0
50.         self.height = cfg.SCREEN_HEIGHT
51.
52.     def get_field_rect(self, field):
53.         return pygame.Rect(
54.             self.x + self.padding,
55.             field['y'] + 25,
56.             self.width - self.padding * 2,
57.             35,
58.         )
59.
60.     # -- Visibility --
61.
62.     def show(self, target_object=None):
63.         from objects import MovingObject
64.
65.         self.target_object = target_object
66.         self.visible = True
67.         self.fields = []
68.         self.active_field = None
69.         self.update_position()
70.
71.         if target_object is None:
72.             self.menu_mode = "settings"
73.             self._setup_settings_fields()
74.         elif isinstance(target_object, MovingObject):
75.             self.menu_mode = "moving_object"
76.             self._setup_moving_object_fields()
77.
78.     def hide(self):
79.         self.visible = False
80.         self.target_object = None
81.         self.active_field  = None
82.
83.     # -- Field builders --
84.
85.     def _setup_settings_fields(self):
86.         y = 100
87.         spacing = 80
88.
89.         # get current collision mode
90.         collision_mode_value = cfg.COLLISION_MODE
91.
92.         for name, label, value in [
93.             ('collision_mode', 'Collision Mode', collision_mode_value),
94.             ('time_scale', 'Time Scale', cfg.TIME_SCALE),
95.             ('collision_elasticity', 'Collision Elasticity', cfg.COLLISION_ELASTICITY),
96.             ('edge_elasticity', 'Edge Elasticity', cfg.EDGE_ELASTICITY),
97.         ]:
98.             self.fields.append({
99.                 'name': name, 'label': label,
100.                 'value': str(value) if not isinstance(value, str) else value,
101.                 'y': y, 'editable': True,
102.             })
103.             y += spacing
104.
105.     def _setup_moving_object_fields(self):
```

```
106.          obj = self.target_object
107.          y = 100
108.          spacing = 80
109.
110.          radius_m = obj.get_width() / (2 * cfg.PIXELS_PER_METER)
111.
112.          for name, label, value in [
113.              ('mass', 'Mass (kg)', obj.get_mass()),
114.              ('radius', 'Radius (m)', radius_m),
115.              ('velocity_x', 'Velocity X (m/s)', obj.get_velocity_x()),
116.              ('velocity_y', 'Velocity Y (m/s)', obj.get_velocity_y()),
117.          ]:
118.              self.fields.append({
119.                  'name': name, 'label': label,
120.                  'value': str(round(value, 3)),
121.                  'y': y, 'editable': True,
122.              })
123.              y += spacing
124.
125.      # -- Event handling --
126.
127.      def handle_event(self, event):
128.          if not self.visible:
129.              return False
130.
131.          if event.type == pygame.MOUSEBUTTONDOWN:
132.              menu_rect = pygame.Rect(self.x, self.y, self.width, self.height)
133.              if not menu_rect.collidepoint(event.pos):
134.                  return False   # click passed through to scene
135.
136.              self.active_field = None
137.              for field in self.fields:
138.                  if field['editable'] and
self.get_field_rect(field).collidepoint(event.pos):
139.                      self.active_field = field
140.                      return True
141.
142.          elif event.type == pygame.KEYDOWN:
143.              if event.key == pygame.K_ESCAPE:
144.                  self.hide()
145.                  return "hide"
146.
147.              if self.active_field:
148.                  if event.key == pygame.K_RETURN:
149.                      self.apply_changes()
150.                      self.active_field = None
151.                      return "apply_changes"
152.                  elif event.key == pygame.K_BACKSPACE:
153.                      self.active_field['value'] = self.active_field['value'][:-1]
154.                  elif self.active_field['name'] == 'collision_mode':
155.                      # Allow 1, 2, D for collision mode (e.g., "1D", "2D")
156.                      if event.unicode in '12Dd':
157.                          self.active_field['value'] += event.unicode.upper()
158.                  else:
159.                      # Normal numeric input for other fields
160.                      if event.unicode in '0123456789.-':
161.                          self.active_field['value'] += event.unicode
162.                  return True
163.
164.          return False
165.
166.      # -- Apply --
167.
168.      def apply_changes(self):
169.          try:
170.              if self.menu_mode == "settings":
171.                  for field in self.fields:
172.
173.                      if field['name'] == 'collision_mode':
```

```
                        value = field['value'].strip().upper()
                        if value in ['1D', '2D']:
                            cfg.COLLISION_MODE = value
                            print(f"Collision mode -> {value}")
                        continue

                    value = float(field['value'])

                    if field['name'] == 'time_scale' and value > 0:
                        cfg.TIME_SCALE = value
                        print(f"Time scale -> {value}")

                    elif field['name'] == 'collision_elasticity' and 0 <= value <= 1:
                        cfg.COLLISION_ELASTICITY = value
                        print(f"Collision elasticity -> {value}")

                    elif field['name'] == 'edge_elasticity' and 0 <= value <= 1:
                        cfg.EDGE_ELASTICITY = value
                        print(f"Edge elasticity -> {value}")

            elif self.menu_mode == "moving_object":
                obj = self.target_object
                for field in self.fields:
                    value = float(field['value'])

                    if field['name'] == 'mass' and value > 0:
                        obj.mass   = value
                        obj.gforce = [0.0, value * cfg.GRAVITY]

                    elif field['name'] == 'radius' and value > 0:
                        diameter_px = value * 2 * cfg.PIXELS_PER_METER
                        obj.radius = value
                        obj.width = diameter_px
                        obj.height = diameter_px
                        obj.rect = obj.image.get_rect(center=obj.get_center())
                        obj.set_click_hitbox()

                    elif field['name'] == 'velocity_x':
                        obj.set_velocity_x(value)

                    elif field['name'] == 'velocity_y':
                        obj.set_velocity_y(value)

            print(f"Changes applied to {self.menu_mode}")

        except ValueError as exc:
            print(f"Error applying changes: {exc}")

    # -- Drawing --

    def draw(self, screen):
        if not self.visible:
            return

        # Panel background
        pygame.draw.rect(screen, self.bg_color,
                         (self.x, self.y, self.width, self.height))
        pygame.draw.line(screen, self.border_color,
                         (self.x, self.y), (self.x, self.height), 3)

        # Title
        titles = {"settings": "Settings", "moving_object": "Edit Moving Object"}
        title_surf = self.title_font.render(
            titles.get(self.menu_mode, "Menu"), True, self.text_color
        )
        screen.blit(title_surf,
                    (self.x + (self.width - title_surf.get_width()) // 2, 30))

        # Input fields
```

```
243.         for field in self.fields:
244.             label_surf = self.label_font.render(field['label'], True, self.text_color)
245.             screen.blit(label_surf, (self.x + self.padding, field['y']))
246.
247.             if not field['editable']:
248.                 continue
249.
250.             field_rect = self.get_field_rect(field)
251.             bg = self.input_active if field is self.active_field else self.input_bg
252.             pygame.draw.rect(screen, bg,               field_rect, border_radius=4)
253.             pygame.draw.rect(screen, self.border_color, field_rect, 2, border_radius=4)
254.
255.             text_surf = self.input_font.render(field['value'], True, self.text_color)
256.             text_x = field_rect.x + 10
257.             text_y = field_rect.y + (field_rect.height - text_surf.get_height()) // 2
258.             screen.blit(text_surf, (text_x, text_y))
259.
260.             if field is self.active_field:
261.                 cursor_x = text_x + text_surf.get_width() + 2
262.                 pygame.draw.line(screen, self.text_color,
263.                                  (cursor_x, field_rect.y + 5),
264.                                  (cursor_x, field_rect.y + field_rect.height - 5), 2)
265.
266.         # Bottom instructions
267.         instructions = [
268.             "Press ENTER to save changes",
269.             "Press ESC to close menu",
270.             "",
271.             "Collision Mode: Type 1D or 2D",
272.             "",
273.             "Controls:",
274.             "RMB Click object - Edit properties",
275.             "RMB Click empty space - Settings",
276.             "F - Pause/Resume",
277.             "R - Reset simulation",
278.             "A - Add object at cursor",
279.             "S - Delete object at cursor",
280.         ]
281.         inst_y = self.height - 300
282.         for i, line in enumerate(instructions):
283.             surf = self.input_font.render(line, True, self.text_color)
284.             screen.blit(surf,
285.                         (self.x + (self.width - surf.get_width()) // 2,
286.                          inst_y + i * 25))
287.
```

6.2.2.5 Vectors.py

```
 1. import math
 2.
 3. def dot_product(v1, v2):
 4.     return v1[0] * v2[0] + v1[1] * v2[1]
 5.
 6. def vector_add(v1, v2):
 7.     return (v1[0] + v2[0], v1[1] + v2[1])
 8.
 9. def vector_subtract(v1, v2):
10.     return (v1[0] - v2[0], v1[1] - v2[1])
11.
12. def scalar_multiply(v, scalar):
13.     return (v[0] * scalar, v[1] * scalar)
14.
15. def vector_length(v):
16.     return math.hypot(v[0], v[1])
17.
18. def normalize_vector(v):
19.     length = vector_length(v)
```

```
    if length == 0:
        return (0, 0)
    return (v[0] / length, v[1] / length)

def distance_between_points(p1, p2):
    return math.hypot(p2[0] - p1[0], p2[1] - p1[1])

```

6.2.3 String simulation

6.2.3.1 Strings.py

```
import math
import pygame
import pathlib
import sys
# -- Source files --
current_dir = pathlib.Path(__file__).resolve()
parent_dir  = current_dir.parent.parent

sys.path.insert(0, str(parent_dir))

# Use config_manager
from config_manager import get_config_manager
cfg = get_config_manager()
import objects

class String:

    def __init__(self, start_x, start_y, elasticity_modulus, natural_length, particle):
        self.start_x             = start_x
        self.start_y             = start_y
        self.elasticity_modulus  = elasticity_modulus
        self.natural_length      = natural_length
        self.particle            = particle

        self.elasticity_coefficient = elasticity_modulus / natural_length

        # logical anchor position in meters
        self.logical_start_x = start_x / cfg.PIXELS_PER_METER
        self.logical_start_y = start_y / cfg.PIXELS_PER_METER

        # camera offset for panning / zooming
        self.camera_offset_x = 0.0
        self.camera_offset_y = 0.0

        # small visible radius for the anchor dot and its click hitbox (10cm)
        self.radius = 0.1  # metres
        self.click_hitbox = self._make_hitbox()

    # -- Hitbox --

    def _make_hitbox(self):
        side = self.radius * cfg.PIXELS_PER_METER * math.sqrt(2)
        return pygame.Rect(
            self.start_x - side / 2,
            self.start_y - side / 2,
            side, side,
        )

    def set_click_hitbox(self):
        self.click_hitbox = self._make_hitbox()

    # -- Position --

    def get_center(self): return (self.start_x, self.start_y)
```

```
55.
56.     def set_center(self, position):
57.         self.start_x, self.start_y = position[0], position[1]
58.         # Account for camera offset when calculating logical position
59.         self.logical_start_x = (self.start_x - self.camera_offset_x) / cfg.PIXELS_PER_METER
60.         self.logical_start_y = (self.start_y - self.camera_offset_y) / cfg.PIXELS_PER_METER
61.         self.click_hitbox = self._make_hitbox()
62.
63.     def sync_logical_to_visual(self):
64.         self.logical_start_x = (self.start_x - self.camera_offset_x) / cfg.PIXELS_PER_METER
65.         self.logical_start_y = (self.start_y - self.camera_offset_y) / cfg.PIXELS_PER_METER
66.
67.     def apply_camera_offset(self, delta_x, delta_y):
68.         self.camera_offset_x += delta_x
69.         self.camera_offset_y += delta_y
70.         # Update visual position from logical + camera offset
71.         self.start_x = self.logical_start_x * cfg.PIXELS_PER_METER + self.camera_offset_x
72.         self.start_y = self.logical_start_y * cfg.PIXELS_PER_METER + self.camera_offset_y
73.         # Update hitbox directly
74.         self.click_hitbox = self._make_hitbox()
75.
76.     def update_visual_position(self):
77.         self.start_x = self.logical_start_x * cfg.PIXELS_PER_METER + self.camera_offset_x
78.         self.start_y = self.logical_start_y * cfg.PIXELS_PER_METER + self.camera_offset_y
79.         self.set_center((self.start_x, self.start_y))
80.
81.     # -- Physics --
82.
83.     def calculate_extension(self):
84.         px, py   = self.particle.get_logical_position()
85.         distance = math.hypot(self.logical_start_x - px, self.logical_start_y - py)
86.         return distance - self.natural_length
87.
88.     def calculate_angle(self):
89.         px, py = self.particle.get_logical_position()
90.         return math.atan2(self.logical_start_y - py, self.logical_start_x - px)
91.
92.     def calculate_tension(self):
93.         extension = self.calculate_extension()
94.         if extension <= 0:
95.             return (0.0, 0.0)
96.
97.         tension = self.elasticity_coefficient * extension
98.         angle   = self.calculate_angle()
99.         return (math.cos(angle) * tension, math.sin(angle) * tension)
100.
101.     def calcualte_elastic_potential_energy(self):
102.         # EPE = ʎx^2 / 2l where ʎ is modulus of elasticity, l is natural length, x is
extension
103.         return
(self.elasticity_modulus*(self.calculate_extension()**2))/(2*self.natural_length)
104.
105.     # -- Drawing --
106.
107.     def draw(self, surface):
108.         color = cfg.COLORS["RED"] if self.calculate_extension() > 0 else cfg.COLORS["BLUE"]
109.         pygame.draw.circle(surface, cfg.COLORS["GREEN"],
110.                            (int(self.start_x), int(self.start_y)),
111.                            int(self.radius * cfg.PIXELS_PER_METER))
112.         pygame.draw.aaline(surface, color,
113.                            (self.start_x, self.start_y),
114.                            self.particle.get_center())
115.
```

6.2.3.2 Main.py

```
# -- Libraries --
import pygame
import pygame.gfxdraw
import sys
import math
import time
import pathlib

# -- Source files --
current_dir = pathlib.Path(__file__).resolve()
parent_dir  = current_dir.parent.parent

sys.path.insert(0, str(parent_dir))

# Use config_manager
from config_manager import get_config_manager
cfg = get_config_manager()

import objects
import mymouse
import strings
import ui

pygame.init()

# -- Close Button --
class CloseButton:

    def __init__(self, x, y, width, height):
        self.rect = pygame.Rect(x, y, width, height)
        self.hovered = False
        self.font = pygame.font.Font(None, 24)

        # -- Colors --
        self.bg_color = [98, 114, 164]
        self.hover_color = [139, 157, 195]
        self.text_color = [248, 248, 242]

    def handle_event(self, event):
        if event.type == pygame.MOUSEMOTION:
            self.hovered = self.rect.collidepoint(event.pos)
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.hovered:
                return True
        return False

    def draw(self, screen):
        color = self.hover_color if self.hovered else self.bg_color
        pygame.draw.rect(screen, color, self.rect, border_radius=6)
        pygame.draw.rect(screen, self.text_color, self.rect, 2, border_radius=6)

        text_surf = self.font.render("Close (Return to Menu)", True, self.text_color)
        text_rect = text_surf.get_rect(center=self.rect.center)
        screen.blit(text_surf, text_rect)

def main():

    # -- Window --
    screen = pygame.display.set_mode((cfg.SCREEN_WIDTH, cfg.SCREEN_HEIGHT))
    pygame.display.set_caption("String Simulation")
    clock  = pygame.time.Clock()

    # -- Scene objects --
    # default moving object
    radius = 0.25
    moving_object = objects.MovingObject(
        radius=radius,
```

```
        color=cfg.COLORS["WHITE"],
        pos_x=cfg.SCREEN_WIDTH  / 2,
        pos_y=cfg.SCREEN_HEIGHT / 2,
        mass=10,
    )

    moving_objects = pygame.sprite.Group()
    moving_objects.add(moving_object)

    # -- Strings --
    # default string 1 meter abouve the moving object
    base_string  = strings.String(
        start_x=cfg.SCREEN_WIDTH  / 2,
        start_y=cfg.SCREEN_HEIGHT / 2 - 1*cfg.PIXELS_PER_METER,
        elasticity_modulus=150,
        natural_length=4,
        particle=moving_object,
    )
    string_objects = [base_string]

    # -- Edit list (objects pickable by right-click or drag) --
    edit_list = [moving_object, base_string]

    # -- Subsystems --
    mouse = mymouse.Mouse()
    edit_menu = ui.EditMenu()
    trail = objects.TrailSystem(max_points=8000)

    # Close button
    close_button = CloseButton(10, 10, 220, 40)

    # -- Simulation state --
    simulation_started = True
    simulation_paused = False

    physics_timestep = 1.0 / cfg.PHYSICS_FPS
    accumulator = 0.0
    last_time = time.perf_counter()

    fps_counter = 0
    fps_timer = time.time()
    current_fps = 0

    # -- Camera state --
    camera_panning = False
    camera_pan_start = (0, 0)

    # -- Main loop --

    run = True
    while run:

        # -- Timing --
        current_time = time.perf_counter()
        dt           = current_time - last_time
        last_time    = current_time

        if simulation_started and not simulation_paused:
            accumulator += dt * cfg.TIME_SCALE
        else:
            accumulator = 0.0

        # -- Events --
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False

            # Close button takes priority
            if close_button.handle_event(event):
```

```
137.                 print("Returning to main menu...")
138.                 run = False
139.                 continue
140.
141.             # Edit menu takes priority
142.             menu_result = edit_menu.handle_event(event)
143.             if edit_menu.menu_mode == "settings" and menu_result == "apply_changes":
144.                 accumulator = 0.0
145.                 continue
146.             if menu_result:
147.                 continue
148.
149.             # -- Camera pan: start / stop --
150.             if event.type == pygame.MOUSEBUTTONDOWN and event.button in (2, 3):
151.                 camera_panning = True
152.                 camera_pan_start = pygame.mouse.get_pos()
153.             if event.type == pygame.MOUSEBUTTONUP and event.button in (2, 3):
154.                 camera_panning = False
155.
156.             # -- Zoom --
157.             if event.type == pygame.MOUSEWHEEL:
158.                 zoom_center_x, zoom_center_y = pygame.mouse.get_pos()
159.                 old_ppm = cfg.PIXELS_PER_METER
160.
161.                 cfg.PIXELS_PER_METER = max(20, min(200, cfg.PIXELS_PER_METER + event.y *
5))
162.                 zoom_ratio = cfg.PIXELS_PER_METER / old_ppm
163.
164.                 # Reposition moving object around zoom center
165.                 # Calculate current visual position
166.                 old_vx = moving_object.logical_x * old_ppm + moving_object.camera_offset_x
167.                 old_vy = moving_object.logical_y * old_ppm + moving_object.camera_offset_y
168.
169.                 # Scale position relative to zoom center
170.                 new_vx = zoom_center_x + (old_vx - zoom_center_x) * zoom_ratio
171.                 new_vy = zoom_center_y + (old_vy - zoom_center_y) * zoom_ratio
172.
173.                 # Update camera offset to maintain logical position
174.                 moving_object.camera_offset_x = new_vx - moving_object.logical_x *
cfg.PIXELS_PER_METER
175.                 moving_object.camera_offset_y = new_vy - moving_object.logical_y *
cfg.PIXELS_PER_METER
176.
177.                 # Update visual position
178.                 moving_object.pos_x = new_vx
179.                 moving_object.pos_y = new_vy
180.                 moving_object.rect.center = (new_vx, new_vy)
181.                 moving_object.click_hitbox = moving_object._make_hitbox()
182.
183.                 # Reposition each string anchor around zoom center
184.                 for s in string_objects:
185.                     old_sx = s.logical_start_x * old_ppm + s.camera_offset_x
186.                     old_sy = s.logical_start_y * old_ppm + s.camera_offset_y
187.
188.                     new_sx = zoom_center_x + (old_sx - zoom_center_x) * zoom_ratio
189.                     new_sy = zoom_center_y + (old_sy - zoom_center_y) * zoom_ratio
190.
191.                     s.camera_offset_x = new_sx - s.logical_start_x * cfg.PIXELS_PER_METER
192.                     s.camera_offset_y = new_sy - s.logical_start_y * cfg.PIXELS_PER_METER
193.
194.                     s.start_x = new_sx
195.                     s.start_y = new_sy
196.                     s.click_hitbox = s._make_hitbox()
197.
198.                 trail.scale_points((zoom_center_x, zoom_center_y), zoom_ratio)
199.
200.             # -- Right-click -> open edit menu --
201.             if event.type == pygame.MOUSEBUTTONDOWN and event.button == 3:
202.                 mouse_pos      = pygame.mouse.get_pos()
```

```
203.                 object_clicked = False
204.                 for obj in edit_list:
205.                     if obj.click_hitbox.collidepoint(mouse_pos):
206.                         edit_menu.show(obj)
207.                         object_clicked = True
208.                         break
209.                 if not object_clicked:
210.                     edit_menu.show()
211.
212.             # -- Keyboard --
213.             if event.type == pygame.KEYDOWN:
214.
215.                 # -- Exit simulation --
216.
217.                 if event.key == pygame.K_ESCAPE:
218.                     edit_menu.hide()
219.
220.                 # -- Pause simulation --
221.
222.                 if event.key == pygame.K_f:
223.                     simulation_paused = not simulation_paused
224.                     simulation_started = not simulation_started
225.
226.                 # -- Reset simulation --
227.
228.                 if event.key == pygame.K_r:
229.                     # Reset to one object + one string, move camera to be on top of the
moving object
230.                     moving_object.reset_speed()
231.                     trail.clear()
232.
233.                     # center camera on the moving object
234.                     screen_center_x = cfg.SCREEN_WIDTH / 2
235.                     screen_center_y = cfg.SCREEN_HEIGHT / 2
236.                     # calculate current visual position of object
237.                     current_visual_x = moving_object.pos_x
238.                     current_visual_y = moving_object.pos_y
239.                     # calculate offset needed to center object on screen
240.                     dx = screen_center_x - current_visual_x
241.                     dy = screen_center_y - current_visual_y
242.                     # apply offset to moving object
243.                     moving_object.apply_camera_offset(dx, dy)
244.                     # apply offset to all strings
245.                     for s in string_objects:
246.                         s.apply_camera_offset(dx, dy)
247.                     # apply offset to trail
248.                     trail.translate_points(dx, dy)
249.
250.                 # -- Add string --
251.
252.                 if event.key == pygame.K_a:
253.
254.                     if len(string_objects) >= 3:
255.                         print("Cannot add more than 3 string objects")
256.                         continue
257.
258.                     mx, my = mouse.get_mouse_pos()
259.                     new_string = strings.String(
260.                         start_x=mx, start_y=my,
261.                         elasticity_modulus=150, natural_length=4,
262.                         particle=moving_object,
263.                     )
264.
265.                     new_string.camera_offset_x = moving_object.camera_offset_x
266.                     new_string.camera_offset_y = moving_object.camera_offset_y
267.                     # recalculate logical position with proper camera offset
268.                     new_string.logical_start_x = (mx - new_string.camera_offset_x) /
cfg.PIXELS_PER_METER
```

```
                    new_string.logical_start_y = (my - new_string.camera_offset_y) /
cfg.PIXELS_PER_METER
                    # update hitbox with new position
                    new_string.click_hitbox = new_string._make_hitbox()

                    string_objects.append(new_string)
                    edit_list.append(new_string)
                    print(f"Added string at ({mx}, {my})")

                # -- Delete string --

                if event.key == pygame.K_s:

                    mouse_pos = mouse.get_mouse_pos()
                    string_to_delete = None
                    for s in string_objects:
                        if s.click_hitbox.collidepoint(mouse_pos):
                            string_to_delete = s
                            break

                    if string_to_delete and len(string_objects) > 1:
                        string_objects.remove(string_to_delete)
                        edit_list.remove(string_to_delete)
                        print(f"Deleted string")

                    elif len(string_objects) <= 1:
                        print("Cannot delete the last string object")
                        continue

        # -- Camera panning (every frame while held) --
        if camera_panning and mouse.draggable_object is None:
            cur = pygame.mouse.get_pos()
            dx  = cur[0] - camera_pan_start[0]
            dy  = cur[1] - camera_pan_start[1]

            moving_object.apply_camera_offset(dx, dy)
            for s in string_objects:
                s.apply_camera_offset(dx, dy)
            trail.translate_points(dx, dy)

            camera_pan_start = cur

        # -- Drag-and-drop --
        if mouse.is_lmb_clicked() and simulation_paused:
            mouse_pos = mouse.get_mouse_pos()

            if mouse.draggable_object is None:
                for obj in edit_list:
                    if obj.click_hitbox.collidepoint(mouse_pos):
                        mouse.draggable_object = obj
                        break
            else:
                mouse.draggable_object.set_center(mouse_pos)

        elif not mouse.is_lmb_clicked() and simulation_paused:
            if mouse.draggable_object is not None:
                mouse.draggable_object.sync_logical_to_visual()
            mouse.draggable_object = None

        # -- Physics --
        physics_steps = 0
        while accumulator >= physics_timestep and physics_steps < 10:
            if simulation_started and not simulation_paused:
                forces = [s.calculate_tension() for s in string_objects]
                forces.append(moving_object.get_gforce())
                moving_object.resolve_forces(*forces)
                moving_object.update_physics(physics_timestep)
            accumulator   -= physics_timestep
            physics_steps += 1
```

```
337.
338.         # -- Trail --
339.         if simulation_started and not simulation_paused:
340.             trail.add_point(moving_object.get_center())
341.
342.         # -- Rendering --
343.         screen.fill(cfg.COLORS["WHITE"])
344.
345.         trail.draw(screen)
346.
347.         for s in string_objects:
348.             s.draw(screen)
349.
350.         moving_object.draw(screen)
351.
352.         # Click hitboxes (debug)
353.         for obj in edit_list:
354.             pygame.draw.rect(screen, cfg.COLORS["BLACK"], obj.click_hitbox, 1)
355.
356.         # Velocity arrow
357.         center = moving_object.get_center()
358.         vel_x, vel_y = moving_object.get_velocity_x(), moving_object.get_velocity_y()
359.         pygame.draw.circle(screen, (0, 0, 255), (int(center[0]), int(center[1])), 5)
360.         pygame.draw.line(screen, (0, 0, 255),
361.                          center,
362.                          (center[0] + vel_x * 10, center[1] + vel_y * 10), 2)
363.
364.         # Edit menu
365.         edit_menu.draw(screen)
366.
367.         # Close button
368.         close_button.draw(screen)
369.
370.         # -- HUD --
371.         font = pygame.font.Font(None, 24)
372.         sim_state = "RUNNING" if simulation_started and not simulation_paused else "PAUSED"
373.         hud_lines = [
374.             f"Graphics FPS: {current_fps}/{cfg.FPS}",
375.             f"Physics FPS: {cfg.PHYSICS_FPS}",
376.             f"Velocity X: {vel_x:.2f} m/s",
377.             f"Velocity Y: {vel_y:.2f} m/s",
378.             f"Acceleration X: {moving_object.get_acceleration_x():.2f} m/s²",
379.             f"Acceleration Y: {moving_object.get_acceleration_y():.2f} m/s²",
380.             f"Mass: {moving_object.get_mass()} kg",
381.             f"Simulation: {sim_state}",
382.             #f"Trail points: {trail.length()}",
383.             f"Distance travelled: {moving_object.distance_travelled:.3f} m",
384.
385.         ]
386.         hud_lines.append("")
387.         for s in string_objects:
388.             t    = s.calculate_tension()
389.             hud_lines.append(f"Tension: {math.hypot(t[0], t[1]):.3f} N") # takes magnitude
of tension
390.             hud_lines.append(f"Extension: {(s.calculate_extension()):.3f} m")
391.             hud_lines.append(f"EPE: {(s.calcualte_elastic_potential_energy()):.3f} J")
392.             hud_lines.append("")
393.
394.         for i, line in enumerate(hud_lines):
395.             surf = font.render(line, True, cfg.COLORS["GREEN"])
396.             screen.blit(surf, (10, 60 + i * 25))
397.
398.         # -- FPS counter --
399.         fps_counter += 1
400.         if time.time() - fps_timer >= 1.0:
401.             current_fps = fps_counter
402.             fps_counter = 0
403.             fps_timer   = time.time()
404.
```

```
405.         pygame.display.flip()
406.         clock.tick(cfg.FPS)
407.
408.     pygame.quit()
409.
410. if __name__ == "__main__":
411.     main()
412.
```

6.2.3.3 Objects.py

```
  1. import pygame
  2. import pygame.gfxdraw
  3. import pathlib
  4. import sys
  5. # -- Source files --
  6. current_dir = pathlib.Path(__file__).resolve()
  7. parent_dir  = current_dir.parent.parent
  8.
  9. sys.path.insert(0, str(parent_dir))
 10.
 11. # Use config_manager
 12. from config_manager import get_config_manager
 13. cfg = get_config_manager()
 14. import math
 15.
 16. class Entity(pygame.sprite.Sprite):
 17.
 18.     def __init__(self, height, width):
 19.         pygame.sprite.Sprite.__init__(self)
 20.         self.height = height * cfg.PIXELS_PER_METER
 21.         self.width = width * cfg.PIXELS_PER_METER
 22.
 23.     def get_height(self): return self.height
 24.     def get_width(self):  return self.width
 25.
 26. class MovingObject(Entity):
 27.     def __init__(self, radius, color, pos_x, pos_y, mass):
 28.         diameter = radius * 2
 29.         super().__init__(diameter, diameter)
 30.
 31.         self.mass = mass
 32.         self.radius = radius
 33.         self.color = color
 34.
 35.         self.velocity_x = 0.0
 36.         self.velocity_y = 0.0
 37.         self.acceleration_x = 0.0
 38.         self.acceleration_y = 0.0
 39.
 40.         self.gforce = [0.0, mass * cfg.GRAVITY]
 41.
 42.         self.logical_x = pos_x / cfg.PIXELS_PER_METER
 43.         self.logical_y = pos_y / cfg.PIXELS_PER_METER
 44.         self.pos_x = float(pos_x)
 45.         self.pos_y = float(pos_y)
 46.
 47.         self.camera_offset_x = 0.0
 48.         self.camera_offset_y = 0.0
 49.
 50.         self.distance_travelled = 0.0
 51.
 52.         # Transparent sprite surface so the object can live in a Group if needed
 53.         self.image = pygame.Surface((int(self.width), int(self.height)), pygame.SRCALPHA)
 54.         self.image.fill((0, 0, 0, 0))
 55.         self.rect = self.image.get_rect(center=(pos_x, pos_y))
 56.         self.click_hitbox = self._make_hitbox()
```

```
57.
58.     # -- Hitbox --
59.
60.     def _make_hitbox(self):
61.         r_px = self.radius * cfg.PIXELS_PER_METER
62.         side = r_px * math.sqrt(2) # largest square inscribed in circle
63.         return pygame.Rect(
64.             self.pos_x - side / 2,
65.             self.pos_y - side / 2,
66.             side, side,
67.         )
68.
69.     def set_click_hitbox(self):
70.         self.click_hitbox = self._make_hitbox()
71.
72.     # -- Position --
73.
74.     def set_center(self, position):
75.         self.pos_x, self.pos_y = float(position[0]), float(position[1])
76.         self.rect.center = (self.pos_x, self.pos_y)
77.         # Account for camera offset when calculating logical position
78.         self.logical_x = (self.pos_x - self.camera_offset_x) / cfg.PIXELS_PER_METER
79.         self.logical_y = (self.pos_y - self.camera_offset_y) / cfg.PIXELS_PER_METER
80.         self.click_hitbox = self._make_hitbox()
81.
82.     def get_logical_position(self): return (self.logical_x, self.logical_y)
83.
84.     def sync_logical_to_visual(self):
85.         self.logical_x = (self.pos_x - self.camera_offset_x) / cfg.PIXELS_PER_METER
86.         self.logical_y = (self.pos_y - self.camera_offset_y) / cfg.PIXELS_PER_METER
87.
88.     def apply_camera_offset(self, delta_x, delta_y):
89.         self.camera_offset_x += delta_x
90.         self.camera_offset_y += delta_y
91.         # Update visual position from logical + camera offset
92.         self.pos_x = self.logical_x * cfg.PIXELS_PER_METER + self.camera_offset_x
93.         self.pos_y = self.logical_y * cfg.PIXELS_PER_METER + self.camera_offset_y
94.         # Update rect and hitbox directly
95.         self.rect.center  = (self.pos_x, self.pos_y)
96.         self.click_hitbox = self._make_hitbox()
97.
98.     # -- Physics --
99.
100.    def resolve_forces(self, *forces):
101.        fx, fy = 0.0, 0.0
102.        for f in forces:
103.            fx += f[0]
104.            fy += f[1]
105.        self.acceleration_x = fx / self.mass
106.        self.acceleration_y = fy / self.mass
107.
108.    def update_physics(self, dt):
109.        # -- Simple physics --
110.
111.        # previous position to calculate change
112.        prev_x, prev_y = self.logical_x, self.logical_y
113.
114.        self.velocity_x += self.acceleration_x * dt
115.        self.velocity_y += self.acceleration_y * dt
116.
117.        self.logical_x += (self.velocity_x) * dt
118.        self.logical_y += (self.velocity_y) * dt
119.
120.        self.pos_x = self.logical_x * cfg.PIXELS_PER_METER + self.camera_offset_x # +
camera offset to maintain consistency of the physics
121.        self.pos_y = self.logical_y * cfg.PIXELS_PER_METER + self.camera_offset_y # +
camera offset to maintain consistency of the physics
122.
123.        dx = self.logical_x - prev_x # meters
```

```
124.         dy = self.logical_y - prev_y # meters
125.
126.         self.distance_travelled += math.sqrt(dx * dx + dy * dy) # meters
127.
128.         self.set_center((self.pos_x, self.pos_y))
129.
130.     def reset_speed(self):
131.         self.velocity_x = 0.0
132.         self.velocity_y = 0.0
133.         self.acceleration_x = 0.0
134.         self.acceleration_y = 0.0
135.
136.     # -- Getters / setters --
137.
138.     def get_center(self): return (self.pos_x, self.pos_y)
139.     def get_mass(self): return self.mass
140.     def get_velocity_x(self): return self.velocity_x
141.     def get_velocity_y(self): return self.velocity_y
142.     def get_acceleration_x(self): return self.acceleration_x
143.     def get_acceleration_y(self): return self.acceleration_y
144.     def set_velocity_x(self, v): self.velocity_x = v
145.     def set_velocity_y(self, v): self.velocity_y = v
146.     def set_acceleration_x(self, a): self.acceleration_x = a
147.     def set_acceleration_y(self, a): self.acceleration_y = a
148.     def get_gforce(self): return self.gforce
149.
150.     # -- Drawing --
151.
152.     def draw(self, screen):
153.         r_px = int(self.radius * cfg.PIXELS_PER_METER)
154.         cx = int(self.pos_x)
155.         cy = int(self.pos_y)
156.         pygame.draw.circle(screen, cfg.COLORS["BLACK"], (cx, cy), r_px, 3)
157.         pygame.draw.circle(screen, cfg.COLORS["BLACK"], (cx, cy), r_px * 0.75, 3)
158.
159. class TrailSystem:
160.     def __init__(self, max_points=5000):
161.         self.points = set()
162.         self.max_points = max_points
163.         self.color = cfg.COLORS["RED"]
164.
165.     # -- Mutation --
166.
167.     def add_point(self, position):
168.         self.points.add((int(position[0]), int(position[1])))
169.         if len(self.points) > self.max_points:
170.             self.points.pop()
171.
172.     def clear(self):
173.         self.points.clear()
174.
175.     # -- Transforms (for camera pan / zoom) --
176.
177.     def translate_points(self, delta_x, delta_y):
178.         self.points = {(x + delta_x, y + delta_y) for x, y in self.points}
179.
180.     def scale_points(self, zoom_center, zoom_ratio):
181.         cx, cy = zoom_center
182.         self.points = {
183.             (cx + (x - cx) * zoom_ratio, cy + (y - cy) * zoom_ratio)
184.             for x, y in self.points
185.         }
186.
187.     # -- Drawing --
188.
189.     def draw(self, screen):
190.         w, h = screen.get_width(), screen.get_height()
191.         points_to_remove = set()
192.         for point in self.points:
```

```
193.             x, y = int(point[0]), int(point[1])
194.             if -200 < x < w + 200 and -200 < y < h + 200:
195.                 try:
196.                     pygame.gfxdraw.pixel(screen, x, y, self.color)
197.                 except (OverflowError, ValueError, TypeError):
198.                     points_to_remove.add(point)
199.         self.points -= points_to_remove
200.
201.     def length(self): return len(self.points)
202.
```

6.2.3.4 Ui.py

```
 1. import pygame
 2. import math
 3. import pathlib
 4. import sys
 5. # -- Source files --
 6. current_dir = pathlib.Path(__file__).resolve()
 7. parent_dir  = current_dir.parent.parent
 8.
 9. sys.path.insert(0, str(parent_dir))
10.
11. # Use config_manager
12. from config_manager import get_config_manager
13. cfg = get_config_manager()
14.
15. class EditMenu:
16.
17.     # -- Init --
18.
19.     def __init__(self):
20.         self.width = 300
21.         self.padding = 20
22.         self.visible = False
23.
24.         self.target_object = None
25.         self.menu_mode = None   # "settings" | "moving_object" | "string"
26.
27.         # Fonts
28.         self.title_font = pygame.font.Font(None, 32)
29.         self.label_font = pygame.font.Font(None, 24)
30.         self.input_font = pygame.font.Font(None, 22)
31.
32.         # Colour palette
33.         self.bg_color = (40,  42,  54)
34.         self.text_color = (248, 248, 242)
35.         self.input_bg = (68,  71,  90)
36.         self.input_active = (98,  114, 164)
37.         self.border_color = (98,  114, 164)
38.
39.         # Input fields
40.         self.fields = []
41.         self.active_field = None
42.
43.         self.update_position()
44.
45.     # -- Layout --
46.
47.     def update_position(self):
48.         self.x = cfg.SCREEN_WIDTH - self.width
49.         self.y = 0
```

```
50.         self.height = cfg.SCREEN_HEIGHT
51.
52.     def get_field_rect(self, field):
53.         return pygame.Rect(
54.             self.x + self.padding,
55.             field['y'] + 25,
56.             self.width - self.padding * 2,
57.             35,
58.         )
59.
60.     # -- Visibility --
61.
62.     def show(self, target_object=None):
63.         from objects import MovingObject
64.         from strings import String
65.
66.         self.target_object = target_object
67.         self.visible = True
68.         self.fields = []
69.         self.active_field = None
70.         self.update_position()
71.
72.         if target_object is None:
73.             self.menu_mode = "settings"
74.             self._setup_settings_fields()
75.         elif isinstance(target_object, MovingObject):
76.             self.menu_mode = "moving_object"
77.             self._setup_moving_object_fields()
78.         elif isinstance(target_object, String):
79.             self.menu_mode = "string"
80.             self._setup_string_fields()
81.
82.     def hide(self):
83.         self.visible = False
84.         self.target_object = None
85.         self.active_field = None
86.
87.     # -- Field builders --
88.
89.     def _setup_settings_fields(self):
90.         self.fields.append({
91.             'name': 'time_scale',
92.             'label': 'Time Scale',
93.             'value': str(round(cfg.TIME_SCALE, 3)),
94.             'y': 100,
95.             'editable': True,
96.         })
97.
98.     def _setup_moving_object_fields(self):
99.         obj = self.target_object
100.        y = 100
101.        spacing = 80
102.
103.        for name, label, value in [
104.            ('mass', 'Mass (kg)', obj.get_mass()),
105.            ('radius', 'Radius (m)', obj.radius),
106.            ('velocity_x', 'Velocity X (m/s)', obj.get_velocity_x()),
107.            ('velocity_y', 'Velocity Y (m/s)', obj.get_velocity_y()),
108.        ]:
109.            self.fields.append({
110.                'name': name, 'label': label,
111.                'value': str(round(value, 3)),
112.                'y': y, 'editable': True,
113.            })
114.            y += spacing
115.
116.    def _setup_string_fields(self):
117.        obj     = self.target_object
118.        y       = 100
```

```
119.          spacing = 80
120.
121.          for name, label, value in [
122.              ('natural_length', 'Natural Length (m)', obj.natural_length),
123.              ('elasticity_modulus',  'Elasticity (N/m)',   obj.elasticity_modulus),
124.          ]:
125.              self.fields.append({
126.                  'name': name, 'label': label,
127.                  'value': str(round(value, 3)),
128.                  'y': y, 'editable': True,
129.              })
130.              y += spacing
131.
132.      # -- Event handling --
133.
134.      def handle_event(self, event):
135.          if not self.visible:
136.              return False
137.
138.          if event.type == pygame.MOUSEBUTTONDOWN:
139.              self.active_field = None
140.              for field in self.fields:
141.                  if field['editable'] and
self.get_field_rect(field).collidepoint(event.pos):
142.                      self.active_field = field
143.                      return True
144.
145.          elif event.type == pygame.KEYDOWN:
146.              if event.key == pygame.K_ESCAPE:
147.                  self.hide()
148.                  return "hide"
149.
150.              if self.active_field:
151.                  if event.key == pygame.K_RETURN:
152.                      self.apply_changes()
153.                      self.active_field = None
154.                      return "apply_changes"
155.                  elif event.key == pygame.K_BACKSPACE:
156.                      self.active_field['value'] = self.active_field['value'][:-1]
157.                  elif event.unicode in '0123456789.-':
158.                      self.active_field['value'] += event.unicode
159.                  return True
160.
161.          return False
162.
163.      # -- Apply --
164.
165.      def apply_changes(self):
166.          try:
167.              if self.menu_mode == "settings":
168.                  for field in self.fields:
169.                      value = float(field['value'])
170.                      if field['name'] == 'time_scale' and value > 0:
171.                          cfg.TIME_SCALE = value
172.                          print(f"Time scale -> {value}")
173.
174.              elif self.menu_mode == "moving_object":
175.                  obj = self.target_object
176.                  for field in self.fields:
177.                      value = float(field['value'])
178.
179.                      if field['name'] == 'mass' and value > 0:
180.                          obj.mass = value
181.                          obj.gforce = [0.0, value * cfg.GRAVITY]
182.
183.                      elif field['name'] == 'radius' and value > 0:
184.                          obj.radius = value
185.                          diameter_px = value * 2 * cfg.PIXELS_PER_METER
186.                          obj.diameter = diameter_px
```

```
187.                         obj.height = diameter_px
188.                         obj.width = diameter_px
189.                         obj.rect = obj.image.get_rect(center=obj.get_center())
190.                         obj.set_click_hitbox()
191.
192.                     elif field['name'] == 'velocity_x':
193.                         obj.set_velocity_x(value)
194.
195.                     elif field['name'] == 'velocity_y':
196.                         obj.set_velocity_y(value)
197.
198.             elif self.menu_mode == "string":
199.                 obj = self.target_object
200.                 for field in self.fields:
201.                     value = float(field['value'])
202.
203.                     if field['name'] == 'natural_length' and value > 0:
204.                         obj.natural_length = value
205.                         obj.elasticity_coefficient = obj.elasticity_modulus / value
206.
207.                     elif field['name'] == 'elasticity_modulus' and value > 0:
208.                         obj.elasticity_modulus = value
209.                         obj.elasticity_coefficient = value / obj.natural_length
210.
211.             print(f"Changes applied to {self.menu_mode}")
212.
213.         except ValueError as exc:
214.             print(f"Error applying changes: {exc}")
215.
216.     # -- Drawing --
217.
218.     def draw(self, screen):
219.         if not self.visible:
220.             return
221.
222.         # Panel background
223.         pygame.draw.rect(screen, self.bg_color,
224.                          (self.x, self.y, self.width, self.height))
225.         pygame.draw.line(screen, self.border_color,
226.                          (self.x, self.y), (self.x, self.height), 3)
227.
228.         # Title
229.         titles = {
230.             "settings": "Settings",
231.             "moving_object": "Edit Moving Object",
232.             "string": "Edit String",
233.         }
234.         title_surf = self.title_font.render(
235.             titles.get(self.menu_mode, "Menu"), True, self.text_color
236.         )
237.         screen.blit(title_surf,
238.                     (self.x + (self.width - title_surf.get_width()) // 2, 30))
239.
240.         # Input fields
241.         for field in self.fields:
242.             label_surf = self.label_font.render(field['label'], True, self.text_color)
243.             screen.blit(label_surf, (self.x + self.padding, field['y']))
244.
245.             if not field['editable']:
246.                 continue
247.
248.             field_rect = self.get_field_rect(field)
249.             bg = self.input_active if field is self.active_field else self.input_bg
250.             pygame.draw.rect(screen, bg, field_rect, border_radius=4)
251.             pygame.draw.rect(screen, self.border_color, field_rect, 2, border_radius=4)
252.
253.             text_surf = self.input_font.render(field['value'], True, self.text_color)
254.             text_x = field_rect.x + 10
255.             text_y = field_rect.y + (field_rect.height - text_surf.get_height()) // 2
```

```
256.             screen.blit(text_surf, (text_x, text_y))
257.
258.             # Blinking cursor for active field
259.             if field is self.active_field:
260.                 cursor_x = text_x + text_surf.get_width() + 2
261.                 pygame.draw.line(screen, self.text_color,
262.                                  (cursor_x, field_rect.y + 5),
263.                                  (cursor_x, field_rect.y + field_rect.height - 5), 2)
264.
265.         # Bottom instructions
266.         instructions = [
267.             "Press ENTER to save changes",
268.             "Press ESC to close menu",
269.             "",
270.             "Controls:",
271.             "RMB Click object - Edit properties",
272.             "RMB Click empty space - Settings",
273.             "F - Pause/Resume",
274.             "R - Reset simulation",
275.             "A - Add string at cursor",
276.             "S - Delete string at cursor"
277.         ]
278.         inst_y = self.height - 300
279.         for i, line in enumerate(instructions):
280.             surf = self.input_font.render(line, True, self.text_color)
281.             screen.blit(surf,
282.                         (self.x + (self.width - surf.get_width()) // 2,
283.                          inst_y + i * 25))
284.
```

6.2.4 Config

6.2.4.1 Config.json

```
1. {
2.   "display": {
3.     "fps": 120,
4.     "physics_fps": 48
5.   },
6.   "physics": {
7.     "gravity": 9.81,
8.     "pixels_per_meter": 100,
9.     "time_scale": 1.0
10.  },
11.  "colors": {
12.    "WHITE": [
13.      255,
14.      255,
15.      255
16.    ],
17.    "BLACK": [
18.      0,
19.      0,
20.      0
21.    ],
22.    "RED": [
23.      255,
24.      0,
25.      0
26.    ],
27.    "GREEN": [
28.      0,
29.      255,
30.      0
31.    ],
32.    "BLUE": [
```

```
      0,
      0,
      255
    ],
    "GRAY": [
      128,
      128,
      128
    ],
    "DARK_GRAY": [
      64,
      64,
      64
    ]
  }
}

```

7. References

1. Cloudfront.net. (2025). Available at: https://d3rw207pwvlq3a.cloudfront.net/attachments/000/029/641/original/Capture20.PNG?1540014303 [Accessed 13 Oct. 2025].

2. Study.com. (2025). Available at: https://homework.study.com/cimages/multimages/16/capture_248641776826009790839.png [Accessed 13 Oct. 2025].

3. Ward, S. (2017). *Level further mathematics for aqa student book 1 (as/year 1).* Cambridge Univ Press.

4. Bostock, L. and Chandler, S. (1996). *Mechanics for A-level*. Nelson Thornes.